

UNIVERSITÀ DI PADOVA

DIPARTIMENTO MATEMATICA

University of Padua
Department of Mathematics

Architectural Trade-offs in Zero-Knowledge Proof Decomposition

A Traveling-Salesman Case Study

Master's Degree in Cybersecurity

Candidate

Leonardo Callegari

Student ID

2159085

Supervisor

Prof. Mauro Conti
University of Padua

External Supervisor

Prof. Martin Ochoa
zkSecurity

Graduation Date
September 25, 2026

Abstract

Zero-knowledge proofs let a prover convince a verifier that a statement is true without revealing anything beyond that fact. This thesis studies succinct, non-interactive zero-knowledge arguments (zk-SNARKs) through the Traveling Salesman Problem (TSP). For a complete graph of n nodes, the prover claims knowledge of a Hamiltonian cycle costing at most a public threshold while keeping the route and its exact cost private.

A monolithic circuit expresses this statement in one proving task and produces a single succinct proof object. The task grows with the graph size n, until its memory exceeds the available memory on fixed hardware.

We develop a refined monolithic construction that serves as a baseline, and then explore composite constructions. In those, we divide the route into K segments, prove them in smaller circuits, and recombine their local claims into the original global statement about the entire route. We implement every construction in Noir, using UltraHonk as the proof system through the Barretenberg backend engine [1].

Composite constructions recombine segment proofs either externally or recursively, resulting in different proof topologies and architectural trade-offs. In external recombination, a separate glue proof establishes the global checks, and the verifier binds its claims to those of the segment proofs. In recursive recombination, the same binding is performed internally on the prover side, where an outer circuit verifies the segment proofs, enforces the global checks, and combines their claims into a single proof object. Both approaches introduce a recombination surface for reconstructing the global claim from the segments; we study this surface and refine the constructions to keep the route and its cost private.

At $n = 4000$ on the consumer host, monolithic proving reaches a peak of 12.95 GiB. External recombination reduces this peak to 8.8 GiB when node coverage remains in the glue circuit, and to 1.6 GiB at $K = 8$ when compact computational coverage summaries are used. Recursive recombination reaches its lowest peak, 4.03 GiB, at $K = 4$, after which the outer circuit becomes the limiting task. All composite architectures produce proofs at $n = 5000$, beyond the largest successful monolithic checkpoint. None reduces sequential proving time on the same host, and lower estimated completion times require additional workers to exploit parallelism.

Acknowledgements

To my parents, thank you for your love, for believing in me, and for supporting me throughout my studies. I am deeply grateful to have you beside me and to share this moment with you.

Mauro Conti, thank you for your time, for introducing me to Martin Ochoa, and for your enthusiasm about working on this project. Martin, thank you for your constant guidance and feedback, and for the time and care you have given to this work. I am grateful to you both for your patience throughout, and for the opportunity to work with you.

To my friends and colleagues, thank you for sharing these years with me, for your support, and for the company that has made them so much more enjoyable.

Finally, I would like to acknowledge the use of AI tools to assist with proofreading, formatting, and text polishing, as well as with parts of the experimental harness and implementation code. This included assistance with Noir syntax, dependency configuration, and code for plot generation and benchmarking.

Contents

Abstract i

Acknowledgements iii

Contents v

List of Figures ix

List of Tables xi

List of Listings xiii

1 Introduction 1

1.1 Motivation . . . 1
1.2 Problem Definition . . . 4
1.2.1 Mathematical Formulation . . . 4
1.2.2 Computational Bottlenecks . . . 4
1.3 Research Questions . . . 5
1.3.1 The Monolithic Baseline . . . 5
1.3.2 The Advantages of Decomposition . . . 6
1.3.3 Recombination and Structural Privacy . . . 6
1.4 Contributions . . . 7
1.5 Outline . . . 7

2 Related Work 9

2.1 Generality and Verifiable Computation . . . 9
2.2 TSP as a Verification Benchmark . . . 10
2.3 Scalable Proving Architectures . . . 10
2.3.1 Backend-Level Distribution . . . 11

2.3.2 Circuit-Level Decomposition 11
2.4 Proof Composition 12
2.4.1 Recursion and Folding 12
2.4.2 Aggregation and Commitment Linking 12
2.5 Graph and Path Specialization 13
2.6 Layers of Decomposition 13

3 Background 15
3.1 Zero-Knowledge Arguments 15
3.1.1 Statements and Witnesses 15
3.1.2 Security Properties 16
3.1.3 Knowledge Soundness 17
3.2 SNARKs and Arithmetic Circuits 17
3.2.1 Arithmetic Circuits 18
3.2.2 The Proving Stack 19
3.2.3 Memory Inside a Circuit 20
3.2.4 Recursive Verification 20
3.3 Cryptographic Primitives 21
3.3.1 Cryptographic Hash Functions 21
3.3.2 Commitment Schemes 22
3.3.3 Merkle Trees 23
3.4 Randomized Algebraic Checks 24
3.4.1 Polynomial Identity Testing 24
3.4.2 Non-Interactive Challenges 25
3.5 Security Model and Trust Boundaries 25
3.5.1 Adversarial Model 25
3.5.2 Security Layers 26
3.5.3 Trust Assumptions and Scope 26

4 Monolithic Proofs 27
4.1 Naive Construction 27
4.1.1 Inputs 27
4.1.2 Constraints 28
4.1.3 Circuit Specification 29
4.2 Permutation Mechanisms 32
4.2.1 Indicator Vector 32
4.2.2 Inverse Permutation Map 33
4.2.3 Sorted Route 34

4.2.4 Mechanism Evaluation . . . 35
4.2.5 Mechanism Selection . . . 38
4.3 Committed Cost Matrix . . . 39
4.4 Refined Circuit Construction . . . 42

5 Composite Proofs 45
5.1 Proof Decomposition . . . 45
5.1.1 Route Decomposition . . . 45
5.1.2 Local Proofs and Global Obligations . . . 46
5.1.3 Recombination Contract . . . 47
5.1.4 External Composition . . . 49
5.1.5 Stitching Tax . . . 50
5.2 Plain Constructions . . . 52
5.2.1 Plain Sort . . . 52
5.2.2 Product Coverage . . . 56
5.2.3 Binding the Challenge to the Route . . . 57
5.2.4 Plain Product . . . 58
5.2.5 Plain Construction Trade-offs . . . 62
5.3 Committed Constructions . . . 63
5.3.1 Commitment Layer . . . 64
5.3.2 Committed Sort . . . 66
5.3.3 Committed Product . . . 68
5.3.4 Committed Construction Trade-offs . . . 71
5.4 Recursive Construction . . . 72
5.4.1 Inner Circuit Selection . . . 72
5.4.2 Outer Circuit . . . 74
5.4.3 Proof Verification . . . 76
5.4.4 Recursive Construction Trade-offs . . . 77
5.5 The Architectural Frontier . . . 78

6 Evaluation 81
6.1 Methodology . . . 81
6.1.1 Experimental Setup . . . 81
6.1.2 Measurement Protocol . . . 83
6.2 Correctness Testing . . . 84
6.3 Monolithic Baseline . . . 86
6.3.1 Circuit Growth . . . 86
6.3.2 Prover Resources and Host Boundary . . . 87

6.4 Composite Gate Accounting 88
6.4.1 Aggregate Gate Work 88
6.4.2 Distribution Across Proving Tasks 91
6.5 Peak Prover Memory 94
6.5.1 End-to-End Memory Scaling 95
6.5.2 Component Bottlenecks 97
6.6 End-to-End Prover Time 99
6.6.1 Sequential and Distributed Scaling 100
6.6.2 Schedule Bottlenecks 102
6.7 Verifier Load 106
6.7.1 Proof Package and Public Surface 106
6.7.2 Backend Verification Time 107

7 Discussion 109
7.1 Research Findings 109
7.1.1 Monolithic Design and Behavior 109
7.1.2 Advantages of Decomposition 110
7.1.3 Recombination and Structural Privacy 111
7.2 Beyond TSP Proofs 112
7.3 Limitations 113
7.4 Future Work 114
7.5 Conclusions 115

References 117

List of Figures

Figure 3.1: Noir-to-UltraHonk proving pipeline. 19
Figure 3.2: Merkle leaf v_5 authentication path. 23
Figure 4.1: Circuit size across permutation mechanisms. 36
Figure 4.2: Witness-generation time across permutation mechanisms. 37
Figure 4.3: Cost scaling across monolithic refinements. 43
Figure 4.4: Witness and verification time across refinements. 44
Figure 5.1: External composition with public summaries. 50
Figure 5.2: External composition with committed summaries. 65
Figure 5.3: Recursive composition with certified inputs. 74
Figure 6.1: Monolithic circuit-size scaling. 86
Figure 6.2: Monolithic prover-cost scaling. 87
Figure 6.3: Aggregate gate overhead of composite constructions. 89
Figure 6.4: Component gate distribution at $n = 3000$. 93
Figure 6.5: Recombiner gate costs at $n = 3000$. 93
Figure 6.6: End-to-end proving-memory scaling. 95
Figure 6.7: Component proving-memory profiles at $n = 4000$. 97
Figure 6.8: Sequential prover-time scaling. 100
Figure 6.9: Distributed prover-time scaling. 102
Figure 6.10: Prover-time schedule composition at $n = 4000$. 102
Figure 6.11: Backend verification time across the evaluation grid. 108

List of Tables

Table 4.1: Asymptotic costs of permutation mechanisms. 35
Table 5.1: Local and global obligations after decomposition. 47
Table 5.2: Components of the stitching tax. 51
Table 5.3: Plain-sort summary fields and recombination roles. 52
Table 5.4: Trade-offs between the plain constructions. 62
Table 5.5: Committed summaries by construction. 66
Table 5.6: Architectural frontier of composite constructions. 79
Table 6.1: Evaluation hardware and software. 82
Table 6.2: Targeted adversarial correctness tests. 85
Table 6.3: Aggregate gate ratios to monolithic Merkle-sort. 91
Table 6.4: Component and aggregate gates at $n = 3000$. 94
Table 6.5: Component and peak proving memory at $n = 4000$. 99
Table 6.6: Sequential and distributed prover time at $n = 4000$. 105
Table 6.7: Final-verifier interface accounting. 106
Table 7.1: Architectural profiles of the evaluated constructions. 111

List of Listings

Listing 4.1: Naive monolithic circuit. 29
Listing 4.2: Sort-based permutation check. 34
Listing 4.3: Authenticated committed-matrix read. 40
Listing 4.4: Refined monolithic Merkle-sort circuit. 42
Listing 5.1: Plain-sort segment circuit. 53
Listing 5.2: Plain-sort recombination circuit. 55
Listing 5.3: Plain-product segment circuit. 59
Listing 5.4: Plain-product recombination circuit. 60
Listing 5.5: Committed-sort segment circuit. 66
Listing 5.6: Committed-sort glue circuit. 67
Listing 5.7: Committed-product segment circuit. 69
Listing 5.8: Committed-product glue circuit. 70
Listing 5.9: Recursive outer circuit. 75

1 Introduction

This chapter introduces zero-knowledge proofs through an interactive example. We then state the problem, research questions, and contributions of this thesis.

1.1 Motivation

Imagine a ring-shaped cave whose entrance branches into passages A and B with a locked door connecting them at the far end. Alice knows the secret word that opens it and wants to prove this to Bob without telling him the word [2]. Bob stays outside the cave while Alice goes inside, and he cannot see which branch she takes. Only then does he randomly call A or B and ask her to return that way. If Alice knows the word, she can cross through the hidden door when necessary and always comply. Without it, she is trapped on the side she chose and succeeds only when Bob happens to name that side, with probability $1/2$.

A single successful round is therefore weak evidence: an impostor may simply have guessed Bob's request correctly. Repeating the protocol makes a false acceptance increasingly unlikely. In each round, Alice *commits* by entering before she knows the request, Bob issues a random *challenge* by requesting one of the two sides, and Alice *responds* by emerging from the requested side. Because each challenge follows the commitment and determines the response, the evidence develops through a live, interactive exchange. To succeed, an impostor must guess every challenge, so after r independent rounds the chance of passing all of them falls to 2^{-r}.

This example showcases the original form of a zero-knowledge proof: an *interactive protocol* [3] between two parties. Alice plays the prover, while Bob plays the verifier. The example also introduces the three crucial properties such a proof must have. An honest prover always succeeds (*completeness*); a prover without the secret succeeds only with negligible probability after enough rounds (*soundness*); and the verifier learns nothing beyond the prover's knowledge of the secret (*zero-knowledge*). We formally define all three properties in Section 3.1.

Zero-knowledge proofs now support applications across several domains. Zcash's shielded transactions use them to hide the payment's origin, destination, and amount while allowing the network to verify that the transaction preserves value [4]. Identity systems can use zero-knowledge proofs to let people prove selected attributes, such as meeting an age threshold, without disclosing additional personal information [5], [6]. These systems build on earlier work on data-minimizing wallets and anonymous credentials [7], [8], [9]. For general computation, zero-knowledge virtual machines (zkVMs) let a prover certify the execution of ordinary code while hiding private inputs or execution data. RISC Zero, SP1, and Jolt are concrete systems built around this model [10], [11], [12].

These diverse applications require a general-purpose proof system that can handle any efficiently checkable statement, without keeping prover and verifier online at the same time, while producing small proofs that remain fast to verify as the computation grows. These properties are *generality*, *non-interactivity*, and *succinctness*.

Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs) provide these guarantees [13]. The prover demonstrates knowledge of a private *witness* that makes a public statement true, while a fixed arithmetic circuit encodes the computation that checks that statement [14]. The statement supplies public inputs, the witness supplies private values, and the circuit constrains them through addition and multiplication gates; the application changes this circuit, whereas the proving machinery remains the same. Because its gates and wires are fixed before proving, the computation must be finite: loops must be unrolled to a fixed bound or represented by a

bounded execution trace, so no finite circuit can certify the completion of a non-terminating computation [14], [15].

Strictly, a SNARK is an *argument*, not a proof [1] [16]: its soundness holds only against computationally bounded provers, under hardness assumptions rather than against an unbounded adversary. Practical SNARK soundness therefore rests on assumptions that certain problems are infeasible for a realistic adversary. For example, recovering x from g and g^x in a suitable cryptographic group is assumed infeasible (the discrete-logarithm problem) [17]. Integer factorization provides another example of a computational hardness assumption [18].

This thesis fixes one zk-SNARK backend and compares proof architectures through the Traveling Salesman Problem (TSP), a well-known problem in the literature [19], [20]. On a complete weighted graph, the optimization problem asks for the cheapest Hamiltonian cycle, whereas the decision version asks whether a cycle of cost at most a threshold T exists. The optimization problem is NP-hard and the decision problem NP-complete, both in the worst case [19]. In the decisional TSP, finding a suitable route can be difficult, but once a candidate is available, a verifier can check it in polynomial time by confirming that every node appears once, summing the arc costs, and comparing the result with T.

This makes decisional TSP a good fit for a zero-knowledge proof: a route is easy to check even when it's hard to find, so there is value in keeping a route private while being able to prove properties about it to another party. In this work, we assume the prover already holds a route below a public threshold, and the starting point is proving a route satisfies a cost threshold while keeping the route and the exact cost private.

This separation between a public bound and a private route models settings in which compliance conflicts with secrecy. For example, a logistics or security firm could prove that a planned delivery or transit route meets a public time limit without revealing proprietary routing choices, GPS traces, timing windows, or stop durations.

[1] Cryptographic writing often still calls arguments *proofs* when the distinction is not central.

1.2 Problem Definition

We now formally express the statement in the context of the TSP problem as a relation, specifying public problem data and a private route.

1.2.1 Mathematical Formulation

We use a complete directed graph $G = (\mathcal{N}, \mathcal{A})$ with $n = |\mathcal{N}|$ nodes. The node set is $\mathcal{N} = \{0, ..., n-1\}$, and the arc set is $\mathcal{A} = \{(i, j) : i, j \in \mathcal{N}, i \neq j\}$, since the graph is assumed to be complete. An $n \times n$ matrix C stores the nonnegative integer arc costs, with $c(i, j)$ denoting the cost from node i to node j.

A *valid* route is a Hamiltonian cycle: it visits every node once and returns to its start. We represent it by the ordering $\pi = (\pi_0, \pi_1, ..., \pi_{n-1})$. The return to the first node, after visiting π_{n-1}, is not included in the sequence but is accounted for when computing the total cost of the route. In other words, the last arc is implicit and we adopt the convention $\pi_n = \pi_0$. Thus π is a permutation of $\mathcal{N}$, a property enforced by the circuits in later chapters.

The cost of such a route is the sum of the costs of its n arcs,

$$\text{cost}(\pi) = \sum_{i=0}^{n-1} c(\pi_i, \pi_{i+1}) \,. \tag{1.1}$$

The problem fixes a public threshold T, and we call a route *feasible* when $\text{cost}(\pi) \leq T$.

We can now formalize the exact proposition we want to prove. The public data consists of the cost matrix C and the threshold T. The private witness is the route π. The conditions for a valid and feasible route are captured by the relation

$$R = \{\langle (C, T), \pi \rangle : \pi \text{ is a permutation of } \mathcal{N} \text{ and } \text{cost}(\pi) \leq T\}. \tag{1.2}$$

The prover claims knowledge of a witness π for which $\langle (C, T), \pi \rangle \in R$, revealing neither the route nor its exact cost.

1.2.2 Computational Bottlenecks

Succinctness keeps a proof with a constant-size public interface small and quick to verify even as the statement grows. It does not remove the prover's

work: compilation, witness generation, proving time, and memory still grow with the instance size n. Of these, memory imposes a hard limit on fixed hardware: a proving task remains feasible, however long it takes, only while it fits in the available capacity; once it does not, proof generation fails.

Chapter 4 expresses relation (1.2) in one circuit whose size and memory requirement grow with n. A fixed memory budget therefore sets a ceiling that hardware, circuit design, and the proving backend can move but cannot remove: one prover must still hold the complete route-level computation. This motivates the composite constructions of Chapter 5.

1.3 Research Questions

> *Leading Research Question*
> How do monolithic and composite zero-knowledge proof architectures behave when proving a globally constrained statement, and what advantages and trade-offs arise from decomposing and recombining the proof?

Three specific questions separate the design of the monolithic baseline, the possible advantages of decomposition, and the problem of private recombination.

1.3.1 The Monolithic Baseline

A meaningful and credible comparison begins with the design of the monolithic construction. A deliberately naive circuit could make decomposition appear beneficial for reasons unrelated to the architecture itself. We treat refinement as part of the study: compare plausible constraint mechanisms, remove dominant avoidable costs, and characterize the resulting construction as the instance grows. The study seeks a well-engineered, refined baseline within the fixed proving stack.

> *Research Question* (1)
> How should the monolithic proof be designed to provide a credible baseline for comparison, and how does the resulting architecture behave as the instance size grows?

1.3.2 The Advantages of Decomposition

A composite construction cuts the route into K segments and proves them separately before recovering the claim at the global-route level. Because each segment is roughly n/K in size, its circuit is structurally smaller and it could allow smaller proving tasks and independent work to be scheduled in parallel, or extend the range of instances that can be handled. Recombination may preserve, reduce, or outweigh those advantages, depending on how the global conditions are restored.

> *Research Question* (2)
> What advantages can decomposition provide over the refined monolithic proof, and how do segmentation and recombination choices determine whether those advantages survive end to end?

1.3.3 Recombination and Structural Privacy

Decomposition removes the global view that lets one circuit enforce node coverage, connection, and total cost directly. Recombination must recover those properties from evidence about the separate segments. The form and placement of that evidence determine both the work of recombination and what reaches the final verifier.

Structural privacy captures what the verifier learns about the route partition. A public recombination interface may disclose segment contents, endpoints, or partial costs. Commitments can hide those values while retaining external proof checks, whereas recursion can move the complete interface into an outer witness. These choices redistribute work between prover and verifier and change the topology of the final proof.

> *Research Question* (3)
> How can segment proofs be recombined into the original global claim while preserving structural privacy, and what trade-offs among prover cost, verifier work, and disclosure arise from different recombination strategies?

1.4 Contributions

This thesis makes three contributions to the design of zk-SNARK architectures for globally constrained statements, using the TSP relation as a benchmark.

- *Refined monolithic baseline.* We compare four permutation mechanisms, select sorting as the canonical exact mechanism, and replace the quadratic public cost-matrix interface with a Merkle root. The resulting circuit provides a comparison-ready monolithic baseline for relation (1.2).
- *Recombination architectures and costs.* Starting from a route that has already been found, we divide its verification into segment proofs and specify the recombination contract required to restore global coverage, continuity, cost binding, and threshold satisfaction. We construct external, commitment-protected, and recursive variants, and measure their *stitching tax*: the work required to turn locally certified segments into the original global claim. The constructions expose trade-offs among peak prover memory, scheduling, verifier work, proof topology, and structural privacy.
- *Controlled architectural comparison.* We evaluate constrained work, peak prover memory, completion schedules, and verifier load under one proving stack. Direct cost comparisons hold coverage and structural privacy fixed, separating architectural effects from savings caused by weaker guarantees or greater disclosure.

1.5 Outline

The remainder of the thesis follows three stages. Chapters 2 and 3 position the problem in related work and establish the cryptographic and technical foundations. Chapters 4 and 5 develop the monolithic baseline and composite constructions. Chapters 6 and 7 evaluate these architectures and interpret their trade-offs.

2 Related Work

This chapter positions the thesis within the work on general-purpose zero knowledge, scalable proving, proof composition, and graph-specific verification. The organizing question is where a large proof computation is divided. A backend may distribute the algebra for one circuit; a generic system may split a circuit or execution into composed pieces; an application may expose smaller semantic statements; or a prover may reduce memory without distributing the computation.

2.1 Generality and Verifiable Computation

Goldwasser, Micali, and Rackoff formalized zero knowledge and its simulation-based privacy requirement [3]. Goldreich, Micali, and Wigderson then showed, assuming one-way functions, that every language in NP has a zero-knowledge proof system [21]. Relation (1.2) belongs to NP because a verifier can check a candidate route, its costs, and the threshold in polynomial time. This establishes existence; efficient implementation remains a separate problem.

Pinocchio and *SNARKs for C* show how general-purpose proving works in practice. A statement is expressed as a computation, compiled into algebraic constraints, and then proved by a reusable backend. Pinocchio translated arithmetic circuits into quadratic arithmetic programs and used circuit-specific preprocessing to obtain short proofs with fast verification [14]. *SNARKs for C* extended the same pipeline to programs with data-dependent

control flow and memory by compiling C to TinyRAM and proving a valid execution trace [15].

This thesis fixes both the constraint language and the proof backend. The TSP relation is written directly in Noir, and every construction uses UltraHonk. Later chapters can therefore compare how the same route checks (permutation, authenticated cost lookups, and the threshold) are encoded in one circuit or divided and recombined across several proofs. Holding the stack fixed isolates the effect of those architectural choices.

2.2 TSP as a Verification Benchmark

Blum used knowledge of a Hamiltonian cycle as a canonical statement [22], while Igari et al. recently constructed physical protocols for Hamiltonian cycle and TSP [23]. Those works are relevant, but they use problem-specific interactive protocols, rather than the succinct circuit-proof setting studied here.

The closest early circuit benchmark is the rectilinear TSP experiment in *SNARKs for C* [15]. Its witness is a low-weight Hamiltonian path beginning at a public source in a fixed coordinate graph. This thesis proves a Hamiltonian cycle over an agreed weighted matrix, including the closing arc. The common core is a private ordering constrained by a public cost bound; the relation and representation differ.

The same paper proposed *computing in blocks* to reduce prover space. Blocking divides backend work while preserving the application relation, circuit, public interface, and final proof. Here, route segmentation divides the relation itself. Each segment loses the global view needed to enforce coverage, cyclic continuity, and the total cost, so recombination becomes part of the application design.

2.3 Scalable Proving Architectures

Scalable proving systems move different resource boundaries. The distinction relevant here is whether work distribution remains hidden below one relation or creates separately certified pieces with an explicit interface.

2.3.1 Backend-Level Distribution

DIZK distributes the state and dominant algebraic operations of one R1CS prover across a cluster [24]. Pianist applies a similar external model to PLONK: workers process circuit portions and a coordinator combines their contributions into one proof [25]. Other systems refine nearby points: deVirgo distributes the prover used by zkBridge, Soloist targets distributed R1CS proving with constant proof size, and Cirrus adds accountability for identifying a misbehaving worker [26], [27], [28]. HyperFond extends this with transparent, post-quantum distributed proving [29]. The protocols differ, but the application still presents one circuit and the verifier still receives one claim.

2.3.2 Circuit-Level Decomposition

HEKATON divides an arbitrary circuit into subcircuits, proves them separately, and aggregates the proofs [30]. A global-memory abstraction enforces agreement on wires crossing the cut. This establishes the generic decompose–prove–aggregate pattern and shows that horizontal scaling need not expose several proofs to the final verifier.

Here, the cut follows semantic boundaries. Consecutive route segments become separate statements, so the lost global conditions are route properties: Hamiltonian coverage, cross-segment continuity, matrix-bound costs, and one total threshold. They have to be restored from the segment proofs in a recombination step that also ensures consistency of all the pieces.

Scribe streams prover intermediate states through disk while retaining one circuit [31]; Sparrow gives space-efficient proving for data-parallel circuits [32]. These systems address the same resource pressure through a different proof system or storage model, without introducing an application-level recombination interface.

Collaborative zk-SNARKs divide something else again: mutually distrustful parties jointly prove a statement over witness shares that no single party owns [33]. This thesis assumes one logical owner of the complete route and divides proof work, not witness ownership.

2.4 Proof Composition

Valid local proofs do not establish the relation among them. Composition must certify both local validity and cross-piece consistency through recursion, folding, aggregation, commitment linking, or a combination of these mechanisms.

2.4.1 Recursion and Folding

Recursive composition verifies earlier proofs inside a new proof. Proof-carrying data generalizes this pattern to distributed computations whose outputs carry proofs of their histories [34]. Later work studied complexity-preserving recursion and applied PCD to MapReduce-style computation [35], [36]. In every case, privacy depends on which values the composed statement exposes.

Halo demonstrated recursive composition without a trusted setup [37]. Nova replaced repeated in-circuit verification with folding, which accumulates instances before a final check [38]. Mangrove combines folding with parallel and low-memory proving [39]. These are generic composition frameworks. The recursive construction in Section 5.4 is a narrower, bounded two-level design: one outer circuit verifies K segment proofs and restores the route-wide relation.

2.4.2 Aggregation and Commitment Linking

Aggregation compresses the verification of several proofs. SnarkPack, for example, aggregates Groth16 proofs into a compact object [40], [41]. Aggregation alone here would not establish common inputs, connected endpoints, complete coverage, or a total threshold; those remain application relations among the certified statements.

Commit-and-prove systems bind separate relations to hidden shared values. LegoSNARK provides a general framework for composing proofs around committed data [42]. The committed constructions of Section 5.3 use the same interface idea without claiming to instantiate that framework: segment summaries remain hidden behind blinded digests, while a recombination proof checks their openings. Recursion moves both proof verification

and reconciliation into the outer circuit; external recombination leaves more work and structure at the verifier interface.

2.5 Graph and Path Specialization

Graph-aware systems exploit structure that a generic compiler would otherwise treat as arbitrary computation.

LaPathZK is the closest recent path-specific system [43]. It uses a distributed lattice-based protocol to prove that a hidden path satisfies a public cost threshold. This thesis proves a different relation: an ordered Hamiltonian cycle must cover every vertex exactly once, connect segment boundaries and the closing arc, and bind each cost to the committed matrix. LaPathZK designs a new distributed proof system; this work fixes Noir and UltraHonk and compares public, commitment-protected, and recursive recombination interfaces.

2.6 Layers of Decomposition

The literature therefore supplies three adjacent foundations. Backend distribution and low-memory systems retain one application relation while changing how its proof is computed or stored. Generic decomposition, recursion, and folding split compiled computation and restore consistency through shared state or proof composition. Graph-specific systems tailor constraints or protocols to graph and path structure.

This thesis provides a controlled architectural comparison at the semantic-relation layer. It implements the same Hamiltonian-cycle claim as a refined monolithic proof and as several segmented proofs under one Noir and UltraHonk stack. The segmentation makes route-wide coverage, continuity, matrix binding, and threshold enforcement explicit recombination duties.

3 Background

This chapter supplies the concepts needed to read the constructions that follow. It gives a compact formal account of zero-knowledge arguments, arithmetic circuits, the proving stack, cryptographic commitments, authenticated data, and randomized algebraic checks. The final section states the security model that connects those tools to the claims made in Chapters 4 and 5.

3.1 Zero-Knowledge Arguments

Chapter 1 introduced zero-knowledge through an interactive protocol. Its formal definitions specify what the prover claims, what the verifier learns, and what acceptance implies. The later constructions are non-interactive arguments of knowledge, but use the same underlying relation between a public statement and a private witness.

3.1.1 Statements and Witnesses

A relation R connects public *statements* $x \in \mathcal{X}$ with private *witnesses* $w \in \mathcal{W}$:

$$R \subseteq \mathcal{X} \times \mathcal{W}. \tag{3.1}$$

We write $R(x, w) = 1$ when w certifies x. The corresponding language $\mathcal{L}_R$ contains every statement with at least one valid witness, and it is defined as follows:

$$\mathcal{L}_R = \{x \in \mathcal{X} : \exists w \in \mathcal{W}, R(x, w) = 1\}, \tag{3.2}$$

Checking (x, w) must be efficient; finding a suitable w need not be.

For relation (1.2), $x = (C, T)$ contains the agreed cost matrix and threshold, while the witness $w = \pi$ is the route. Later circuits replace the matrix entries on the proof interface with a Merkle root. That is a representation change. The semantic claim remains that π is valid and feasible for the agreed C and T.

In the interactive form, the prover P knows (x, w) and exchanges messages with a verifier V that receives x. Their ordered messages form a transcript, security is parameterized by λ, and $\mathrm{negl}(\lambda)$ denotes a negligible function. Formal treatments of these conventions and the definitions below are given by [3] and [16].

3.1.2 Security Properties

Completeness requires an honest prover with a valid witness to be accepted. For every (x, w) such that $R(x, w) = 1$,

$$\Pr[\langle P(w), V\rangle(x) = 1] \geq 1 - \mathrm{negl}(\lambda), \tag{3.3}$$

where $\langle P(w), V\rangle(x)$ denotes their interaction on common input x, with the prover having additional knowledge of w.

For every false statement, *soundness* bounds the success of a cheating prover $P^\star$:

$$\Pr[\langle P^\star, V\rangle(x) = 1] \leq \mathrm{negl}(\lambda). \tag{3.4}$$

The constructions in this thesis are *arguments*: soundness is guaranteed against efficient provers under computational assumptions, rather than against an unbounded prover [16].

Zero-knowledge requires that every efficient verifier strategy $V^\star$ have an efficient simulator S that can reproduce its view without the witness:

$$\mathrm{View}_{V^\star}^{P(w)}(x) \approx S_{V^\star}(x). \tag{3.5}$$

Here, $\mathrm{View}_{V^\star}^{P(w)}(x)$ includes the transcript, randomness, and internal state available to $V^\star$, while $\approx$ denotes indistinguishability. Whatever the verifier obtains from the real interaction could therefore have been produced from the public statement alone [3].

In *perfect* zero-knowledge the two distributions are identical. In *statistical* zero-knowledge their distance is negligible even to an unbounded observer. In *computational* zero-knowledge only an efficient observer is unable to distinguish them. This classification concerns disclosure and is independent of whether soundness is that of a proof or an argument.

Because the simulator receives x, zero-knowledge does not hide the public statement. If a composite construction publishes endpoints, node sets, or partial costs, those values are disclosed by its interface even when the backend is zero-knowledge. This boundary between public statement and private witness determines the structural privacy studied in Section 1.3.3.

3.1.3 Knowledge Soundness

Soundness rules out false statements; *knowledge soundness* additionally ties acceptance to possession of a witness. If a strategy $P^\star$ convinces the verifier with sufficient probability, an efficient extractor using $P^\star$ as a subroutine must obtain

$$\mathrm{Ext}^{P^\star}(x) \to w \quad \text{such that} \quad R(x, w) = 1. \tag{3.6}$$

The extractor is part of the definition, not the verification procedure. Precise formulations account for its running time, its access to the prover, and a possible knowledge error [44]. This thesis relies on the knowledge-soundness guarantee of the backend rather than constructing an extractor for each circuit.

The backend establishes knowledge of an assignment satisfying a circuit; the construction must show that the circuit expresses the intended relation. Chapters 4 and 5 therefore justify the permutation, cost-binding, threshold, and composition constraints rather than treating backend soundness as a proof of correct encoding.

3.2 SNARKs and Arithmetic Circuits

The relation and its inputs follow two paths. The static path arithmetizes R into constraints and compiles them into backend gates. The input-dependent path generates an assignment from (x, w). Proving joins both paths to

produce a proof. This section follows that flow, then introduces memory and recursion because both shape the constructions evaluated later.

3.2.1 Arithmetic Circuits

An arithmetic circuit is a fixed collection of algebraic constraints over a finite field. Addition and multiplication fit this definition directly; Boolean logic, comparisons, bounded integers, and array access must be *arithmetized* into field constraints [14]. Arithmetization employs a constraint system to turn a computation into a set of equations. A Rank-1 Constraint System, R1CS for short, is a widely used constraint system in this context. To understand the intuition behind it, consider the following example.

Let public y and private u satisfy $y = u \cdot (u + 1) \cdot (u + 2)$. If we want to express this using an R1CS, we first need to perform flattening, since one R1CS constraint multiplies only two linear forms. We thus flatten by using the auxiliary value $t = u \cdot (u + 1)$.

Each flattened constraint has the R1CS form [40]

$$\langle \boldsymbol{a}, \boldsymbol{z} \rangle \cdot \langle \boldsymbol{b}, \boldsymbol{z} \rangle - \langle \boldsymbol{c}, \boldsymbol{z} \rangle = 0. \tag{3.7}$$

For the example assignment $\boldsymbol{z} = (1, y, u, t)$, where the leading 1 supplies constants, the coefficient vectors are

constraint	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$
$u \cdot (u + 1) - t = 0$	$[0, 0, 1, 0]$	$[1, 0, 1, 0]$	$[0, 0, 0, 1]$
$t \cdot (u + 2) - y = 0$	$[0, 0, 0, 1]$	$[2, 0, 1, 0]$	$[0, 1, 0, 0]$

The rows select $(u, u + 1, t)$ and $(t, u + 2, y)$, reproducing the flattened claim. The constraints are fixed by the relation, while $\boldsymbol{z}$ supplies the values for one public statement and private witness.

Source languages hide much of this expansion. A comparison or array access may become range checks, auxiliary values, lookups, or memory constraints, so source-level operation counts do not determine backend gate counts.

Once the constraints are fixed, the prover must construct a complete assignment. Route finding occurs first and supplies w; *witness generation*

derives intermediate values and may compute other auxiliary values off circuit as *hints*. Constraints must check every property of a hint that affects acceptance.

3.2.2 The Proving Stack

The arithmetic circuit and witness generation now provide fixed constraints and a complete assignment. The concrete stack compiles those constraints into backend gates, then a SNARK combines the gates and assignment into a proof [13]. Figure 3.1 shows where the two paths meet.

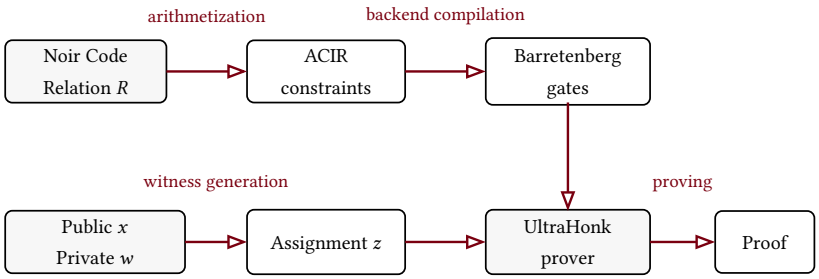


Figure 3.1: From relation to proof: static compilation and input-dependent witness generation join at the UltraHonk prover.

Noir arithmetizes the source relation into the Abstract Circuit Intermediate Representation (ACIR) and completes the witness [45]. Barretenberg compiles ACIR into UltraHonk gates, then proves or verifies the circuit [46]. The verifier receives the proof and public inputs, not the private assignment or represented computation.

UltraHonk belongs to the Plonkish family originating with PLONK and uses a universal structured reference string [47]. A new ceremony is not required for every circuit, but the system is not setup-free. The associated assumption is stated in Section 3.5.3.

This staging explains the evaluation metrics. ACIR opcodes have nonuniform gate expansions, while gate count describes the backend circuit but omits witness construction. The evaluation therefore measures opcodes, gates, witness-generation time, proving time, verification time, proof size, and peak memory separately.

Keeping the stack fixed isolates alternative arithmetizations and public-private interfaces rather than competing SNARK protocols. Succinctness also does not remove public-input processing, so interface size affects privacy and performance. Memory makes the gap between source operations and backend work especially visible.

3.2.3 Memory Inside a Circuit

An ordinary program can choose an array address at runtime; a fixed circuit cannot change its wiring. A constant index connects a predetermined wire. If the witness determines the address of some memory access, the circuit must instead prove that the claimed address and value are consistent with the encoded memory. Matrix access through a private route has this form.

The prover performs such accesses during witness generation, records their addresses and values, and proves a consistency condition over the resulting log. This is an *offline memory check* [48].

Read-only memory binds each claimed value to immutable contents at its claimed address. Read-write memory must additionally respect access order: a read returns the latest preceding write to that address.

Lookup arguments can prove table membership efficiently [49], but memory consistency must also bind the value to its address and, for updates, its place in the access order. Witness-selected access therefore creates constraint and witness work that fixed wiring avoids.

The permutation and matrix-access mechanisms compared in Chapter 4 differ along this fixed-versus-witness-selected boundary.

3.2.4 Recursive Verification

The preceding pipeline produces one proof for one circuit. Composite constructions may instead need one circuit to consume proofs produced by other circuits. Proof verification is itself a fixed computation, so an *outer* circuit can receive an *inner* proof and its public inputs, verify it, and prove that verification accepted. This is *recursive verification* or *proof recursion* [37].

Inner public values may become private inputs of the outer circuit. The final verifier then sees only the outer statement, while the outer prover pays for in-circuit verification and a dependent proving stage.

The inner verification key must be fixed or constrained by the outer circuit; otherwise the prover could substitute a proof for another relation. The final verifier must then authenticate the outer key, which identifies the circuit and its embedded inner key. Chapter 5 instantiates this chain.

3.3 Cryptographic Primitives

The later circuits need short public values that remain linked to larger or private data. Hash functions compress encoded values into digests. Commitments add an opening relation and can also hide the committed value. Merkle trees extend hash-based binding to an ordered collection and allow individual positions to be opened.

3.3.1 Cryptographic Hash Functions

A hash function maps an encoded input message to a fixed-length digest:

$$H : \mathcal{D} \to \mathcal{Y}, \quad m \in \mathcal{D} \mapsto d = H(m) \in \mathcal{Y}. \tag{3.8}$$

The constructions use two standard properties of cryptographic hash functions [50]:

- *Preimage resistance:* given d, an efficient adversary cannot find m such that $H(m) = d$.
- *Collision resistance:* an efficient adversary cannot find $u \neq v$ such that $H(u) = H(v)$.

Neither property makes a deterministic hash hiding. If m comes from a small or predictable set, an observer can hash each candidate and compare the result with d.

A *random oracle* is an idealized function that assigns an independently sampled output to each new input and returns the same output on repeated queries [51]. The later analyses use this model for blinded hash commitments and Fiat–Shamir challenges. The implementation instead evaluates the

concrete Poseidon2 hash, whose field-native operations are designed for efficient arithmetization in Plonkish systems [52]. Treating it as a random oracle is a modeling assumption.

3.3.2 Commitment Schemes

A commitment publishes a short handle D for a message m and later verifies an opening:

$$D \leftarrow \mathrm{Commit}(m; r), \quad \mathrm{Open}(D, m, r) = 1, \tag{3.9}$$

where r denotes fresh randomness. Its two security properties are independent [53]:

- *Binding:* an efficient committer cannot produce accepted openings (m, r) and (m', r') for the same D with $m \neq m'$.
- *Hiding:* an efficient observer cannot distinguish which candidate message produced D.

The blinded hash commitments used later take the form

$$D = H(\mathrm{encode}(m, r)). \tag{3.10}$$

Collision resistance gives computational binding. With fresh, secret r drawn from a sufficiently large space, the construction is computationally hiding in the random-oracle model. Reusing r can reveal equality or other relationships [54].

A commitment is not encryption: the verifier or a later circuit cannot recover m from D. To use m inside another proof, the prover supplies (m, r) privately and the circuit checks the opening. Other commitment families offer different properties and interfaces: Pedersen commitments are perfectly hiding and computationally binding, while KZG commitments provide compact polynomial-evaluation openings under additional algebraic and setup assumptions [55], [56].

The thesis uses commitments in two roles. Chapter 4 needs only a binding representative of the public cost matrix, which the verifier can recompute. Chapter 5 commits to private segment summaries, so those commitments must also hide their openings from the final verifier.

3.3.3 Merkle Trees

A *Merkle tree* applies a hash recursively to bind an ordered collection [57]. For example, let $\boldsymbol{v} = (v_0, ..., v_i, ..., v_7)$. Starting from leaves $u_i^0 = v_i$, it computes each higher level as

$$u_j^{\ell+1} = H\left(u_{2j}^{\ell}, u_{2j+1}^{\ell}\right), \tag{3.11}$$

until one *Merkle root* remains.

To open a leaf at a given index, an authentication path supplies one sibling and one left-or-right direction per level, on the path from that leaf to the root. Padding N leaves to the next power of two gives depth $\lceil \log_2 N \rceil$ and a path from any leaf to the root with a length of $\mathcal{O}(\log N)$.

The verifier follows the directions, combines the opened leaf with each sibling, and reconstructs the root in $\mathcal{O}(\log N)$ hashes. Figure 3.2 shows the opening of leaf v_5 in an eight-leaf tree.

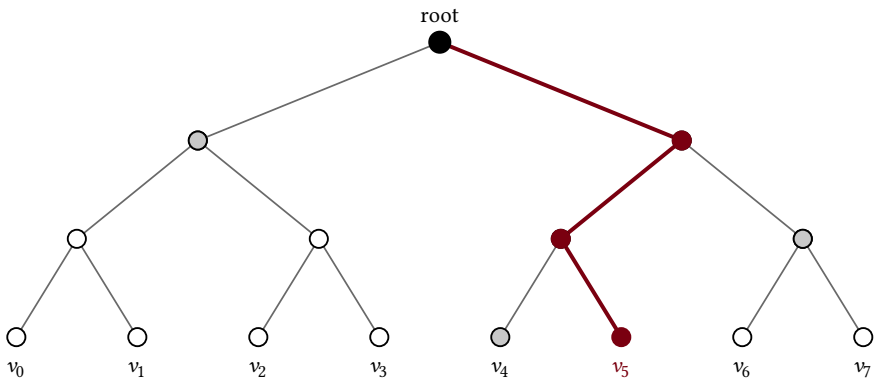


Figure 3.2: Authentication path for leaf v_5 in an eight-leaf Merkle tree. The dark path is recomputed from the opened leaf; the gray nodes are the three sibling hashes supplied with the opening. Their left or right positions bind the opening to index 5.

Reconstructing the expected root proves that the leaf is consistent with the tree along the supplied path. To authenticate a required position, the verifier or circuit must also check that the directions encode the required index; otherwise the prover could open a convenient value elsewhere in the tree.

Under collision resistance, the root computationally binds the ordered leaves once their encoding, count, and padding convention are fixed. It

therefore acts as a binding commitment to the collection. Hiding requires an additional mechanism, since predictable candidate collections can be reconstructed and compared with the root, while authentication paths reveal further hashes.

A valid path proves consistency with a root, so the application must authenticate that root. For public data, the verifier can recompute it. Private data instead requires an authenticated reference, such as a published root or a signature over the root and its instance metadata.

3.4 Randomized Algebraic Checks

The composite constructions compress large equality checks into polynomial evaluations. This introduces a soundness error whose bound depends on the polynomial degree, field size, and order in which the claim and challenge are fixed.

3.4.1 Polynomial Identity Testing

Let p and q be distinct univariate polynomials over a finite field $\mathbb{F}$, with $\deg(p-q) \le d$. For uniform $X \in \mathbb{F}$,

$$\Pr[p(X) = q(X)] \le \frac{\deg(p-q)}{|\mathbb{F}|} \le \frac{d}{|\mathbb{F}|}. \tag{3.12}$$

This is the univariate Schwartz–Zippel bound used here [58], [59]. A genuine identity always passes; a false one passes only when X is one of at most d roots of $p-q$.

When $d \ll |\mathbb{F}|$, one evaluation is a compact fingerprint of a much larger equality. Constructing that fingerprint may still process every element, so the interface shrinks without necessarily removing the underlying work. Chapter 5 applies this test to product polynomials whose roots encode route-node multisets.

The polynomials must be fixed before X is known. Otherwise the prover can choose a false polynomial tailored to the evaluation point, outside the condition covered by the bound.

3.4.2 Non-Interactive Challenges

An interactive verifier can sample X after receiving a message that fixes the claim. Fiat–Shamir reproduces this dependency non-interactively by hashing the statement and preceding transcript [60]:

$$X = H(\text{encode}(x, \tau)). \tag{3.13}$$

The transcript τ must bind every value that determines the polynomial. Changing the claim then changes the hash input and, in the random-oracle model, produces a fresh challenge [51].

The prover may still try several claims and keep one whose derived challenge is favorable, in a process called *grinding*. After Q fresh random-oracle queries, a union bound gives error at most $Q \cdot d/|\mathbb{F}|$. These are hash queries rather than completed proofs, so the guarantee relies on the prover being computationally bounded; the bound remains small when $Q \cdot d \ll |\mathbb{F}|$.

3.5 Security Model and Trust Boundaries

An accepted proof is relative to a circuit, verification key, and public instance. The security model separates what the proof establishes from what the surrounding application must authenticate.

3.5.1 Adversarial Model

The prover is computationally bounded but may otherwise deviate arbitrarily. It controls the route, auxiliary values, Merkle paths, blindings, segment summaries, and proof data. No trust is placed in the solver, witness generator, or other prover-side preprocessing; their outputs acquire meaning only through constraints and cryptographic checks.

The verifier follows the complete procedure: it uses the expected key, parses public inputs in their specified order, performs the required cross-proof checks, and obtains the matrix root and threshold independently of the proof package.

Privacy is defined against any efficient party that receives the proof and its public inputs. The backend protects private witness values; the architecture determines what appears in the public statement.

3.5.2 Security Layers

Three layers connect acceptance to relation (1.2). First, UltraHonk completeness, knowledge soundness, and zero-knowledge govern satisfying assignments under a fixed verification key. These backend properties are assumed rather than reproved for each circuit.

Second, the encoding of the relation must show that a satisfying assignment contains a route that covers the target nodes, uses the matrix's costs, and meets the threshold. Deterministic constraints give exact implications. Merkle openings additionally rely on collision resistance; product-based coverage relies on the randomized soundness bound and challenge model of Section 3.4.

Third, composition must bind the local segment statements into one route. External constructions enforce the interface equalities in the verifier procedure; recursion verifies and reconciles the segment proofs inside the outer circuit.

3.5.3 Trust Assumptions and Scope

The stack uses a universal, updatable structured reference string: one setup supports circuits up to a size bound, followed by circuit-specific key generation [47]. Security requires at least one honest contributor to destroy its update randomness [61] and the backend's stated cryptographic assumptions. The verification key fixes the circuit, interface, and structural parameters; the verifier must obtain it from an authenticated artifact, registry, manifest, or configuration. Under recursion, the outer key authenticates the embedded segment key.

The verifier similarly derives the matrix root from the public matrix and obtains T from the application. If only a root is available, an external source must bind it to the intended matrix; accepting a prover-selected root or threshold proves a prover-selected instance.

4 Monolithic Proofs

This chapter develops the monolithic proof used as the baseline for the rest of the thesis. We first translate relation (1.2) directly, then refine two independent parts of the resulting circuit: the permutation check and the representation of the cost matrix.

4.1 Naive Construction

A monolithic proof handles the whole route in one circuit and enforces every part of the relation itself. The naive construction follows that relation directly: it exposes the original problem data, keeps the route private, and translates validity and feasibility into three constraint groups.

4.1.1 Inputs

Relation (1.2) separates the values known to the verifier from the witness known only to the prover. For the naive construction, the public instance is $x = (C, T)$ and the private witness is the route $w = \pi$. The implementation flattens C row by row into

$$\textsc{matrix}\big[0 \,..\, n^2 - 1\big], \tag{4.1}$$

where the cost of the arc from node i to node j is stored at $\textsc{matrix}[i \cdot n + j]$. The matrix and threshold T are public inputs to the proof system. This is the most literal interface: the backend receives the same matrix that defines the TSP instance, in the representation used by the cost lookup.

Making the matrix a proof-system public input reveals no additional information because C already belongs to the public problem instance. The choice still has a cost. The backend must process all n^2 entries even though one route selects only n of them. Section 4.3 later replaces the full matrix with a Merkle root derived from it.

The witness is the array $\textsc{route}[0 \,..\, n-1]$, corresponding to the sequence $\pi = (\pi_0, \pi_1, ..., \pi_{n-1})$ of Section 1.2.1. The entry $\textsc{route}[i]$ identifies the node visited at step i. The array stores nodes rather than arcs, so the circuit derives each arc from two consecutive entries. The closing arc is implicit as explained in Section 1.2.1: $\textsc{route}[n-1]$ returns to $\textsc{route}[0]$, and $(i+1) \bmod n$ therefore defines the destination for every i, including the last position.

The route's total cost also remains private. It is not an independent witness value that the prover may choose: the circuit derives it by retrieving and summing the costs of these n arcs. This leaves only the route as the private input to the construction.

4.1.2 Constraints

Given this interface, the circuit must establish that the route is valid and that its cost satisfies the threshold. It divides this claim into three constraint groups.

- **Permutation.** For every $0 \le i < n$, the circuit requires $\textsc{route}[i] < n$; for every $i < j$, it requires $\textsc{route}[i] \neq \textsc{route}[j]$. The range checks restrict the entries to the n valid node identifiers, while pairwise distinctness prevents any identifier from appearing twice. Since the route contains n distinct values drawn from an n-element set, the pigeonhole principle guarantees that it contains every node exactly once. Throughout the thesis, *permutation* and *coverage* refer to this same property. Without this condition, a prover could reduce the claimed cost by repeating cheap nodes and omitting expensive ones.
- **Cost binding.** For each position i, the circuit obtains the selected arc cost from

$$\textsc{matrix}[\textsc{route}[i] \cdot n + \textsc{route}[(i+1) \bmod n]]. \tag{4.2}$$

The two route entries identify the source and destination, and the row-major expression identifies their unique entry in the matrix. Reading that entry binds the sum to the graph instead of allowing the prover to supply convenient costs. Section 4.3 changes how the circuit authenticates the selected value, but preserves this requirement.

- **Threshold.** The circuit sums the bound costs and enforces

$$\text{cost}(\pi) = \sum_{i=0}^{n-1} \textsc{matrix}[\textsc{route}[i] \cdot n + \textsc{route}[(i+1) \bmod n]] \leq T. \quad (4.3)$$

Cost binding ensures that every term in (4.3) is the cost of the corresponding graph arc. The final inequality can therefore establish feasibility by comparing the legitimate total with T.

Together, these groups hold exactly when the witness encodes a valid route whose cost is at most T. Their meaning remains fixed throughout the thesis, while later constructions replace the enforcement mechanisms for permutation and cost binding.

4.1.3 Circuit Specification

The circuit in Listing 4.1 realizes the three groups: it range-checks the route and compares every pair of positions, reads each selected cost from the public matrix, and constrains the resulting sum by T.

Throughout the listings, `require` P means that P is added to the circuit constraints. The Noir implementation uses `assert`, but the operation does not wait until runtime and abort on failure like an ordinary program assertion. Arithmetization translates it into algebraic conditions, and proof generation succeeds only when the witness satisfies them, as described in Section 3.2.1.

The pairwise check illustrates the gap between source code and circuit work. The source expression `require` $a \neq b$ resembles a primitive comparison, but an arithmetic circuit must express the inequality through field equations. Let $v = a - b$. A field element is nonzero exactly when it has a multiplicative inverse, so witness generation supplies the auxiliary value $u = v^{-1}$ and the circuit constrains $v \cdot u = 1$.

Listing 4.1: The naive monolithic circuit. The matrix and the threshold are public, while the route is private. Group 1 is enforced by range checks plus pairwise distinctness. Group 2 is a direct lookup into the public cost matrix. Group 3 is a simple inequality constraint.

```
1  circuit monolithic-full-pairwise(n)
2      public MATRIX[0 .. n^2 − 1], T
3      private ROUTE[0 .. n − 1]
       Group 1: permutation by range check + pairwise distinctness
4      for i in range(0, n):
5          require ROUTE[i] < n
6      for i < j in range(0, n):
7          require ROUTE[i] ≠ ROUTE[j]
       Group 2: cost binding by direct lookup into the public matrix
8      TOTAL ← 0
9      for i in range(0, n):
10         TOTAL ← TOTAL + MATRIX[ROUTE[i] · n + ROUTE[(i + 1) mod n]]
       Group 3: threshold
11     require TOTAL ≤ T
```

When $a = b$, we have $v = 0$, and no choice of u can satisfy the equation. When they differ, the inverse exists and the constraint holds. Computing u adds no constraint, but remains witness-generation work; checking it adds one multiplication constraint. This is the hint pattern introduced in Section 3.2.1: the prover derives an auxiliary value outside the circuit, while the circuit verifies the property that affects acceptance. Pairwise distinctness repeats this process for all $n(n-1)/2$ pairs.

The matrix read exposes a second gap between the source program and its circuit. The positions in $\text{ROUTE}[i]$ and $\text{ROUTE}[(i+1) \bmod n]$ are fixed when the loop is unrolled. The values stored there are private, but the compiler already knows which two route wires to connect. Those values

then determine the outer matrix index. Since they arrive with the witness, the compiler does not know which matrix position the route will select.

This timing determines the kind of array access. A circuit is a static wiring diagram built before the witness exists. A fixed-index read connects a predetermined array entry to the next operation. A read from MATRIX[k], where k depends on the route, cannot choose that connection after compilation. Public matrix values can therefore require dynamic-access machinery when a private witness selects their address.

Noir realizes this witness-indexed access as a read-only lookup backed by an offline memory check, as described in Section 3.2.3. Each dynamic read contributes records, constraints, and witness-solving work; a fixed-index read needs only a predetermined wire.

The naive circuit has two independent quadratic costs. Pairwise distinctness contributes $n(n-1)/2 = \mathcal{O}(n^2)$ constrained inequalities. The matrix representation contributes exactly n^2 public inputs even though the route performs only n dynamic reads. The first cost follows from how permutation is enforced; the second follows from how the cost matrix is represented.

These source-level counts do not directly determine gate counts. Inequalities, range checks, memory operations, and public inputs compile into different backend primitives, so equal asymptotic growth can hide a large concrete difference. The measurements later in this chapter show that the full public matrix costs substantially more gates than the pairwise comparisons and dominates the naive circuit.

The two costs also affect different parties. The prover pays for the internal permutation constraints, while the verifier must process the public matrix entries. Exposing n^2 entries therefore makes verification time grow quadratically in n, although proof size remains constant. The next section replaces the pairwise permutation check; Section 4.3 later replaces the public matrix interface and its direct cost lookup.

4.2 Permutation Mechanisms

Section 4.1 enforces coverage with range checks and pairwise inequalities. The construction is exact, but its $n(n-1)/2$ comparisons make it quadratic. We therefore compare three scalable alternatives: an indicator vector, an inverse permutation map, and a sorted route.

Only the coverage mechanism changes. Cost binding and threshold enforcement remain fixed, so the comparison isolates how each alternative uses constraints, memory, and auxiliary witness computation. All three mechanisms are exact; their concrete cost and their ability to transfer to decomposition determine which one is carried forward. Chapter 5 introduces product coverage separately, when decomposition creates an interface that its probabilistic compression can reduce.

For each alternative, we first establish why its constraints imply coverage, then separate the work enforced in-circuit from the auxiliary work performed during witness generation.

4.2.1 Indicator Vector

The indicator mechanism initializes a boolean array $\textsc{visited}[0 \,..\, n-1]$ to false and scans the route once. At position i, the circuit requires $\textsc{route}[i] < n$, reads $\textsc{visited}[\textsc{route}[i]]$, requires the cell to be false, and then marks it true. A repeated node reaches a cell that was already marked, so the witness fails.

The range check completes the argument. It restricts all n route entries to the n valid node identifiers, while the state update makes those entries distinct. The pigeonhole principle then forces the route to contain every identifier exactly once, so a final pass requiring all cells to be true would add no guarantee. The range check also guards the memory access itself: because $\textsc{route}[i]$ selects an address in $\textsc{visited}$, the circuit must prove that this address lies within the allocated array.

The mechanism performs n initialization writes, n reads, n marking writes, and n range checks. Its constrained work is linear, but it uses read-write memory because every later read must respect the preceding updates. The indicator vector keeps a simple witness interface because it introduces no explicit auxiliary permutation hint.

4.2.2 Inverse Permutation Map

The inverse-map mechanism asks the prover for a private array INV-PERM, where INV-PERM$[v]$ claims the position of node v in the route. This array is an explicit hint: witness preparation computes the inverse relation, while the circuit checks every claim rather than reconstructing the map itself. For every $0 \leq v < n$, it enforces

$$\text{INV-PERM}[v] < n, \quad \text{ROUTE}[\text{INV-PERM}[v]] = v. \tag{4.4}$$

This changes the direction of the proof. Pairwise distinctness compares route positions with one another; the inverse map instead asks once for the location of each expected value. The circuit can therefore replace all-pairs comparisons with one checked claim per node.

The equality checks make the claimed positions distinct without comparing them directly. If INV-PERM$[a]$ = INV-PERM$[b]$ for $a \neq b$, the same route entry would have to equal both values. The n claims therefore identify distinct positions, and the range checks place them in an array with exactly n positions. By the pigeonhole principle, the claims cover every route position. Since the second constraint also places each node at its claimed position, the route is a permutation of $\{0, \ldots, n-1\}$.

Checking the hint requires n range checks and n read-only accesses at prover-chosen positions. The route itself never changes, so these accesses require no read-write consistency; they only need to bind each retrieved value to the requested address. This gives the mechanism a linear in-circuit account, although the dynamic reads still incur the offline-memory work described in Section 3.2.3.

Constructing INV-PERM requires one off-circuit scan of the route. In this implementation, that scan and the serialization of the resulting array occur before witness generation, so the reported witness time measures only the in-circuit checks. An end-to-end implementation must also pay for the omitted preprocessing. Its linear cost is unlikely to dominate at the evaluated sizes, so including it would provide more precise measurements, but it would not significantly affect the overall behavior and the comparison.

4.2.3 Sorted Route

A route covers $\{0, \ldots, n-1\}$ exactly when sorting it produces the identity sequence. Computing a full sort through constrained comparisons would add unnecessary circuit work, so the built-in follows the hint pattern: witness generation constructs a candidate ordering, and the circuit verifies it. The mechanism therefore enforces

$$\text{sort}(\textsc{route}) = (0, \ldots, n-1). \tag{4.5}$$

Listing 4.2 gives the circuit specification. The built-in sort constructs the candidate during witness generation, and the circuit then fixes each position to its expected node identifier.

Listing 4.2: The sort mechanism for the permutation constraint group, shown on its own and operating on the private ROUTE.

```
1 sort-permutation-check(ROUTE)
2     SORTED ← sort(ROUTE)    built-in sort function
3     for i in range(0, n):
4         require SORTED[i] = i
```

The candidate SORTED is auxiliary witness data, so the circuit cannot accept it without checking its relation to ROUTE. In the pinned Noir release, the built-in constructs SORTED with an unconstrained quicksort, constrains adjacent entries to be ordered, and uses an auxiliary shuffle map to prove that SORTED and ROUTE contain the same multiset [45]. The identity checks in Listing 4.2 then fix that multiset to $\{0, \ldots, n-1\}$, so the complete check establishes exact coverage without trusting the unconstrained sort. The ordering check is redundant for this particular use: the identity checks already make SORTED ordered. Noir nevertheless performs it as part of the general-purpose sort, so the circuit still pays for its $n-1$ inequalities.

In-circuit, the mechanism uses n identity checks, $n-1$ ordering inequalities, and about $2n$ read-only accesses for the shuffle-map constraints that bind SORTED to ROUTE. These operations grow linearly with n. The built-in generates both the sorted candidate and shuffle map during witness

generation, so their cost is included in the measured witness time, unlike the externally prepared inverse map.

Off-circuit, the implementation performs an average-case $\mathcal{O}(n \log n)$ quicksort, with quadratic worst case, and an $\mathcal{O}(n^2)$ shuffle-map search.

4.2.4 Mechanism Evaluation

Operation counts predict growth but cannot price the mechanisms directly. Range checks, read-write operations, read-only lookups, and direct comparisons compile differently, so Table 4.1 keeps their costs separate and records the auxiliary work performed during witness construction.

Table 4.1: The four permutation mechanisms by asymptotic cost: the work each constrains in-circuit, the unconstrained work the prover performs to build its hints, and the dynamic memory used by the permutation mechanism. The common read-only memory used for cost binding is excluded. The sorting bound is average-case for the built-in quicksort, whose worst case is quadratic.

Mechanism	In-Circuit	Off-Circuit	Memory
Pairwise Distinctness	$\mathcal{O}(n)$ range, $\mathcal{O}(n^2)$ inequalities	$\mathcal{O}(n^2)$ inverses	None
Indicator Vector	$\mathcal{O}(n)$ range, $\mathcal{O}(n)$ read-write ops	None	Read-Write
Inverse Permutation	$\mathcal{O}(n)$ range, $\mathcal{O}(n)$ dynamic reads	$\mathcal{O}(n)$ scan	Read-Only
Sorted Route	$\mathcal{O}(n)$ equalities, $\mathcal{O}(n)$ orderings, $\mathcal{O}(n)$ dynamic reads	$\mathcal{O}(n \log n)$ sort, $\mathcal{O}(n^2)$ shuffle map	Read-Only

Pairwise is quadratic both in constrained comparisons and in the inverses constructed for those comparisons. The other mechanisms remain linear in constrained work but mix different operations and memory models, so their shared growth rate does not rank them.

We benchmark the four complete circuits up to $n = 480$, a range large enough for the quadratic pairwise trend to separate from the alternatives.

Every circuit uses the same public matrix, route, cost binding, and threshold; only coverage enforcement changes. We record gates, ACIR opcodes, witness and proving time, verification time, proof size, and maximum Barretenberg-reported proving memory under the protocol of Section 6.1.2. Gate count captures the circuit presented to the backend, while witness time exposes hint construction and memory-solving work that gates may not predict.

Figure 4.1 compares the resulting gate and opcode counts.

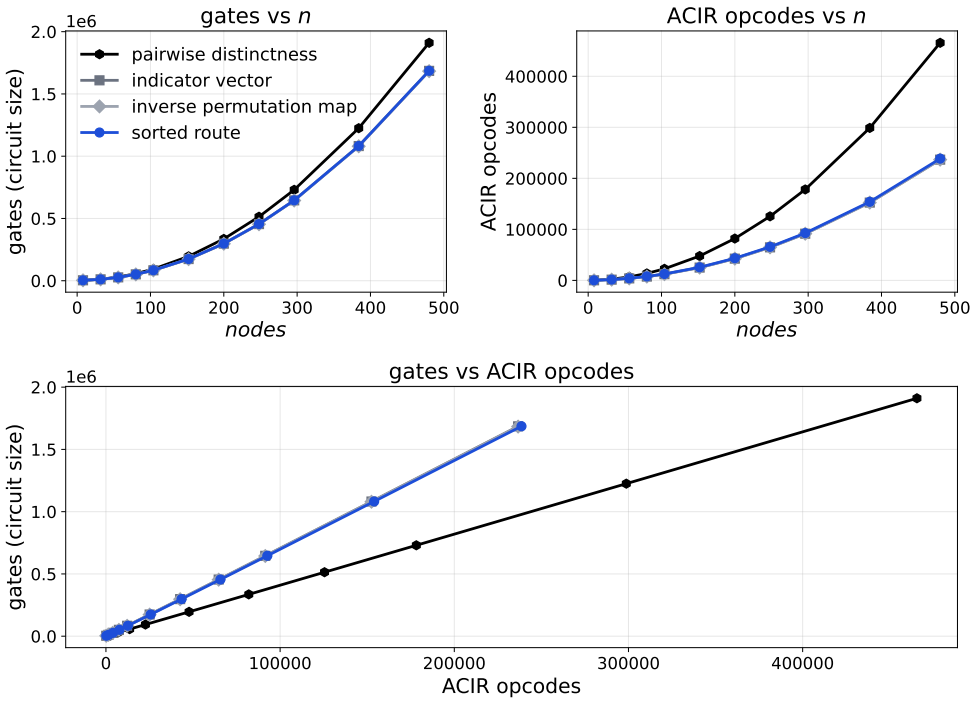


Figure 4.1: Three views of circuit size for different permutation mechanisms. Top left and top right show gate count and ACIR opcodes against the number of nodes, respectively. Bottom relates the two whole-circuit totals; it should not be interpreted as the gate price of an individual opcode.

Pairwise emits substantially more ACIR operations because it compares every pair of positions. The difference narrows after lowering to backend gates: all four circuits still process the same n^2 matrix inputs, whose common gate cost dominates the coverage mechanism. The three linear alternatives are consequently almost indistinguishable in gates, while pairwise is about thirteen percent larger at $n = 480$.

Gate count omits the work needed to construct and solve the witness. Figure 4.2 therefore plots witness-generation time against both n and gate count.

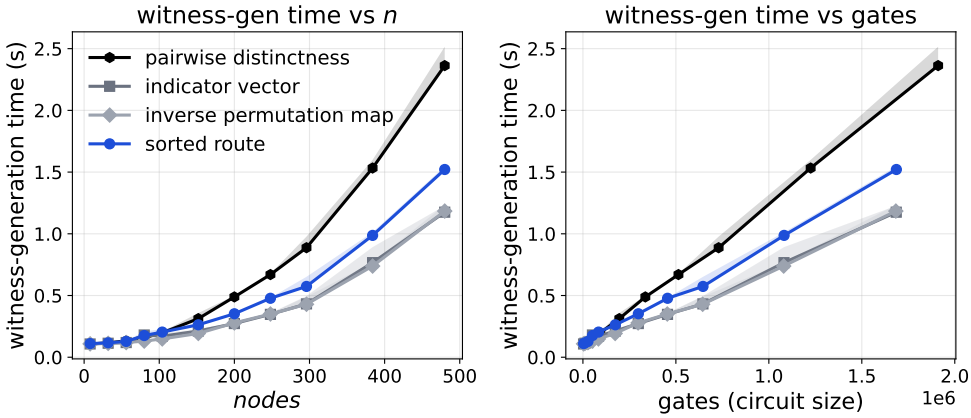


Figure 4.2: Witness-generation time for the four permutation mechanisms on one canonical route per size. Left: time against the number of nodes. Right: the same time against gate count. Each line is the minimum of five sequential runs, used as a lower-envelope estimate; the shaded band spans the observed minimum and maximum.

Witness time separates the mechanisms that gate count leaves tied. Pairwise is slowest, consistent with the inverse hints generated for its quadratic inequalities. At the endpoint, the indicator vector and inverse map each require about 1.18 seconds, while sorting requires 1.52 seconds, roughly thirty percent more.

Pairwise is unsuitable as the scalable mechanism because it adds quadratic circuit and witness work. The three alternatives are effectively tied in circuit size, while witness generation favors the indicator vector and inverse map. Proving time and maximum reported memory largely follow the compiled circuit size and its padded proving domain; their curves reinforce the rejection of pairwise but do not rank the linear mechanisms, so we omit them as redundant. Verification time and proof size likewise show no mechanism-dependent difference.

4.2.5 Mechanism Selection

The inverse map is the strongest measured monolithic candidate: it has the smallest circuit and shares the fastest witness-time tier, subject to its preprocessing qualification. We nevertheless select sorting as the canonical exact mechanism because the same check transfers directly to composite proofs, where the route is decomposed into segments.

For the decomposition considered in the next chapter, assume that K divides n and split the route into K consecutive, non-overlapping segments of $M = n/K$ nodes each. Let R_i be the private visit order of segment i, and let S_i be its M-entry target list, ordered by increasing node identifier. A segment must prove that both arrays contain the same nodes, while recombination must prove that the concatenation $S_0 \mathbin{+\!\!+} S_1 \mathbin{+\!\!+} \ldots \mathbin{+\!\!+} S_{K-1}$ covers $\{0, \ldots, n-1\}$. The target of a segment is therefore arbitrary instance data rather than the canonical identity sequence, and its representation must cross the boundary between the segment proof and recombination.

The indicator vector loses its direct addressing rule in this setting. An n-cell bitmap preserves $\textsc{visited}[v]$ but defeats segment locality; an M-cell bitmap needs a constrained translation from each global node identifier to its position in S_i, while retaining read-write memory.

The inverse map adapts through a private positional hint pos_i, where $\mathrm{pos}_i[j]$ gives the claimed position of target node $S_i[j]$ in the segment order R_i. A segment can then enforce $\mathrm{pos}_i[j] < M$ and $R_i[\mathrm{pos}_i[j]] = S_i[j]$. Once recombination establishes that the target entries are distinct, the claimed positions cover the segment. This target-relative construction remains linear and is a credible optimization, but it adds an explicit hint.

Sorting keeps the same form at both levels:

$$\mathrm{sort}(R_i) = S_i, \tag{4.6}$$

for every segment, followed by the global check

$$\mathrm{sort}(S_0 \mathbin{+\!\!+} S_1 \mathbin{+\!\!+} \ldots \mathbin{+\!\!+} S_{K-1}) = (0, \ldots, n-1), \tag{4.7}$$

where $+\!\!+$ denotes concatenation. The monolithic, segment, and recombination circuits can therefore use the same exact standard-library mechanism against different targets.

The inverse map may retain its witness-time advantage after decomposition and therefore should remain a suitable candidate, but sorting provides the simpler controlled baseline for comparing monolithic and composite proofs.

4.3 Committed Cost Matrix

The naive circuit supplies all n^2 matrix entries as proof-system public inputs although the route reads only n costs. This quadratic surface, identified in Section 4.1, also sets the scale of the mechanism comparison: the three linear coverage mechanisms in Section 4.2 differ in gates by only a fraction of a percent because they all carry the same matrix.

We replace those entries with one Merkle root and authenticate each selected cost against it. Only the proof-system representation changes: C and T remain public problem data, but the matrix contributes one public field element to the proof-system interface alongside T. The verifier derives the expected root from C and accepts a proof for (C, T) only when its public root matches that value.

Following the interface of Section 3.3.3, leaf $i \cdot n + j$ stores the row-major entry $C_{i,j}$. The n^2 leaves are padded with zeros to the next power of two, giving

$$\textsc{depth} = \left\lceil \log_2\left(n^2\right) \right\rceil. \tag{4.8}$$

The construction instantiates the binary Merkle tree of [57] with the Poseidon2 hash H [52]. Every internal node is $H(\text{left}, \text{right})$; the fixed row-major order, leaf count, and padding convention make the matrix encoding unambiguous.

The circuit can no longer retrieve a selected cost from a public array. For the arc leaving route position i, the prover instead supplies $\textsc{arc_costs}[i]$,

one sibling hash per tree level, and one Boolean direction bit per level. The paths for all n arcs are flattened: level d of path i is stored at

$$\text{SIBLINGS}[i \cdot \text{DEPTH} + d], \quad \text{PATH_BITS}[i \cdot \text{DEPTH} + d]. \tag{4.9}$$

Starting from $\text{ARC_COSTS}[i]$, the circuit uses each direction bit to place the running node on the left or right of its sibling, hashes the pair, and requires the final node to equal the public root. This authenticates the supplied cost at the leaf encoded by the path. It does not yet show that the leaf is the one selected by the route: without a position check, the prover could open a genuine cheaper entry elsewhere in the matrix.

The same direction bits therefore reconstruct the opened leaf index. Writing $b_{i,d} = \text{PATH_BITS}[i \cdot \text{DEPTH} + d]$ in least-significant-bit-first order, the path claims

$$p_i = \sum_{d=0}^{\text{DEPTH}-1} b_{i,d} 2^d. \tag{4.10}$$

The row-major index required by the route is

$$q_i = \text{ROUTE}[i] \cdot n + \text{ROUTE}[(i+1) \bmod n], \tag{4.11}$$

the same address used by the direct lookup in Section 4.1. The circuit requires $p_i = q_i$. Root equality binds the cost to the committed matrix; index equality binds it to the required arc. Listing 4.3 specifies both checks.

Applying this read to every route arc replaces n^2 public inputs with n paths of $\text{DEPTH} = \mathcal{O}(\log n)$ hashes. Cost binding therefore contributes $\mathcal{O}(n \log n)$ Poseidon2 invocations. Building the tree still touches all n^2 entries, but that is reusable instance preprocessing: one tree serves every proof against the same agreed matrix.

Under collision resistance, the root computationally binds the openings to C, following Section 3.3.2 and Section 3.3.3. It need not hide because C is public, and the verifier authenticates the root by recomputing it. A confidential-matrix relation would instead require a hiding commitment and an authenticated reference root, as described by the trust boundary in Section 3.5.3.

Listing 4.3: One authenticated read for the arc leaving ROUTE$[i]$. The index check (a) binds the path to the matrix entry selected by the route; the root check (b) binds the supplied cost to the committed matrix.

```
1  procedure merkle-tree-cost-binding(n, i, ROOT, DEPTH)
2      private ROUTE[0 .. n − 1], ARC_COSTS[0 .. n − 1]
3      private SIBLINGS[0 .. n · DEPTH − 1], PATH_BITS[0 .. n · DEPTH − 1]

       (a) Index constraint: the proof must open the correct cost
4      LEAF_IDX ← 0
5      for d in range(0, DEPTH):
6          if PATH_BITS[i · DEPTH + d]:
7              LEAF_IDX ← LEAF_IDX + 2^d
8      require LEAF_IDX = ROUTE[i] · n + ROUTE[(i + 1) mod n]

       (b) Merkle proof
9      HASH_ACC ← ARC_COSTS[i]
10     for d in range(0, DEPTH):
11         if PATH_BITS[i · DEPTH + d]:
12             HASH_ACC ← H(SIBLINGS[i · DEPTH + d], HASH_ACC)
13         else:
14             HASH_ACC ← H(HASH_ACC, SIBLINGS[i · DEPTH + d])
15     require HASH_ACC = ROOT
```

Other commitment interfaces offer different costs. KZG commitments provide constant-size evaluation openings, while KZG-backed Verkle trees use a higher branching factor to shorten authentication paths [56], [62]. Verifying those openings inside the circuit adds elliptic-curve pairing arithmetic and another setup-dependent commitment layer. Committed-lookup arguments such as Caulk, cq, and Lasso instead batch the position–cost rows selected from the table [63], [64], [65]. We retain a binary Poseidon2 tree because it is field-native, transparent, and supported by the existing stack. It provides a controlled baseline for measuring the representation change without claiming an optimal committed lookup.

4.4 Refined Circuit Construction

Combining the committed matrix with the selected sorted-route permutation mechanism gives the refined monolithic circuit in Listing 4.4. The circuit takes only the Merkle root and the threshold T as public inputs. The route, selected arc costs, and authentication paths remain private.

Listing 4.4: The refined circuit composes sort-based coverage with the authenticated committed-matrix read of Listing 4.3, then checks the accumulated cost against T.

```
1  circuit monolithic-merkle-sort(n, DEPTH)
2      public ROOT, T
3      private ROUTE[0 .. n − 1], ARC_COSTS[0 .. n − 1]
4      private SIBLINGS[0 .. n · DEPTH − 1], PATH_BITS[0 .. n · DEPTH − 1]

       Group 1: permutation by sorting
5      SORTED ← sort(ROUTE)
6      for i in range(0, n):
7          require SORTED[i] = i

       Group 2: cost binding by authenticated reads
8      TOTAL ← 0
9      for i in range(0, n):
10         merkle-tree-cost-binding(n, i, ROOT, DEPTH)
11         TOTAL ← TOTAL + ARC_COSTS[i]

       Group 3: threshold
12     require TOTAL ≤ T
```

We call this circuit *monolithic-merkle-sort*. We isolate the two refinements through three circuits. *Monolithic-full-pairwise* is the naive construction of Section 4.1; *monolithic-full-sort* changes only coverage; *monolithic-merkle-sort* additionally changes the matrix representation. The first transition therefore measures the permutation mechanism, and the second measures commitment independently of it.

All three circuits use the same instances and routes through $n = 1000$, their largest shared checkpoint. Following Section 6.1.2, gate counts are deterministic, timings use the lower envelope, and memory is the median of the run-level maximum Barretenberg-reported checkpoints. Proof size is omitted because every configuration produces the same 14,656-byte proof. Figure 4.3 and Figure 4.4 give the growth curves.

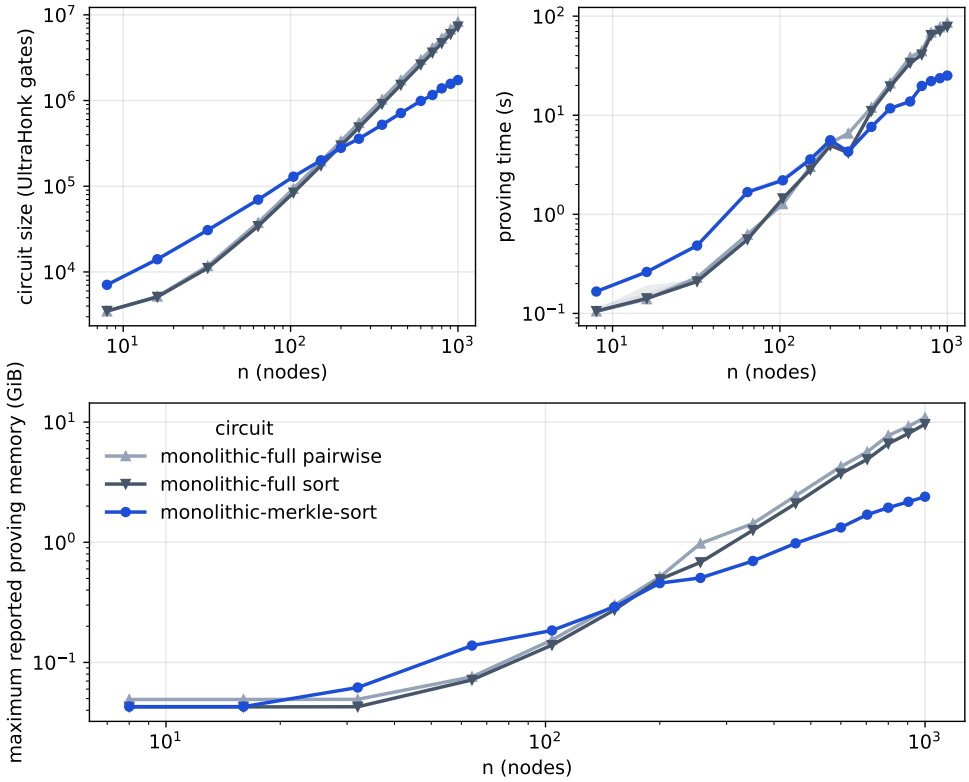


Figure 4.3: Circuit size, proving time, and maximum Barretenberg-reported proving memory through $n = 1000$, on logarithmic axes. The *full* circuits retain the quadratic public-matrix surface; Merkle commitment replaces it with a constant-size interface. Timing curves show minima, memory curves show medians of the per-run maxima, and bands span the observed ranges, following Section 6.1.2.

The two transitions affect different terms. Sorting removes the quadratic pairwise check, but both full circuits retain the n^2 public matrix, so their overall growth remains quadratic and the improvement is a constant factor. Commitment removes that common term and replaces the selected reads

with $\mathcal{O}(n \log n)$ hashing. Poseidon2's fixed cost makes the committed circuit larger at small n; gates and reported memory cross near $n \approx 176$, while proving time crosses near $n \approx 290$. Beyond those points, the removed matrix dominates the added hashes.

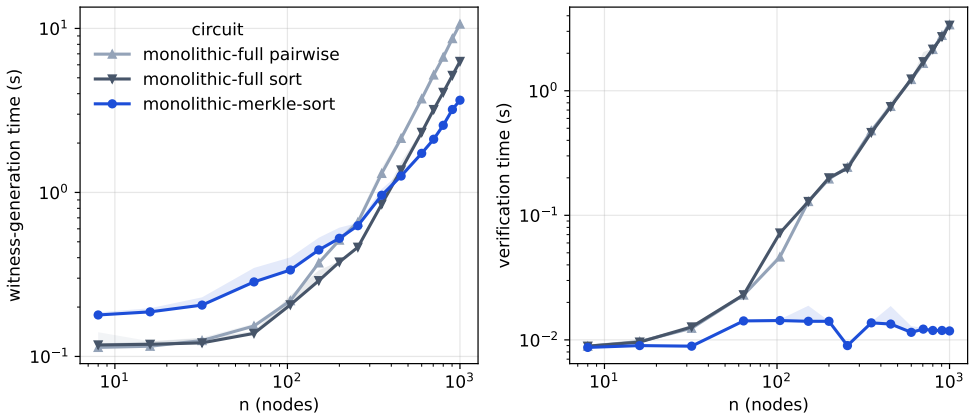


Figure 4.4: The two metrics not captured by gate count. Left: witness-generation time. Right: verification time. Times follow the lower-envelope convention of Figure 4.2.

The matrix representation also determines verifier work. The full circuits expose n^2 matrix entries and require about 3.4 seconds to verify at the endpoint; the committed circuit exposes one root and verifies in about 12 milliseconds. Figure 4.4 shows the full-interface cost growing with n, while the two-value interface keeps the committed verifier near the same scale.

Witness time reflects the operations hidden by the gate total. Sorting removes the $\mathcal{O}(n^2)$ inverse hints of pairwise inequality. Commitment removes the witness-indexed lookup over the public matrix described in Section 4.1: instead of laying out an n^2 table and resolving its access log, the solver evaluates the fixed-index path arrays and straight-line hashes of Section 4.3.

Sorting reduces constants within the full-matrix architecture; commitment changes its dominant term and public interface. Past the observed crossovers, *monolithic-merkle-sort* is smaller and faster across the varying metrics, but it still places the whole route in one growing proving task. The next chapter changes the architecture through decomposition.

5 Composite Proofs

This chapter develops composite proof architectures in response to the monolithic memory wall. We divide the route into independently provable segments, identify how the cut affects the constraints we have to enforce, and restore the global statement through recombination. The constructions differ in how they represent information, what they disclose, and whether recombination is external or recursive.

5.1 Proof Decomposition

The refined monolithic circuit holds the complete route in one witness, so it can sort all n nodes, authenticate every selected arc, and enforce the threshold directly. Decomposition replaces it with K independently proved segments. Each segment sees only part of the witness, so recombination must recover the global constraints and bind the segment proofs into one claim about relation (1.2).

5.1.1 Route Decomposition

We partition $\pi = (\pi_0, \ldots, \pi_{n-1})$ into K consecutive, non-overlapping segments. We require K to divide n, so each segment contains $M = n/K$ nodes:

$$R_i = \left(\pi_{iM}, \ldots, \pi_{(i+1)M-1}\right), \quad 0 \le i < K. \tag{5.1}$$

All segments therefore share one circuit shape for fixed (n, M).[2]

[2]The constructions additionally require $M > 2$. Uneven segments would require constrained padding or multiple circuit shapes and verification keys.

The cut preserves route order and keeps most arcs within a segment. Each segment contains $M-1$ internal arcs, giving $K(M-1) = n-K$ across the route. The remaining K arcs cross segment boundaries: boundary i connects $R_i[M-1]$ to $R_{(i+1) \bmod K}[0]$. The modular index includes the wrap-around arc from the last segment to the first.

Because the segments do not overlap, each route position belongs to one segment circuit. An overlapping cut would process boundary nodes twice and require an additional check that both segments used the same value.

The segment proofs can be generated independently: one worker may produce them sequentially while holding one segment circuit at a time, or multiple workers may run them in parallel. Each proving unit contains M route nodes rather than n, creating the possibility of lower per-task memory. The complete architecture must still perform recombination, so segmentation alone does not establish a lower end-to-end peak.

5.1.2 Local Proofs and Global Obligations

The cut leaves relation (1.2) unchanged; it changes which circuit can see enough witness information to enforce each constraint. The relation requires coverage of every node exactly once, authentication of every selected arc cost, and a total cost at most T, as introduced in Section 4.1.2.

Coverage and threshold depend on the complete route, so no segment can settle them alone. Arc authentication needs only one arc's endpoints, claimed cost, and Merkle opening against the public root, so it can be placed wherever both endpoints are available. Segment i holds both endpoints of its $M-1$ internal arcs, allowing it to authenticate them and certify their sum as the partial cost

$$L_i = \sum_{j=0}^{M-2} C_{R_i[j],R_i[j+1]}. \tag{5.2}$$

No segment holds both endpoints of a boundary arc. The cut therefore divides the original guarantees and introduces a fourth obligation to keep the separate proofs consistent. Table 5.1 summarizes the resulting split.

Table 5.1: What each segment can establish locally and what recombination must recover for the complete route.

Required guarantee	A segment establishes	What remains global
Coverage	Evidence bound to its own M nodes	Every node $0, \ldots, n-1$ occurs once
Arc authentication	Its $M-1$ internal arcs	The K boundary arcs
Threshold	Its authenticated partial cost L_i	Aggregate all partial and boundary costs and enforce T
Consistency	Validity of its own statement	Bind all segment claims to one instance

Coverage remains global even when every segment honestly certifies its own node content. Two valid local claims may repeat one node and omit another, and neither segment can observe the conflict. Likewise, an authenticated partial cost L_i accounts for only one part of the route cost and cannot establish the full threshold.

Independence also removes implicit consistency. Verifying one segment proof establishes only its own statement, so K valid proofs may use different roots or describe unrelated route pieces. Recombination must bind the segment claims to the same instance, creating the composition obligation identified in Section 3.5.2.

Recombination must therefore establish global coverage, authenticate the K boundary arcs, aggregate all costs and enforce T, and bind the independently certified segment claims into one statement. These four obligations form the recombination contract developed next.

5.1.3 Recombination Contract

To discharge these four obligations, recombination needs each segment to certify coverage evidence, its two endpoints, and its authenticated partial cost. We collect them in the logical summary

$$\sigma_i = (\mathrm{cov}_i, \mathrm{start}_i, \mathrm{end}_i, L_i). \tag{5.3}$$

Here, cov_i denotes the construction-specific evidence that binds coverage to R_i. It may contain the segment nodes themselves or a product summary

together with the values that bind its challenge. The remaining fields identify the boundary endpoints and the authenticated partial cost of the $M - 1$ internal arcs.

Given the K summaries, recombination first combines their coverage evidence to establish the global coverage claim. It then uses the endpoints to restore the arcs removed by the cut. For every $0 \leq i < K$, it obtains the boundary cost

$$b_i = C_{\text{end}_i,\text{start}_{(i+1) \bmod K}} \tag{5.4}$$

and authenticates it with a Merkle path under the matrix root. Finally, it combines the boundary costs with the segment partial costs and enforces the threshold T.

These global checks are meaningful only if they use the summaries certified by the segment proofs. If recombination could use independently chosen values, it could establish coverage and the threshold for segments that were never proved. *Composition binding* closes this gap by tying every recombination summary to its corresponding segment claim. It also binds every segment to the authenticated matrix root used for the boundary openings and ensures that recombination uses the intended T.

With these bindings in place, backend knowledge soundness associates each accepted segment proof with a valid witness R_i. Their ordered concatenation defines the candidate route

$$\hat{\pi} = R_0 \mathbin{+\!\!+} R_1 \mathbin{+\!\!+} \ldots \mathbin{+\!\!+} R_{K-1}. \tag{5.5}$$

The certified coverage evidence establishes that $\hat{\pi}$ contains every required node exactly once: exactly under sorting, or except with the stated failure probability under product coverage. The segment proofs authenticate the internal arcs, while recombination authenticates the boundary arcs, all under the same root. These two sets partition the route arcs, so

$$\text{cost}(\hat{\pi}) = \sum_{i=0}^{K-1} L_i + \sum_{i=0}^{K-1} b_i \leq T. \tag{5.6}$$

The reconstructed route therefore witnesses relation (1.2) for the authenticated instance.

The constructions that follow differ in where they place these summaries. Plaintext and committed constructions use external composition, exposing either the summaries or their commitment digests; recursion carries the summaries and segment proofs into an outer circuit. That placement determines structural disclosure at the public-interface boundary defined in Section 3.5.1 and studied in Section 1.3.3. We begin with external composition, where the final verifier binds independent segment and recombination proofs.

5.1.4 External Composition

External composition realizes the binding requirement of Section 5.1.3 with K segment proofs and one independent recombination proof, which we call the *glue proof*. The glue establishes the global recombination checks over its own summary copies but never receives the segment proofs. The verifier completes the contract by binding these copies to the summaries authenticated by the segment proofs.

Segment proof i exposes $\left(\sigma_i^{\text{seg}}, \textsc{root}\right)$, while the glue exposes the corresponding σ_i^{glue} values, the same root, and T. After checking all $K+1$ proofs under the authenticated keys and checking ROOT and T against the intended instance, the verifier requires

$$\sigma_i^{\text{seg}} = \sigma_i^{\text{glue}}, \quad 0 \le i < K. \tag{5.7}$$

Together with the common-root check, these equalities prevent a valid glue claim from being paired with segment proofs for different summaries or a different instance. They establish composition binding, after which the route-level implication follows from Section 5.1.3 under the trust boundary of Section 3.5.3. Figure 5.1 shows this direct public-summary topology.

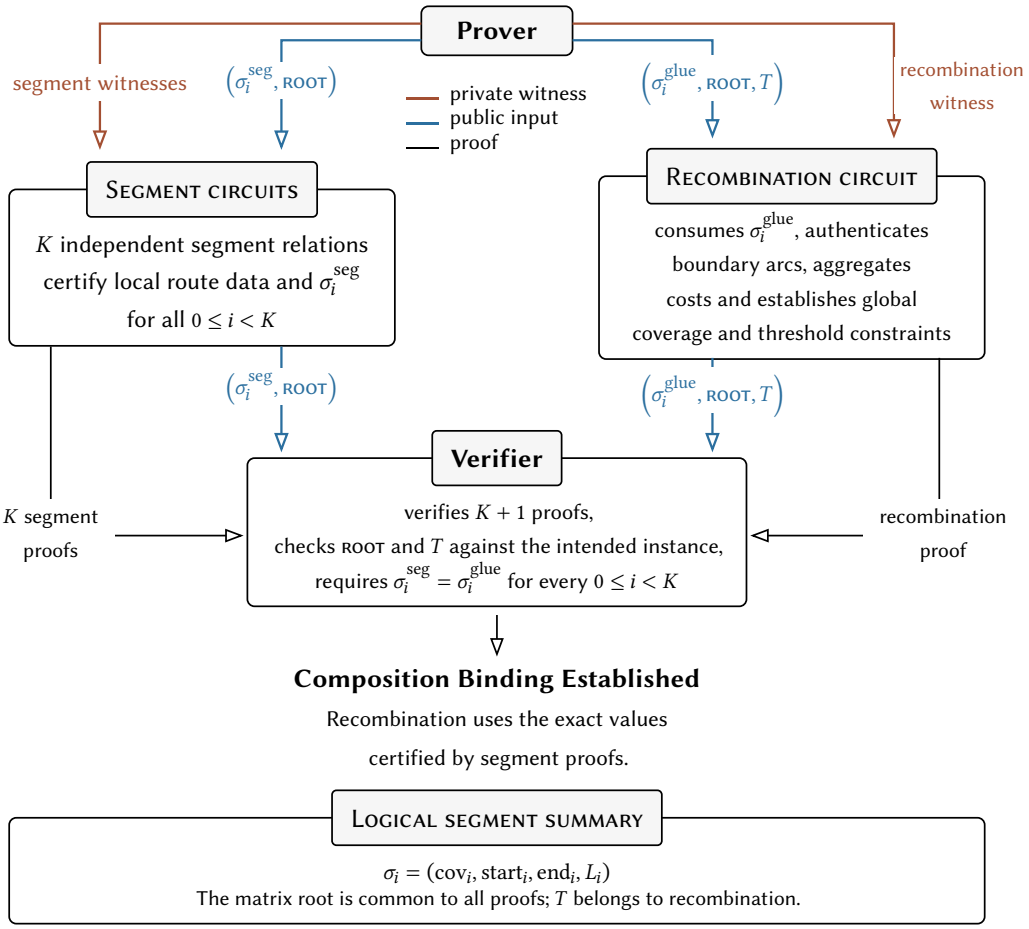


Figure 5.1: Direct external composition with public summaries. The verifier checks K segment proofs and one independent recombination proof, checks their root and threshold against the intended instance, and binds the two copies of each segment summary.

5.1.5 Stitching Tax

The cut changes where the original relation is enforced, but relocated work is not composition overhead. The *stitching tax* introduced in Section 1.4 is the net work added to recover one global claim from the segment proofs. Table 5.2 summarizes its main components.

Table 5.2: How decomposition redistributes the original relation and which operations add composition work.

Component	Placement after decomposition	Net effect
Arc authentication	$n - K$ internal openings in the segments and K boundary openings in recombination	Conserved at n openings
Cost accumulation	Segments form L_i; recombination adds every L_i and b_i	$\mathcal{O}(K)$ partial-cost aggregation
Coverage	Local evidence followed by a global check whose form depends on the mechanism	Representation-dependent
Proof binding	External equality checks or in-circuit recursive verification	Introduced by the cut

Arc authentication is conserved exactly. The segments open the $n - K$ internal arcs and recombination opens the K boundary arcs, giving the same n authenticated costs as the refined monolithic circuit. Every route cost is likewise included once. The additional arithmetic comes from combining the K segment partial costs with the boundary costs, which adds $\mathcal{O}(K)$ aggregation.

Coverage depends on its representation. With sorting, the recombination circuit retains the monolithic n-node sort, while the segment circuits add K local M-node sorts to bind their private routes to public coverage evidence. A fixed-size fingerprint can remove the n-entry interface and change the global check, but producing the fingerprints and binding their challenge adds work elsewhere. This motivates the product construction developed later in the chapter.

Decomposition makes each segment proving unit smaller, while the complete architecture still proves the full relation and pays for every stage. Smaller segments may lower per-task memory and can be scheduled independently, but the end-to-end peak still includes the glue or outer stage. Chapter 6 measures whether the smaller proving units survive this recombination cost as a complete memory advantage.

With the contract, external topology, and cost accounting fixed, we first instantiate external recombination through two plaintext-summary

constructions. Plain-sort retains exact sorting, while plain-product replaces the public node lists with fixed-size product fingerprints.

5.2 Plain Constructions

These constructions use external composition with plaintext segment summaries. Each segment proof certifies its summary against a private route segment, an independent glue proof consumes a corresponding copy, and the verifier binds the two statements through the equalities of Section 5.1.4.

Both variants share the same route and cost structure. The segment proofs authenticate internal arcs under a common matrix root and bind their endpoints and partial costs. The glue uses those endpoints to authenticate the missing boundary arcs, combines all costs, and enforces the threshold. The variants differ in their coverage evidence: plain-sort publishes each segment's sorted node multiset, while plain-product replaces that list with a product fingerprint and the values needed to bind its challenge.

5.2.1 Plain Sort

Plain-sort applies the exact sort mechanism of Section 4.2.5 at both levels. For an ordered segment R_i, let $S_i = \mathrm{sort}(R_i)$ be its sorted node multiset. The public segment summary is

$$\sigma_i^{\mathrm{sort}} = (S_i, \mathrm{start}_i, \mathrm{end}_i, L_i). \tag{5.8}$$

The glue cannot access R_i, so each public field must carry one part of the local claim across the proof boundary. The segment circuit binds those fields to the same private route segment; Table 5.3 records how recombination uses them.

The segment circuit witnesses the M nodes of R_i in visit order, the $M - 1$ internal costs, and their Merkle authentication paths. Its public statement contains S_i, the two endpoints, L_i, and the common root.[3]

The circuit sorts R_i and requires the result to equal S_i position by position. The constrained sort follows Section 4.2.5: witness generation proposes the

[3]These are statement inputs, not return values: the circuit constrains their relation to the witness, it does not compute them on the prover's behalf.

Table 5.3: How each public value is bound to the private segment and used by recombination.

PUBLIC VALUE	SEGMENT BINDING	RECOMBINATION USE
S_i	$S_i = \mathrm{sort}(R_i)$	Exact global coverage
Start and end	First and last nodes of R_i	Boundary-arc authentication
L_i	Authenticated sum of the $M-1$ internal costs	Full-route threshold
ROOT	Internal Merkle paths reach the common root	Boundary paths use the same matrix instance

sorted array, while the circuit proves that it is ordered and is a permutation of R_i. The circuit also binds the published endpoints to $R_i[0]$ and $R_i[M-1]$, authenticates each internal arc, and requires L_i to equal their sum. These checks make every summary field refer to the same private segment.

Before forming a Merkle index, the circuit also requires every node of R_i to be smaller than n. Global coverage would eventually reject an out-of-range node, but the local guard protects the segment's own authenticated reads. Without it, the index $R_i[j] \cdot n + R_i[j+1]$ could enter the zero-padded region of the tree and open a zero-cost leaf. Listing 5.1 collects these checks.

Listing 5.1: One plain-sort segment: range check (a), exact multiset binding (b), endpoint binding (c), and authenticated internal costs accumulated into the partial cost (d). The authenticated read is Listing 4.3 with global leaf index $u \cdot n + v$.

```
1 circuit plain-sort-segment(n, M, DEPTH)
2     public SORTED_NODES[0 .. M − 1], START, END, PARTIAL_COST, ROOT
3     private SEGMENT[0 .. M − 1], EDGE_COSTS[0 .. M − 2]
4     private SIBLINGS[0 .. (M − 1) · DEPTH − 1]
5     private PATH_BITS[0 .. (M − 1) · DEPTH − 1]

      (a) range: each node is a valid index, guarding the leaf-index arithmetic
6     for i in range(0, M):
7         require SEGMENT[i] < n
```

```
    (b) the published multiset is this segment's nodes
8   SORTED ← sort(SEGMENT)
9   for i in range(0, M):
10      require SORTED[i] = SORTED_NODES[i]

    (c) endpoints are the segment's true first and last nodes
11  require START = SEGMENT[0]
12  require END = SEGMENT[M − 1]

    (d) the M − 1 internal arcs, authenticated against the committed root
13  TOTAL ← 0
14  for i in range(0, M − 1):
15      authenticated-read(SEGMENT[i], SEGMENT[i + 1], EDGE_COSTS[i],
                           n, i, ROOT, DEPTH)
16      TOTAL ← TOTAL + EDGE_COSTS[i]

17  require TOTAL = PARTIAL_COST
```

Binding S_i to R_i does not establish that the segment nodes are distinct, and one segment cannot detect a node repeated by another. Coverage therefore remains global. The glue receives $S_0 \mathbin{\#} S_1 \mathbin{\#} \ldots \mathbin{\#} S_{K-1}$, sorts their n entries, and requires the identity sequence $0, \ldots, n-1$. Any repetition leaves some required node absent, whether the duplicate occurs within one segment or across two proofs, so this single check establishes exact global coverage.

The glue also receives the K starts, ends, and partial costs, together with T and the common root. Its private witness contains the K boundary costs and their Merkle authentication paths. For every $0 \le i < K$, it authenticates the arc from the end of segment i to the start of segment $(i+1) \bmod K$; the final index wraps to segment 0 and closes the cycle. Each path must reach the common root at row-major leaf index $\textsc{ends}[i] \cdot n + \textsc{starts}[(i+1) \bmod K]$.

Writing b_i for the authenticated outgoing boundary cost of segment i, the glue enforces

$$\sum_{i=0}^{K-1} L_i + \sum_{i=0}^{K-1} b_i \leq T. \tag{5.9}$$

The glue cannot recompute the partial costs because it receives each segment's node multiset, not its visit order. Their validity comes from the segment proofs, while external composition binds the values used by the glue to those certified claims. Listing 5.2 gives the resulting circuit.

Listing 5.2: The recombination circuit for plain-sort. It checks that the concatenated public node multisets cover $0, \ldots, n-1$, each exactly once (a), authenticates the K boundary arcs $\text{ENDS}[i] \to \text{STARTS}[(i+1) \bmod K]$ against the committed root while accumulating their costs (b), and combines the boundary costs with the published segment partial costs for the final threshold check (c). The external equality checks that bind these public arrays to the corresponding segment proofs follow Section 5.1.4.

```
1  circuit plain-sort-glue(n, K, DEPTH)
2      public ALL_SORTED[0 .. n − 1], STARTS[0 .. K − 1], ENDS[0 .. K − 1]
3      public PARTIAL_COSTS[0 .. K − 1], T, ROOT
4      private BOUNDARY_COSTS[0 .. K − 1]
5      private SIBLINGS[0 .. K · DEPTH − 1], PATH_BITS[0 .. K · DEPTH − 1]

       (a) coverage: the segment node multisets tile the global node set
6      SORTED ← sort(ALL_SORTED)
7      for i in range(0, n):
8          require SORTED[i] = i

       (b) connection: authenticate each missing boundary arc
9      TOTAL ← 0
10     for i in range(0, K):
11         u ← ENDS[i]
12         v ← STARTS[(i + 1) mod K]
13         authenticated-read(u, v, BOUNDARY_COSTS[i],
                              n, i, ROOT, DEPTH)
14         TOTAL ← TOTAL + BOUNDARY_COSTS[i]
```

```
    (c) threshold: partial costs plus authenticated boundary costs
15  for i in range(0, K):
16      TOTAL ← TOTAL + PARTIAL_COSTS[i]
17  require TOTAL ≤ T
```

5.2.2 Product Coverage

Plain-product compresses this growing interface while keeping the external architecture of Figure 5.1 fixed. It replaces each sorted list S_i with one public field element P_i. At a common challenge point X, segment i represents the multiset of its M nodes by

$$P_i = \prod_{j=0}^{M-1} (X + R_i[j]). \tag{5.10}$$

If the K segments together contain exactly the nodes $0, \ldots, n-1$, their products satisfy

$$\prod_{i=0}^{K-1} P_i = \prod_{j=0}^{n-1} (X + j). \tag{5.11}$$

The equality replaces the global sort over $S_0 \mathbin{+\!\!+} \ldots \mathbin{+\!\!+} S_{K-1}$. Each segment proof gives P_i its meaning by deriving it from the same private R_i used for the segment's endpoints and internal costs. The verifier-side equalities then bind the certified P_i to the corresponding value consumed by recombination. The left-hand side of (5.11) is therefore the product induced by the nodes the segments actually contain, with repetitions retained.

To state the coverage condition before sampling X, define the polynomials

$$F_{\text{segments}}(Z) = \prod_{i=0}^{K-1} \prod_{j=0}^{M-1} (Z + R_i[j]), \quad F_{\text{target}}(Z) = \prod_{j=0}^{n-1} (Z + j). \tag{5.12}$$

Factorization records each node with its multiplicity, so $F_{\text{segments}} = F_{\text{target}}$ exactly when the combined segment multiset contains every target node once. Otherwise, $F_{\text{diff}}(Z) = F_{\text{segments}}(Z) - F_{\text{target}}(Z)$ is nonzero. Both source polynomials are monic of degree n, so their leading terms cancel and we

have $\deg(F_{\text{diff}}) \le n-1$. By the Schwartz–Zippel lemma of Section 3.4.1, if X is unpredictable once the candidate multiset is fixed, then

$$\Pr\left[F_{\text{diff}(X)} = 0\right] \le \frac{n-1}{q}, \tag{5.13}$$

where q is roughly 2^{254} in the field used here. One evaluation replaces the M-element node-list component of each public segment summary with one fingerprint; the route-binding values introduced next add a constant number of fields per segment. Recombination replaces the n-node sort with the K-factor left-hand product and the n-factor target product, but the guarantee is now probabilistic. If the prover can choose X after fixing a false multiset, the few roots of F_{diff} become values to search for rather than a negligible random event. Plain-product must therefore bind X to the route without relying on an interactive verifier.

5.2.3 Binding the Challenge to the Route

An interactive verifier could enforce the required order by first receiving a binding commitment to the route and then sampling X. The polynomial would be fixed before its evaluation point is revealed. Plain-product has no later verifier message, so it derives the challenge through the non-interactive mechanism of Section 3.4.2, directly from the route.

For the complete ordered route $\pi = (\pi_0, \ldots, \pi_{n-1})$, define

$$h_j = \begin{cases} 0 & \text{if } j = 0 \\ H\left(h_{j-1}, \pi_{j-1}\right) & \text{if } 1 \le j \le n \end{cases} \tag{5.14}$$

Here H is the Poseidon2 hash [52]. Let $h^{\text{term}} = h_n$ be the terminal value and derive the challenge as $X = H\left(h^{\text{term}}\right)$. Changing the route order or its contents changes the route-bound challenge except through a hash collision. A route with repeated nodes still induces a valid chain, but recombination will detect it and reject the proof.

Every route node must enter the chain. If it stopped before a suffix of the route, the prover could learn X and then choose that suffix against the known challenge. The initial value is likewise fixed at $h_0 = 0$. A prover-

controlled start would create multiple terminal values, and therefore multiple challenge trials, for the same route, Merkle openings, and costs.

Because no segment circuit sees the complete route, the chain's iterative state must cross the segment boundaries. Segment i publishes an incoming anchor IN_ANCHOR and an outgoing anchor OUT_ANCHOR, and proves that folding its private R_i from IN_ANCHOR reaches OUT_ANCHOR. The glue collects these values into IN_ANCHORS and OUT_ANCHORS and stitches the certified stretches by requiring

$$\begin{aligned} \textsc{in_anchors}[0] &= 0, \\ \textsc{in_anchors}[i+1] &= \textsc{out_anchors}[i] \quad (0 \le i < K-1), \\ \textsc{out_anchors}[K-1] &= h^{\mathrm{term}}. \end{aligned} \tag{5.15}$$

These equations connect only the copies supplied to the glue. The external verifier must also equate each anchor and product with the corresponding value certified by its segment proof, and require all $K+1$ proofs to use the same h^{term} and X. The glue checks $X = H\big(h^{\mathrm{term}}\big)$, while every segment binds its product and chain transition to the same private R_i. Together, these checks reconstruct one chain over all n nodes and evaluate every segment product at the challenge derived from that route.

5.2.4 Plain Product

Plain-product realizes the preceding product and challenge mechanisms within the external architecture already used by plain-sort. It retains the segment range, endpoint, internal-arc authentication, and partial-cost checks, together with the glue's boundary and threshold checks. The published sorted list and the local and global constrained sorts disappear. From the same private segment R_i, the segment circuit instead certifies a product fingerprint and one stretch of the route hash chain.

For segment i, the public statement is

$$\big(\mathrm{start}_i, \mathrm{end}_i, L_i, \textsc{root}, P_i, h_i^{\mathrm{in}}, h_i^{\mathrm{out}}, h^{\mathrm{term}}, X\big). \tag{5.16}$$

The endpoints, partial cost, and root retain their roles from Section 5.2.1. The product carries the new coverage fingerprint, the chain anchors bind

its challenge to the route, and h^{term} and X are common route-wide values. The private inputs remain R_i, its $M-1$ internal costs, and their Merkle authentication paths.

The range, endpoint, authenticated-read, and partial-cost constraints therefore follow Listing 5.1. Starting from h_i^{in}, the circuit folds every node of R_i through the hash chain and requires the result to equal h_i^{out}. Over the same R_i, it multiplies the factors $X + R_i[j]$ and binds the result to P_i. The circuit also checks $X = H\big(h^{\text{term}}\big)$. This check is redundant once the glue derivation and verifier-side equalities hold, but keeps each segment statement internally consistent. Because all these constraints use the same private array, the product, chain transition, endpoints, and internal costs cannot describe different segment orders. Distinctness remains global: a repeated node contributes a repeated factor to P_i and is handled by (5.11) under the product-coverage guarantee established above. Listing 5.3 records the interface and the constraints introduced by plain-product.

Listing 5.3: Plain-product segment interface. Range, endpoint, authenticated-cost, and partial-cost checks follow Listing 5.1(a), (c), and (d); its sort binding (b) is omitted. The listing shows the hash-chain (a), product (b), and challenge (c) constraints.

```
1  circuit plain-product-segment(n, M, DEPTH)
2      public START, END, PARTIAL_COST, ROOT
3      public PRODUCT, IN_ANCHOR, OUT_ANCHOR, TERMINAL, X
4      private SEGMENT[0 .. M − 1], EDGE_COSTS[0 .. M − 2]
5      private SIBLINGS[0 .. (M − 1) · DEPTH − 1]
6      private PATH_BITS[0 .. (M − 1) · DEPTH − 1]

       inherited: apply Listing 5.1(a), (c), and (d) to the inputs above
       the sorted-list binding in Listing 5.1(b) is omitted

       (a) the segment links the route hash chain
7      h ← IN_ANCHOR
8      for j in range(0, M):
9          h ← H(h, SEGMENT[j])
10     require h = OUT_ANCHOR
```

```
    (b) the published segment product matches the private nodes
11  COMPUTED_PRODUCT ← 1
12  for j in range(0, M):
13      COMPUTED_PRODUCT ← COMPUTED_PRODUCT · (X + SEGMENT[j])
14  require COMPUTED_PRODUCT = PRODUCT

    (c) the challenge is bound to the full-chain terminal
15  require X = H(TERMINAL)
```

The segment proof now certifies the product and chain anchors that replace S_i in the shared summary. It cannot join the K chain stretches or test coverage over the complete route, so these checks remain with recombination.

For every segment i, the recombination circuit receives the public summary $\left(\text{start}_i, \text{end}_i, L_i, P_i, h_i^{\text{in}}, h_i^{\text{out}}\right)$, together with the common root, threshold T, terminal h^{term}, and challenge X. Its private witness remains the K boundary costs and their Merkle authentication paths.

The circuit first stitches the anchors into one chain beginning at 0 and ending at h^{term}, then checks $X = H\left(h^{\text{term}}\right)$. At that challenge, it multiplies the K segment products and compares them with the target product over the sequence $0, \ldots, n-1$. It then authenticates the K boundary arcs under the common root, adds their costs to the segment partial costs, and enforces T. Chain stitching, challenge derivation, and the product equality replace the global sort; boundary authentication and the threshold check retain the recombination logic of Listing 5.2. Listing 5.4 records the interface and the replacement constraints.

Listing 5.4: Plain-product recombination interface. Chain stitching (a), challenge derivation (b), and product coverage (c) replace the global sort. Boundary authentication and the threshold check follow Listing 5.2(b) and (c).

```
1 circuit plain-product-glue(n, K, DEPTH)
2     public STARTS[0 .. K − 1], ENDS[0 .. K − 1], PRODUCTS[0 .. K − 1]
3     public PARTIAL_COSTS[0 .. K − 1], T, ROOT
```

```
4  public IN_ANCHORS[0 .. K − 1], OUT_ANCHORS[0 .. K − 1], TERMINAL, X
5  private BOUNDARY_COSTS[0 .. K − 1]
6  private SIBLINGS[0 .. K · DEPTH − 1]
7  private PATH_BITS[0 .. K · DEPTH − 1]

   (a) chain stitching: the supplied anchors form one chain
8  require IN_ANCHORS[0] = 0
9  for i in range(0, K − 1):
10     require IN_ANCHORS[i + 1] = OUT_ANCHORS[i]
11 require OUT_ANCHORS[K − 1] = TERMINAL

   (b) challenge derivation check
12 require X = H(TERMINAL)

   (c) coverage test: compare the segment and target products
13 COMBINED_PRODUCT ← 1
14 for i in range(0, K):
15     COMBINED_PRODUCT ← COMBINED_PRODUCT · PRODUCTS[i]
16 TARGET_PRODUCT ← 1
17 for j in range(0, n):
18     TARGET_PRODUCT ← TARGET_PRODUCT · (X + j)
19 require COMBINED_PRODUCT = TARGET_PRODUCT

   inherited: apply Listing 5.2(b) and (c) to the inputs above
```

The glue proves these conditions over its public summary copies, so the verifier completes the external binding contract of Section 5.1.4. The verifier does not recompute the chain or product equality. The glue establishes those facts, while the external equalities ensure that it uses the segment-certified values. With these bindings, the accepted package of K segment proofs and one glue proof establishes the existence of K valid segments whose summaries satisfy the chain, coverage, connection, and threshold checks of the recombination contract.

5.2.5 Plain Construction Trade-offs

Both plain constructions use the same external topology: K segment proofs, one glue proof, and verifier-side equalities that bind their public summary copies. For either variant under a sequential schedule,

$$\text{peak-mem}_{\text{external}} = \max\bigl(\text{peak-mem}_{\text{segment}},\ \text{peak-mem}_{\text{glue}}\bigr). \quad (5.17)$$

The segments remain independently schedulable, but neither a lower end-to-end peak nor a wall-clock speedup follows before the glue stage is included. The two variants differ in the coverage-specific interface and the work used to check it. Their endpoints, partial costs, root, threshold, boundary checks, and proof topology are common and omitted from Table 5.4.

Table 5.4: Coverage-specific differences between plain-sort and plain-product. Shared route and cost fields, boundary checks, and proof topology are omitted.

Dimension	Plain-sort	Plain-product
Coverage evidence	$S_i = \text{sort}(R_i)$	P_i, h_i^{in}, and h_i^{out}, with common (h^{term}, X)
Coverage-specific public surface	n node identifiers	K products, $2K$ anchors, and (h^{term}, X)
Coverage work	K local M-node sorts and one global n-node sort	Segment products and n chain hashes; the glue forms the K-factor and n-factor products
Coverage guarantee	Exact multiset equality	Failure at most $Q(n-1)/q$ under the route-bound random-oracle model
Additional disclosure	Exact membership of each segment	Tests for candidate segment multisets and orders

The two public interfaces share substantial disclosure, but in different forms. Public endpoints reveal the segment boundaries, while each L_i reveals one segment's internal cost. Since C is public, the endpoints also determine every boundary cost. The verifier can therefore recover

$$\sum_{i=0}^{K-1} L_i + \sum_{i=0}^{K-1} C_{\text{end}_i,\text{start}_{(i+1)\bmod K}} \quad (5.18)$$

even though relation (1.2) asks only whether this cost is at most T. Zero-knowledge hides the values that remain in the witnesses; it does not conceal these statement fields.

Plain-sort additionally reveals the exact node multiset S_i of every segment. Once its two endpoints are fixed, at most $(M-2)!$ internal orders remain, and the public matrix and partial cost can reject candidates with the wrong cost. Plain-product replaces this explicit membership with two confirmation surfaces. Given public X, anyone can evaluate $\prod_j (X + a_j)$ for a candidate multiset and compare it with P_i. The anchors similarly test a candidate order by folding it from h_i^{in} and comparing the result with h_i^{out}. Plain-product therefore avoids enumerating segment membership while still allowing plausible guesses to be checked.

Every recombination input and the matrix C are public in both variants, so the verifier could perform the glue relation directly. It could run the selected coverage check, recover the boundary costs, aggregate the full cost, and enforce T. The glue proof delegates this public computation succinctly. Removing it would move those checks to the verifier rather than eliminate them.

The monolithic circuit applies coverage, arc authentication, and the threshold to one route witness and exposes only $(\textsc{root}, T)$. No cross-proof consistency condition is needed. Under backend knowledge soundness, an accepted composite package instead yields K valid segment witnesses and a valid glue witness. The authenticated keys, common instance, and external equalities bind the glue's summary copies to the values certified by those segments. Their ordered concatenation then witnesses relation (1.2) by the argument of Section 5.1.3.

5.3 Committed Constructions

The committed constructions retain the recombination contract and $K + 1$ proofs. Segment i derives a summary V_i but publishes only its commitment digest D_i. The glue receives the opening as private witness, checks the digest,

and applies the same coverage, connection, and threshold constraints as before.

The *committed-sort* construction applies this layer to the exact sort-based construction; *committed-product* applies it to the hash-chain and grand-product construction. Their coverage mechanisms are inherited from Section 5.2; only the treatment of their segment summaries at the public interface changes.

5.3.1 Commitment Layer

The Merkle root in Section 4.3 binds the public matrix C while replacing its n^2 entries with one public value. A segment commitment instead protects a private recombination summary. As distinguished in Section 3.3.2, it must provide *binding*, so the segment and glue cannot use different summaries, and *hiding*, so its public value does not reproduce the disclosure it removes.

For a value sequence $V = (v_0, \ldots, v_{\ell-1})$ and a blinding value r, define its fold through the two-input Poseidon2 hash as

$$d_j = \begin{cases} r & \text{if } j = 0 \\ H(d_{j-1}, v_{j-1}) & \text{if } 1 \le j \le \ell \end{cases}. \tag{5.19}$$

The final value d_ℓ is the commitment to V, written $\mathrm{Commit}_r(V)$. For segment i, V_i denotes the summary, r_i its secret blinding value, (V_i, r_i) its opening, and $D_i = \mathrm{Commit}_{r_i}(V_i)$ its public digest. Each circuit fixes the length and order of V_i, giving the opening one interpretation. Treating Poseidon2 as a random oracle in the sense of Section 3.3.1, a fresh unknown r_i makes D_i computationally hiding because testing a summary guess also requires guessing the blinding. Collision resistance provides binding: two distinct openings for the same digest would induce a collision in H.

The host-side builder samples 128 fresh random bits from the operating system for each r_i and embeds them in the field. The implementation uses the same two-input Poseidon2 compression for Merkle nodes, route-chain links, and commitment folds without prepending the domain tags described in Section 3.3.1. Fixed circuit interfaces remove ambiguity within each

construction, but explicit domain separation remains an unimplemented deployment hardening measure.

The segment circuit constrains (V_i, r_i) to D_i^{seg}, the glue constrains $(\tilde{V}_i, \tilde{r}_i)$ to D_i^{glue}, and the verifier equates the two public digest copies after verifying the $K + 1$ proofs. The verifier also applies the shared-root and instance checks of Section 5.1.4. Figure 5.2 shows which parts of this binding flow are private, public, and external to the circuits.

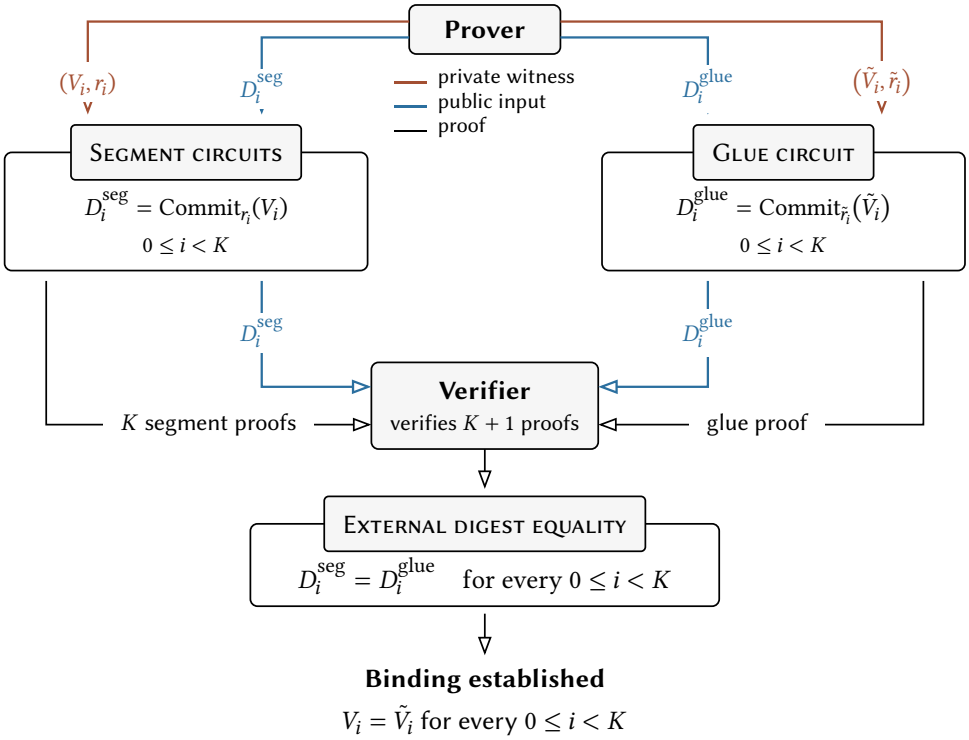


Figure 5.2: Committed specialization of Figure 5.1. Each circuit binds a private opening to its public digest copy, and the verifier requires the two copies to match.

The binding flow is identical across variants; only V_i changes. Table 5.5 lists the two sequences and their fixed lengths.

Table 5.5: Committed sequence for each variant.

VARIANT	COMMITTED SEQUENCE	LENGTH
sort	$V_i = (R_i[0], \ldots, R_i[M-1], L_i)$	$M+1$
product	$V_i = \left(P_i, h_i^{\text{in}}, h_i^{\text{out}}, \text{start}_i, \text{end}_i, L_i\right)$	6

Committed-sort can use the hidden visit order to establish coverage and derive the endpoints. Committed-product carries only the six derived values consumed by its existing recombination checks.

5.3.2 Committed Sort

Committed-sort replaces the public plain-sort summary with a commitment to the segment in visit order and its internal partial cost:

$$D_i = \text{Commit}_{r_i}(R_i[0], \ldots, R_i[M-1], L_i). \tag{5.20}$$

Plain-sort published $S_i = \text{sort}(R_i)$ because exposing R_i would also expose the internal traversal. That representation required every segment circuit to sort its private route and publish its endpoints separately, since sorting removed their positional meaning. Here the opening remains private. Both circuits can therefore use the same visit-ordered R_i: the segment binds it to D_i, while the glue sorts it for global coverage and reads its first and last positions for the boundary arcs. The K local sorts and separate endpoint fields disappear; the one global n-node sort remains.

The segment circuit exposes only (ROOT, D_i). Its witness contains R_i, the internal costs and their Merkle paths, and r_i. It retains the range and internal-arc checks of plain-sort, derives L_i, and constrains (5.20). Listing 5.5 shows the resulting interface and the new commitment block.

Listing 5.5: Committed-sort segment interface. Range and authenticated internal-cost checks follow Listing 5.1(a) and (d); the listing shows the commitment to the visit-ordered nodes and their derived partial cost.

```
1 circuit committed-sort-segment(n, M, DEPTH)
2 |   public ROOT, COMMITMENT
```

```
3   private SEGMENT[0 .. M − 1], EDGE_COSTS[0 .. M − 2]
4   private SIBLINGS, PATH_BITS, BLIND

    inherited: range-check the segment as in Listing 5.1(a)
    inherited: authenticated internal arc costs and derive PARTIAL_COST as
    in Listing 5.1(d)

    bind the ordered nodes and partial cost to the public digest
5   VALUES ← SEGMENT ⧺ [PARTIAL_COST]
6   require COMMITMENT = commit(BLIND, VALUES)
```

The glue exposes (ROOT, T, $D_0, \ldots, D_{K-1}$) and receives all K openings privately. After checking them, it concatenates the ordered segments and sorts their n nodes to establish exact coverage. Their fixed positions identify each boundary arc, so the glue authenticates those K costs, combines them with the opened partial costs, and enforces T. Listing 5.6 specifies this complete flow.

Listing 5.6: Committed-sort glue circuit: private openings (a), exact coverage (b), positional boundary authentication (c), and threshold enforcement (d).

```
1  circuit committed-sort-glue(n, K, M, DEPTH)
2    public ROOT, T, COMMITMENTS[0 .. K − 1]
3    private ALL_NODES[0 .. n − 1], PARTIAL_COSTS[0 .. K − 1],
4    private BLINDS[0 .. K − 1]
5    private BOUNDARY_COSTS[0 .. K − 1], SIBLINGS, PATH_BITS

     (a) privately open every public commitment
6    for i in range(0, K):
7      VALUES ← ALL_NODES[i · M .. (i + 1) · M − 1] ⧺
                          [PARTIAL_COSTS[i]]
8      require COMMITMENTS[i] = commit(BLINDS[i], VALUES)

     (b) establish exact global coverage
9    SORTED ← sort(ALL_NODES)
```

```
    for j in range(0, n):
        require SORTED[j] = j

    (c) derive positional endpoints and authenticate every boundary arc
    TOTAL ← 0
    for i in range(0, K):
        u ← ALL_NODES[i · M + M − 1]
        v ← ALL_NODES[((i + 1) mod K) · M]
        authenticated-read(u, v, BOUNDARY_COSTS[i], n, i, ROOT, DEPTH)
        TOTAL ← TOTAL + BOUNDARY_COSTS[i] + PARTIAL_COSTS[i]

    (d) enforce the public threshold
    require TOTAL ≤ T
```

5.3.3 Committed Product

Committed-product retains plain-product's fixed-size recombination summary and hides its six segment-specific values behind one digest:

$$D_i = \mathrm{Commit}_{r_i}\left(P_i, h_i^{\mathrm{in}}, h_i^{\mathrm{out}}, \mathrm{start}_i, \mathrm{end}_i, L_i\right). \tag{5.21}$$

The fixed order in (5.21) is part of the circuit interface. From the same private R_i, the segment derives P_i and the chain anchors, takes the endpoints from the first and last positions, and computes L_i from the authenticated internal arcs. The glue consumes these six consequences rather than the route nodes themselves. Binding them to D_i therefore gives the glue the segment-certified coverage, chain, boundary, and cost summary through an opening whose length is independent of M.

The segment exposes $(\textsc{root}, X, D_i)$. Its incoming chain anchor and blinding join the route, internal costs, and Merkle paths in the witness; the terminal h^{term} is no longer a segment input. The circuit retains the local checks of plain-product, derives the six values from one R_i, and folds them into D_i. Listing 5.7 keeps the complete interface and the constraints that form the committed summary.

Listing 5.7: Committed-product segment interface. Local checks follow Listing 5.1(a) and (d); the shown blocks derive and commit the six-value summary.

```
1  circuit committed-product-segment(n, M, DEPTH)
2      public ROOT, X, COMMITMENT
3      private SEGMENT[0 .. M − 1], EDGE_COSTS[0 .. M − 2],
4      private SIBLINGS, PATH_BITS
5      private IN_ANCHOR, BLIND

       inherited: range-check the segment as in Listing 5.1(a)
       inherited: authenticate its internal arcs and derive PARTIAL_COST as
       in Listing 5.1(d)

       (a) derive the route-chain transition and segment product
6      h ← IN_ANCHOR
7      PRODUCT ← 1
8      for j in range(0, M):
9          h ← H(h, SEGMENT[j])
10         PRODUCT ← PRODUCT · (X + SEGMENT[j])

       (b) bind all six derived summary values to the public digest
11     VALUES ← [PRODUCT, IN_ANCHOR, h,
                                SEGMENT[0], SEGMENT[M − 1], PARTIAL_COST]
12     require COMMITMENT = commit(BLIND, VALUES)
```

The glue exposes $(\text{ROOT}, T, X, D_0, \ldots, D_{K-1})$ and receives the six-value openings, blindings, and terminal h^{term} as witness. It checks every digest, stitches the private anchors from 0 to h^{term}, requires $X = H(h^{\text{term}})$, and recombines the private products for global coverage. The opened endpoints and partial costs then feed the same boundary and threshold checks as plain-product. Listing 5.8 specifies the new opening, chain, and coverage logic.

Listing 5.8: Committed-product glue interface. The listing shows private openings (a), chain and challenge binding (b), and product coverage (c); boundary authentication and threshold enforcement follow Listing 5.2(b) and (c) over the opened endpoints and partial costs.

```
1  circuit committed-product-glue(n, K, DEPTH)
2      public ROOT, T, X, COMMITMENTS[0 .. K − 1]
3      private PRODUCTS[0 .. K − 1], IN_ANCHORS[0 .. K − 1],
4      private OUT_ANCHORS[0 .. K − 1]
5      private STARTS[0 .. K − 1], ENDS[0 .. K − 1], PARTIAL_COSTS[0 .. K − 1]
6      private BLINDS[0 .. K − 1], TERMINAL
7      private BOUNDARY_COSTS, SIBLINGS, PATH_BITS

       (a) privately open every public commitment
8      for i in range(0, K):
9          VALUES ← [PRODUCTS[i], IN_ANCHORS[i], OUT_ANCHORS[i],
                                 STARTS[i], ENDS[i], PARTIAL_COSTS[i]]
10         require COMMITMENTS[i] = commit(BLINDS[i], VALUES)

       (b) stitch the route chain and bind its terminal to the public challenge
11     require IN_ANCHORS[0] = 0
12     for i in range(0, K − 1):
13         require IN_ANCHORS[i + 1] = OUT_ANCHORS[i]
14     require OUT_ANCHORS[K − 1] = TERMINAL
15     require X = H(TERMINAL)

       (c) recombine the segment products and test global coverage
16     COMBINED_PRODUCT ← 1
17     for i in range(0, K):
18         COMBINED_PRODUCT ← COMBINED_PRODUCT · PRODUCTS[i]
19     TARGET_PRODUCT ← 1
20     for j in range(0, n):
21         TARGET_PRODUCT ← TARGET_PRODUCT · (X + j)
22     require COMBINED_PRODUCT = TARGET_PRODUCT
```

inherited: apply Listing 5.2(b) and (c) to the opened endpoints and partial costs

5.3.4 Committed Construction Trade-offs

Both committed variants retain the external topology of Section 5.1.4: K segment proofs, one glue proof, and verifier-side equalities that bind their shared public inputs. Relative to the plain constructions, digest equalities replace plaintext-summary equalities. This interface change can be summarized as

$$\text{public summary} \longrightarrow (\text{private opening}, \text{public digest}) . \tag{5.22}$$

The segment derives a summary and folds it into D_i; the glue receives the same summary and blinding as witness, repeats the fold, and applies the inherited coverage, boundary, and threshold checks. Commitment binding makes the digest equalities meaningful, as established in Section 5.3.1.

Committed-sort folds $M + 1$ values in each segment and repeats every fold in the glue. Since $K \cdot M = n$, this gives $2K(M + 1) = 2(n + K)$ fold steps. It removes the K local sorts used by plain-sort, but retains the global sort over n nodes and node-dependent commitment work. Committed-product folds six values per segment and repeats those folds in the glue, for $2K \cdot 6 = 12K$ steps. Its commitment work is therefore fixed per segment, while the chain, product, and target-product checks remain.

The digests move node membership, products, chain anchors, endpoints, and partial costs from the public interface into the glue witness. Zero-knowledge hides those witnesses, while computational hiding prevents testing a guessed summary against a digest without knowing its blinding. Because the endpoints and partial costs are no longer public, the public matrix C no longer suffices to recover the boundary costs or the exact route cost.

Both variants reveal K through the K digests and the $K + 1$ proof topology. Committed-product also publishes the route-derived challenge X, which allows a guessed complete ordered route to be tested by recomputing its chain. Committed-sort has no corresponding unblinded route-derived

value. The monolithic construction exposes only (ROOT, T); committed-sort adds the topology and digests, while committed-product additionally exposes X. Commitments therefore improve structural privacy over the plain constructions without reproducing the monolithic public surface.

Soundness follows the same recombination contract as before. The external digest equalities and commitment binding force the glue to use the summaries certified by the segments. The argument of Section 5.1.3 then yields one route witness satisfying coverage, arc authentication, and the threshold.

5.4 Recursive Construction

The recursive construction replaces external composition with an *outer* relation. The outer circuit receives the K segment proofs and their public inputs as private witness, verifies each proof, and applies the recombination contract of Section 5.1.3 to the certified inputs. It produces one outer proof with public statement (ROOT, T), which is the only proof checked by the final verifier.

The proofs authenticate the $n - K$ internal arcs, while the outer circuit restores global coverage, authenticates the K boundary arcs, aggregates all costs, and enforces T. The construction therefore requires both a segment relation whose public-input vector the outer circuit can certify and consume, and an outer relation that performs recombination over those certified inputs.

5.4.1 Inner Circuit Selection

Each segment proof now serves as an inner proof. Its public-input vector must be supplied to the in-circuit verifier and constrained within the outer circuit, once for every segment. Since recursion already invokes K proof verifiers, the selected interface should avoid an additional term that grows with the segment length M.

The commitment layer of Section 5.3.1 hides summaries that remain public under external composition. Here each inner proof and its nominally public inputs become private witness to the outer proof, whose zero-knowledge guarantee hides them from the final verifier. A committed inner

relation would still require the outer circuit to receive the opening and check its digest before recombination. That alternative trades fewer inner public inputs for commitment-opening constraints and is not evaluated here; this construction selects between the direct plain-sort and plain-product relations.

A plain-sort proof carries M sorted node identifiers, two endpoints, one partial cost, and one root. Plain-product instead carries the fixed nine-field public-input vector specified in Section 5.2.4:

$$x_i^{\text{inner}} = \left(\text{start}_i, \text{end}_i, L_i, \textsc{root}_i, P_i, h_i^{\text{in}}, h_i^{\text{out}}, h_i^{\text{term}}, X_i\right). \tag{5.23}$$

Across all K segments, the corresponding interface widths are

$$\text{plain-sort:} \quad K(M+4) = n + 4K, \quad \text{plain-product:} \quad 9K. \tag{5.24}$$

The selection concerns dependence on M. Plain-sort contributes an n-entry node-list term, whereas plain-product keeps this part of the outer workload proportional to K. We therefore select the plain-product segment of Listing 5.3 as the inner relation. The outer circuit remains nonconstant: it must still invoke K proof verifiers and perform global recombination, including the target product involving n factors.

This choice carries plain-product's route-bound coverage guarantee into recursion. Its failure probability remains at most $Q(n-1)/q$ under Section 5.2.3 and is negligible for feasible Q at the field and instance sizes considered here. Recursion therefore does not recover the exact coverage guarantee of sorting.

With the inner relation fixed, Figure 5.3 shows how its proofs flow through recursive composition.

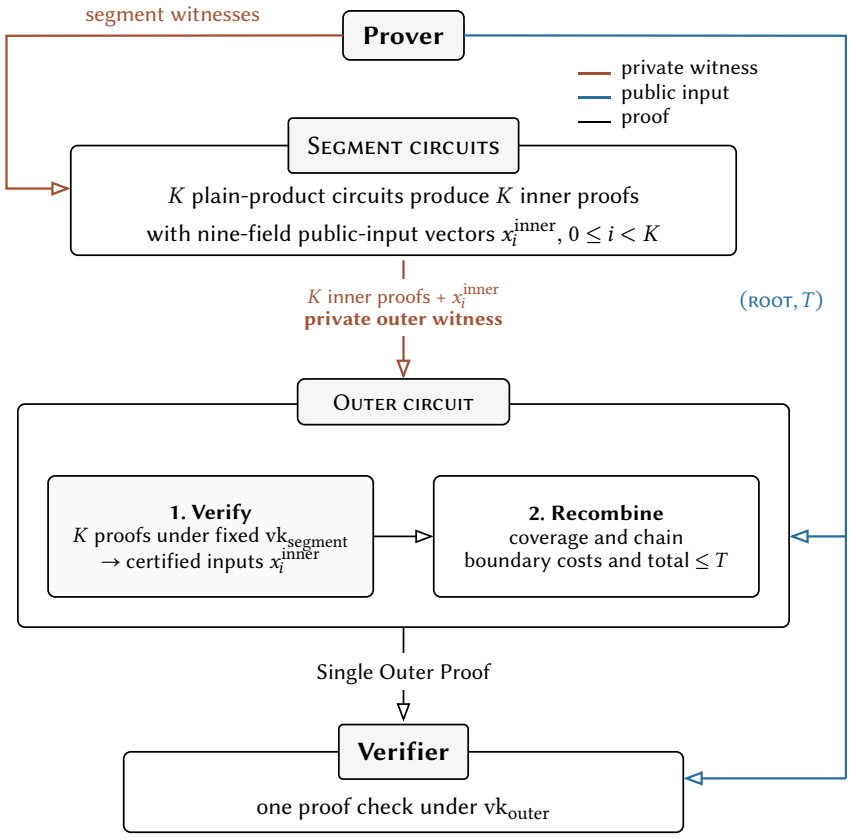


Figure 5.3: Recursive composition of the segment proofs. Each inner proof exposes a nine-field public-input vector x_i^{inner} at the inner-proof boundary; the proof and its vector together become private witness to the outer circuit. The outer circuit verifies the proofs under the fixed segment key, recombines their certified inputs, and emits a single outer proof with public interface (ROOT, T).

5.4.2 Outer Circuit

The outer circuit exposes only the matrix root and threshold (ROOT, T). Its witness contains the K segment proofs, their nine-field public-input vectors x_i^{inner} from (5.23), bounded representations of the endpoints and partial costs, and the K boundary costs with their Merkle paths. Acceptance means that the proofs certify segments that recombine into a route whose costs are authenticated against ROOT and whose total is at most T.

Each witnessed x_i^{inner} is initially only a claimed input to an inner proof. The outer circuit therefore verifies the corresponding proof against x_i^{inner} under the fixed segment key for shape (n, M, DEPTH). A successful check

certifies the vector's endpoints, partial cost, root, product, chain anchors, terminal, and challenge under the intended plain-product segment relation. Without this check, the prover could choose values that satisfy recombination but arise from no valid segment.

The certified endpoints and partial costs are field elements, whereas Merkle indices and bounded cost arithmetic require bounded integers. The witness therefore supplies bounded mirrors, and the circuit constrains each mirror to equal its proof-certified field element. These equalities prevent the prover from using one value for inner-proof verification and another for the bounded Merkle and cost operations.

Recombination then uses only these certified inputs. Following the plain-product glue of Listing 5.4, the circuit requires every segment root to equal ROOT and all terminals and challenges to agree. It stitches the chain anchors from 0 to the common terminal h^{term}, checks $X = H(h^{\text{term}})$, and multiplies the certified segment products to establish coverage of $0, \ldots, n-1$. Finally, it authenticates the K boundary costs using the certified endpoints, adds them to the certified partial costs, and enforces the threshold T.

In Listing 5.9, *verify-proof* abstracts the backend's in-circuit verification gadget; its key arguments are compile-time constants authenticated as described in Section 5.4.3.

Listing 5.9: The recursive outer circuit. Each inner proof is checked against its witnessed public-input vector under the fixed segment key (a). Tuple equalities bind the proof-certified endpoints and partial cost to bounded mirrors and require common ROOT, h^{term}, and X (b). The inherited plain-product recombination constraints then consume only certified inputs (c). Only the root and threshold are public.

```
1 circuit recursive-outer(n, K, DEPTH)
2     constant vk_segment, vkhash_segment
3     public ROOT, T
4     private PROOFS[0 .. K − 1], INNER_INPUTS[0 .. K − 1]
5     private STARTS[0 .. K − 1], ENDS[0 .. K − 1]
6     private PARTIAL_COSTS[0 .. K − 1]
7     private BOUNDARY_COSTS[0 .. K − 1]
8     private SIBLINGS[0 .. K · DEPTH − 1], PATH_BITS[0 .. K · DEPTH − 1]
```

```
    (a) certify and unpack each inner public-input vector
9   for i in range(0, K):
10      verify-proof(vk_segment, vkhash_segment, PROOFS[i], INNER_INPUTS[i])
11      (STARTS_F[i], ENDS_F[i], PARTIAL_COSTS_F[i]) ←
        INNER_INPUTS[i][0 .. 2]
12      (ROOTS[i], PRODUCTS[i]) ← INNER_INPUTS[i][3 .. 4]
13      (IN_ANCHORS[i], OUT_ANCHORS[i]) ← INNER_INPUTS[i][5 .. 6]
14      (TERMINALS[i], CHALLENGES[i]) ← INNER_INPUTS[i][7 .. 8]

    (b) bind the certified fields to the outer relation
15  for i in range(0, K):
16      require (STARTS[i], ENDS[i], PARTIAL_COSTS[i])
                    = (STARTS_F[i], ENDS_F[i], PARTIAL_COSTS_F[i])
17      require (ROOTS[i], TERMINALS[i], CHALLENGES[i])
                    = (ROOT, TERMINALS[0], CHALLENGES[0])

    (c) recombine only the certified values
    inherited: apply Listing 5.4(a)–(c) to the products and chain values
    inherited: apply Listing 5.2(b)–(c) to the bounded values and boundary
    witness under ROOT and T
```

5.4.3 Proof Verification

The final verifier receives one outer proof and the public inputs (ROOT, T). The root must derive from the agreed matrix and T must equal the intended threshold. Two keys govern this verification at different levels: the outer circuit uses the embedded *segment key* $\text{vk}_{\text{segment}}$ to verify the witnessed inner proofs, while the final verifier uses the *outer key* vk_{outer} to verify the proof it receives.

In the external constructions, the final verifier selects the segment and glue keys and checks those proofs directly. Recursion delegates segment verification to the outer circuit, so the final verifier no longer selects the segment key directly. The key and its expected hash must therefore be

compile-time constants rather than prover inputs, and the outer key must authenticate that fixed choice.

For fixed $(n, K, \textsc{depth})$, the canonical plain-product segment circuit is compiled first. Its verification key and expected hash are embedded when the outer circuit is compiled, producing the canonical outer key. The final verifier obtains that key through the authenticated mechanism of Section 3.5.3 and rejects any key supplied unauthenticated with the proof. A different embedded segment key produces a different outer key, so the resulting proof does not verify under the canonical one.

5.4.4 Recursive Construction Trade-offs

Recursion folds the K segment-proof checks and the recombination relation into one outer proof. The external constructions instead expose K segment proofs and one glue proof, leaving the verifier to check all $K + 1$ proofs and reconcile their public copies. The recursive inner proofs and x_i^{inner} become private outer witness. Refined monolithic sort has the same final proof count, but proves a whole-route witness directly rather than verifying inner proofs and recombining their claims. The recursive prover must first generate the K inner proofs and then prove the dependent outer circuit.

The endpoints, partial costs, products, chain anchors, h^{term}, and X remain witness values of the outer proof, leaving the final public interface $(\textsc{root}, T)$ as in refined monolithic sort. Unlike the externally recombined variants, this interface contains no per-segment summaries, digests, or route-derived challenge. The outer proof's zero-knowledge guarantee protects those values under the public-interface boundary of Section 3.5.1, so a separate segment-commitment layer is unnecessary.

This protection concerns the final verifier. The outer prover necessarily holds the inner proofs and their nominally public inputs because they form its witness. The segment count K also remains visible through the selected outer circuit and key. It identifies the circuit parameterization but reveals no segment boundaries, node membership or order, summaries, or costs.

The segment proofs retain the independent generation established in Section 5.1.1, but the outer proof cannot begin until all of them exist. Each

segment circuit contains $M = n/K$ route nodes rather than n. Under the sequential single-worker schedule used for comparison with the external constructions, complete recursive peak memory is

$$\text{peak-mem}_{\text{recursive}} = \max\Big(\text{peak-mem}_{\text{segment}}, \text{peak-mem}_{\text{outer}}\Big). \quad (5.25)$$

The outer stage therefore adds a dependent memory and proving-time floor: it executes K in-circuit proof verifiers together with the recombination relation.

Under backend knowledge soundness and the authenticated outer key, an accepted outer proof yields inner proofs that verify under the intended plain-product segment key. Soundness of those proofs yields valid segment witnesses and their certified inputs. Because the outer relation verifies and consumes the same x_i^{inner}, the recombination contract of Section 5.1.3 binds those segments into one route claim. The external constructions obtain the corresponding binding through verifier-side equality checks over plaintext or digest copies; recursion needs neither those external equalities nor segment-commitment binding.

Refined monolithic sort and recursion both authenticate all n selected arcs under ROOT and enforce the complete threshold. Their coverage guarantees differ. Monolithic sort establishes exact coverage algebraically, whereas recursion inherits plain-product's route-bound product check, its random-oracle assumption, and the failure bound $Q(n-1)/q$ from Section 5.2.3. Recursion additionally depends on the authenticated inner-to-outer key chain and correct in-circuit proof verification. Both constructions retain the backend knowledge-soundness and Merkle-authentication assumptions.

5.5 The Architectural Frontier

Decomposition replaces one whole-route proving unit with K segment units and a recombination stage. Recombination must restore global coverage, authenticate the K boundary arcs, aggregate all costs and enforce T, and bind the independently certified segment values into one statement. The five constructions satisfy this common contract but differ in their coverage

evidence, binding topology, and composition disclosure. Table 5.6 organizes those choices.

Two coverage mechanisms divide the rows. Sorting restores exact coverage; route-bound product fingerprints replace the growing node-list interface with the computational guarantee of Section 5.2.3. Commitments change what external composition reveals without changing its $K + 1$-proof topology. Recursion changes both the topology and the final public surface: it hides per-segment values behind one outer proof while moving K proof verifications and recombination into a dependent outer proving stage.

Table 5.6: Architectural frontier of the five composite constructions. All authenticate the K boundary arcs, aggregate the complete cost, and enforce T; the common public instance (ROOT, T) is omitted from the disclosure column.

Construction	Coverage restoration	Binding and topology	Composition disclosure
plain sort	Exact global sort	Public equalities; $K + 1$ proofs	Node sets, endpoints, and partial costs
plain product	Grand product at X	Public equalities; $K + 1$ proofs	Endpoints, costs, and testable summaries
committed sort	Exact global sort	Digest equalities; $K + 1$ proofs	K blinded digests
committed product	Grand product at X	Digest equalities; $K + 1$ proofs	K digests and public X
recursive	Grand product at X	In-circuit verification; one final proof	No per-segment values; K remains architectural metadata

The five constructions now have defined coverage guarantees, recombination topologies, and disclosure profiles. Chapter 6 tests whether their smaller segment proving units retain an advantage after the stitching tax, interpreting the measured costs against those disclosure differences.

6 Evaluation

Chapters 4 and 5 developed the refined monolithic baseline and five composite architectures. This chapter evaluates them under one proving stack, measuring aggregate constrained work, peak prover memory, completion time under sequential and distributed schedules, and final-verifier load. Because the constructions differ in both coverage and public-interface disclosure, costs are ranked directly only where the coverage mechanism and structural-privacy level are held fixed.

6.1 Methodology

The evaluation compares the architectures under one fixed proving stack and matched inputs. At each route length, every construction receives the same graph and route, while its segment, recombination, and recursive stages are measured as separate proving tasks. The later sections compose these measurements according to the dependencies of each architecture.

6.1.1 Experimental Setup

All measurements were collected on one consumer laptop, with Barretenberg using its default native parallelism. The resulting times and capacity limits characterize this host and toolchain, under which every construction is measured. Table 6.1 records the fixed configuration.

Noir compiles each circuit to ACIR, which Barretenberg lowers to UltraHonk under the pinned releases in the table [45], [46]. For each n,

seed 42 determines one complete symmetric Euclidean graph and one valid Hamiltonian cycle of cost w.

Table 6.1: Hardware and software configuration used for the evaluation.

Component	Configuration
Processor	11th Gen Intel Core i7-1165G7; 4 cores, 8 threads; 2.80 GHz nominal, 4.70 GHz maximum
Memory	16 GiB installed, approximately 15 GiB visible to the operating system; 4 GiB swap
Storage	512 GB Intel NVMe solid-state drive
Operating system	Ubuntu 24.04 LTS, x86-64
Circuit compiler	Noir and Nargo 1.0.0-beta.20, commit b4236c1957d0c26c
Proving backend	Barretenberg 5.0.0-nightly.20260324; UltraHonk
Recursive library	bb_proof_verification, tag 5.0.0-nightly.20260518
Auxiliary tools	Python 3.12 for orchestration; Rust for proof-compatible off-circuit data construction

The public threshold is $T = \lceil 1.1w \rceil$. This gives every benchmark a known feasible witness. At fixed n, all constructions and runs reuse the same graph, route, matrix commitment, and authentication data. The comparison therefore controls instance choice to study architectural behavior; it does not estimate performance over a distribution of graphs.

The evaluation varies the route length n and, for composite proofs, the number of equal-length segments K. The segment length and Merkle authentication depth follow as

$$M = \frac{n}{K}, \quad d = \lceil \log_2(n^2) \rceil, \tag{6.1}$$

where M is the number of nodes per segment and d is the authentication-path depth over the n^2 matrix entries. The composite grid uses $K \in \{2, 4, 8\}$. Its route lengths up to 1000 are $n \in \{48, 96, 128, 256, 512, 768, 1000\}$; the larger checkpoints are $n \in \{1504, 2000, 2504, 3000, 4000, 5000\}$.

Every selected route length is divisible by eight; 1504 and 2504 are used in place of 1500 and 2500. Monolithic measurements span $n = 8$ to $n = 4000$, with denser sampling at smaller sizes. Monolithic cells have three runs through $n = 1000$ and two at the larger checkpoints; each external cell has three runs and each recursive cell has two. Measurements that exceed the host's capacity are reported as unavailable rather than extrapolated.

6.1.2 Measurement Protocol

Each circuit is compiled and its verification key generated once per parameterization. The pipeline then generates the witness, produces the proof, and verifies it. Segment, glue, inner, and outer invocations remain separate in the raw data so that later sections can compose them according to each architecture. Compilation and key generation are setup costs rather than recurring online prover time.

The recorded metrics cover the compiler, backend, and verifier interfaces. ACIR opcodes measure the compiler-level circuit, while UltraHonk gates measure its backend expansion. Public-input slots and proof bytes describe the verifier-facing artifacts. A monotonic high-resolution timer records compilation, witness generation, proving, and verification wall-clock intervals, including the tools' normal internal parallelism. Memory is the largest checkpoint printed by Barretenberg during one proving invocation, reported as *maximum Barretenberg-reported proving memory* rather than operating-system resident-set size.

Gate counts, opcode counts, public-input counts, and proof sizes are deterministic for a fixed parameterization and toolchain. Wall-clock measurements can contain system noise, so their central curve uses the smallest observation among the repeated successful runs and retains the

full observed minimum–maximum interval where the plot permits it. This lower envelope estimates the least-contended execution observed on the benchmark host, not expected user time.

Memory uses the median of the run-level maximum checkpoints, with the observed range retained even where it collapses onto the central curve. Complete architectural quantities are derived within each matched run before the corresponding timing or memory summary is selected, so no result combines components taken from different runs.

The measured boundary excludes graph and route generation, Merkle construction, authentication-path extraction, input serialization, verification-key derivation, and deployment orchestration. Distributed completion times compose isolated component measurements according to the architectural dependencies; no networked cluster is measured, and communication and coordination remain outside the model.

6.2 Correctness Testing

Before measuring performance, each implementation was exercised on a valid reference instance and targeted negative cases derived from Chapters 4 and 5. Each case changes one value or required relationship while keeping the remaining inputs consistent, so rejection isolates the relevant boundary: circuit constraints, external reconciliation, or recursive verification. In external composition, individually valid segment and glue proofs can still form an inconsistent package and are rejected only during reconciliation. These development-time checks exercise the evaluated code; no formal validation procedure or maintained test suite was used.

Every valid reference was accepted, and each manipulation in Table 6.2 was rejected at the intended boundary. These results provide useful sanity checks that the implemented constraints and composition interfaces match their specifications. Cryptographic soundness is not formally derived from them; it still relies on the soundness of UltraHonk, the collision resistance of Poseidon2, commitment binding, and the analytical error bound for product coverage.

Table 6.2: Targeted adversarial manipulations and the enforcement boundary exercised by each. Valid reference cases were accepted, while every listed manipulation was rejected at the indicated boundary.

Protected property	Targeted manipulation	What acceptance would permit	Rejection boundary
Route validity	Repeat one node and omit another, or supply an out-of-range identifier	A non-Hamiltonian route	Range and permutation/ coverage constraints
Authenticated costs	Alter an edge cost, path direction, sibling value, or root	A cost not drawn from the committed matrix	Merkle index and root checks
Cost aggregation and threshold	Change a partial cost, or set T below the authenticated total	An underreported cost or an infeasible route	Local summation and global threshold constraints
Global coverage	Place a node in two segments and omit another	Locally valid segments that are not a global permutation	Glue or outer sort/product check
Boundary consistency	Alter an endpoint, boundary cost, or boundary authentication path	Segments that do not connect into the authenticated cycle	Recombination topology and Merkle checks
Product and chain binding	Alter P_i, a chain anchor, or the common challenge while retaining the other values	A summary not derived from the certified ordered segments	Product, chain, and challenge constraints
External proof binding	Combine valid segment proofs for one route with a valid recombination proof for another	Separately valid proofs that certify inconsistent statements	Post-verification reconciliation
Commitment binding	Alter a hidden summary or opening while retaining its digest, or disagree on the two digest copies	Segment and glue proofs using different concealed values	Commitment recomputation and digest equality
Recursive binding	Alter an inner proof or summary, or substitute an inner relation bound to a different key	An uncertified summary or a proof of the wrong relation	In-circuit proof verification and canonical outer key

6.3 Monolithic Baseline

Chapter 4 developed *monolithic-merkle-sort* as the refined monolithic baseline, addressing the design part of the first research question. We now measure how its circuit and prover requirements grow with n under the environment and reporting conventions of Section 6.1.

6.3.1 Circuit Growth

The complete route remains in one circuit: coverage sorts n node identifiers, while cost binding authenticates n selected arcs through paths of depth $\lceil \log_2(n^2) \rceil$. Figure 6.1 reports the resulting compiler and backend circuit sizes.

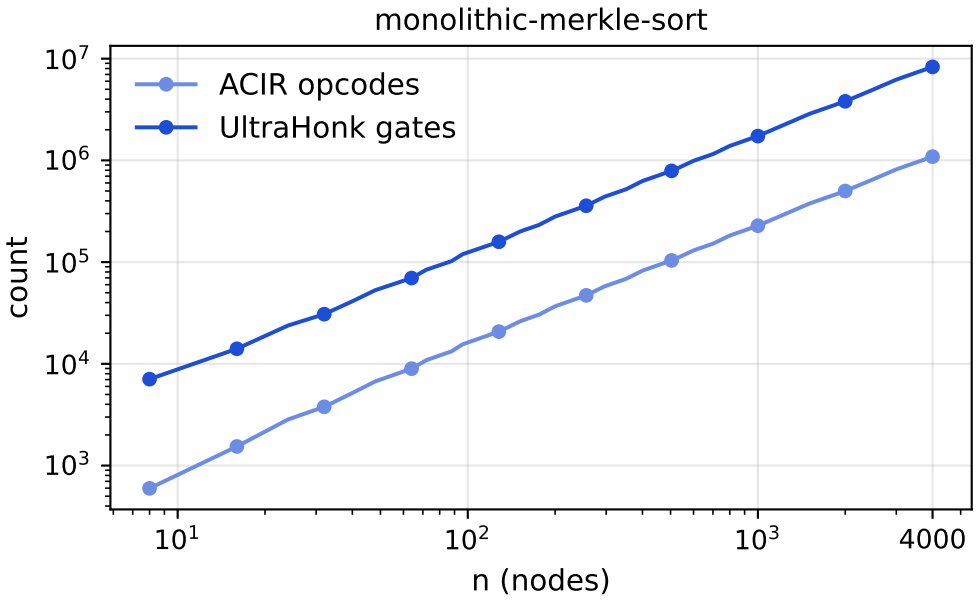


Figure 6.1: Compiler-level ACIR opcodes and backend UltraHonk gates for the refined monolithic circuit as the route length n increases. Both are deterministic counts for the fixed toolchain and are shown on logarithmic axes.

Both representations grow with n, with backend lowering placing the gate curve above the opcode curve. At $n = 4000$, the circuit reaches approximately 8.30 million UltraHonk gates. This behavior follows the construction in Section 4.4: the Merkle root removes the quadratic public-input surface, but sorting and authenticating the route still grow with n.

6.3.2 Prover Resources and Host Boundary

The practical boundary depends on maximum reported proving memory, witness-generation time, and proving time, shown together in Figure 6.2.

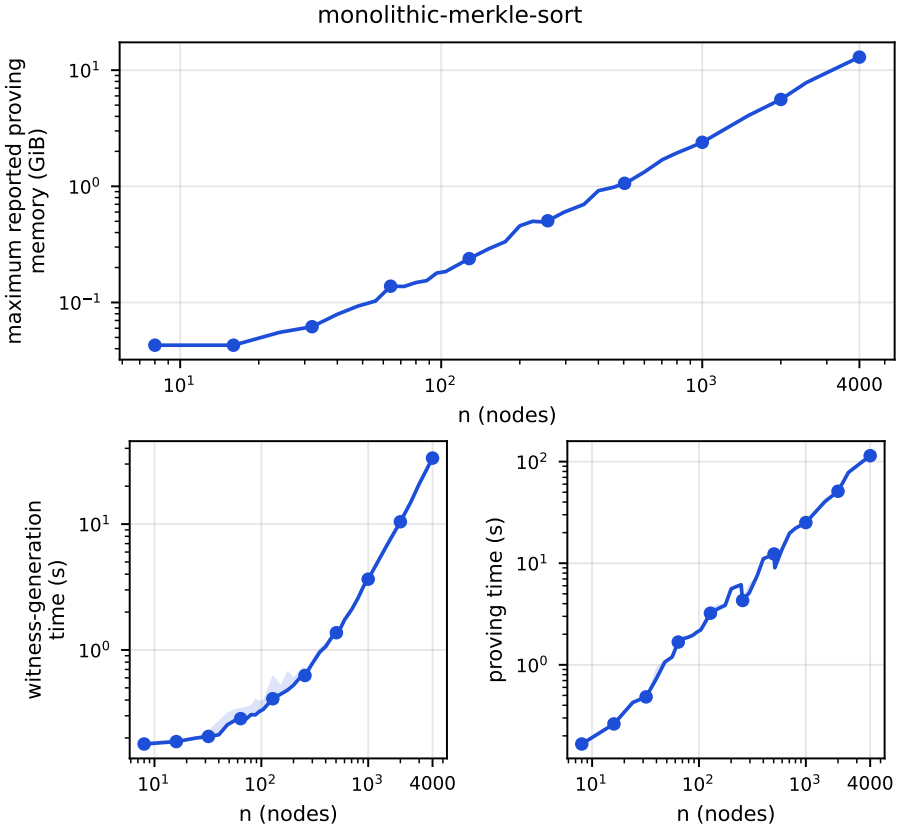


Figure 6.2: Maximum Barretenberg-reported proving memory, witness-generation time, and proving time for the refined monolithic circuit, on logarithmic axes. For each n, the memory curve is the median of the per-run peaks, while the timing curves use the smallest successful observation. Bands span the observed minimum–maximum ranges, following Section 6.1.2.

All three resources increase over the measured range. At $n = 4000$, the circuit uses 12.95 GiB of maximum reported proving memory, takes 33.45 seconds to generate the witness, and takes 114.38 seconds to prove.

Proof size and verification do not constrain the measured range: UltraHonk proofs remain 14,656 bytes, and the Merkle root fixes the public interface at $(\textsc{root}, T)$, keeping verification at the millisecond scale already observed in Figure 4.4.

The endpoint $n = 4000$ is the largest successful monolithic checkpoint, and its reported memory already approaches the approximately 15 GiB visible to the host. The failed $n = 5000$ attempt places the fixed-host boundary between those checkpoints without locating its exact value. Different hardware, backend behavior, or circuit components could move it; under the evaluated stack, the complete route remains one proving task whose resource requirement grows with n.

6.4 Composite Gate Accounting

The first quantitative test for RQ2 is whether smaller segments reduce the constrained work needed for the complete claim or redistribute it among more proving tasks. Aggregate gate accounting includes the route-dependent work retained by the segments and the stitching tax added by recombination (Section 5.1.5).

For an externally recombined construction, the aggregate gate count is

$$G_{\text{external}}(n, K) = K \cdot G_{\text{segment}}(n, K) + G_{\text{glue}}(n, K). \tag{6.2}$$

For recursion, it is

$$G_{\text{recursive}}(n, K) = K \cdot G_{\text{inner}}(n, K) + G_{\text{outer}}(n, K). \tag{6.3}$$

At fixed (n, K), the segments share one compiled circuit shape, and each is proved with a different witness. All K invocations therefore contribute to the total. Recursion likewise includes the inner proofs and the dependent outer proof that verifies and recombines them.

6.4.1 Aggregate Gate Work

For a construction A, define

$$R_A(n, K) = \frac{G_A(n, K)}{G_{\text{monolithic-sort}}(n)}. \tag{6.4}$$

The denominator is the refined monolithic Merkle-sort circuit from Section 6.3 at the same n. The product panels also include *monolithic Merkle-product*, a mechanism-matched control. It retains the committed-matrix

cost binding and threshold check of the refined monolithic circuit, but replaces exact sorting with the route-bound product check of Section 5.2.2 and Section 5.2.3. The complete route remains in one circuit with public interface (ROOT, T), so the control separates coverage-mechanism cost from decomposition. Figure 6.3 groups measurements by coverage, while the discussion follows Chapter 5′s plain, committed, and recursive progression.

Both plain constructions amortize their composition overhead. Plain-sort ranges from 1.132×–1.454× at $n = 48$ and narrows to 1.014×–1.016× at $n = 4000$. Its absolute overhead grows, but becomes a smaller fraction of the shared route authentication and cost work. Larger K repeats fixed segment constraints and adds boundary summaries, separating the curves most clearly at small n. At $n = 3000$, plain-product reaches 1.032×–1.035× against a 1.030× monolithic-product control. The control accounts for most of its distance from the canonical sort baseline, leaving external recombination as the smaller component.

For the committed constructions, hiding cost follows summary width. At $n = 4000$, committed-sort remains at 1.074×–1.076×, above the 1.014×–1.016× plain-sort range, because both circuits fold node-dependent openings. Committed-product stays within 1.032×–1.035×, essentially the plain-product range, because it commits to one fixed-size summary per segment. The product variants therefore have nearly equal gate counts despite their different disclosure surfaces (Section 5.2.5, Section 5.3.4).

Recursion changes the topology by constraining K proof verifications inside the outer circuit (Section 5.4.3). Aggregate cost ranges from 28.88×–116.03× at $n = 48$ and falls to 1.209×–1.765× at $n = 4000$. The product control isolates the mechanism, while the sort denominator locates recursion against the canonical thesis baseline.

The common denominator places every construction on one accounting scale. Plain, committed, and recursive composition offer different disclosure surfaces, while sort and product offer different coverage guarantees. Direct cost rankings therefore require matched structural privacy and coverage; cross-group comparisons quantify what is paid for a changed interface or mechanism.

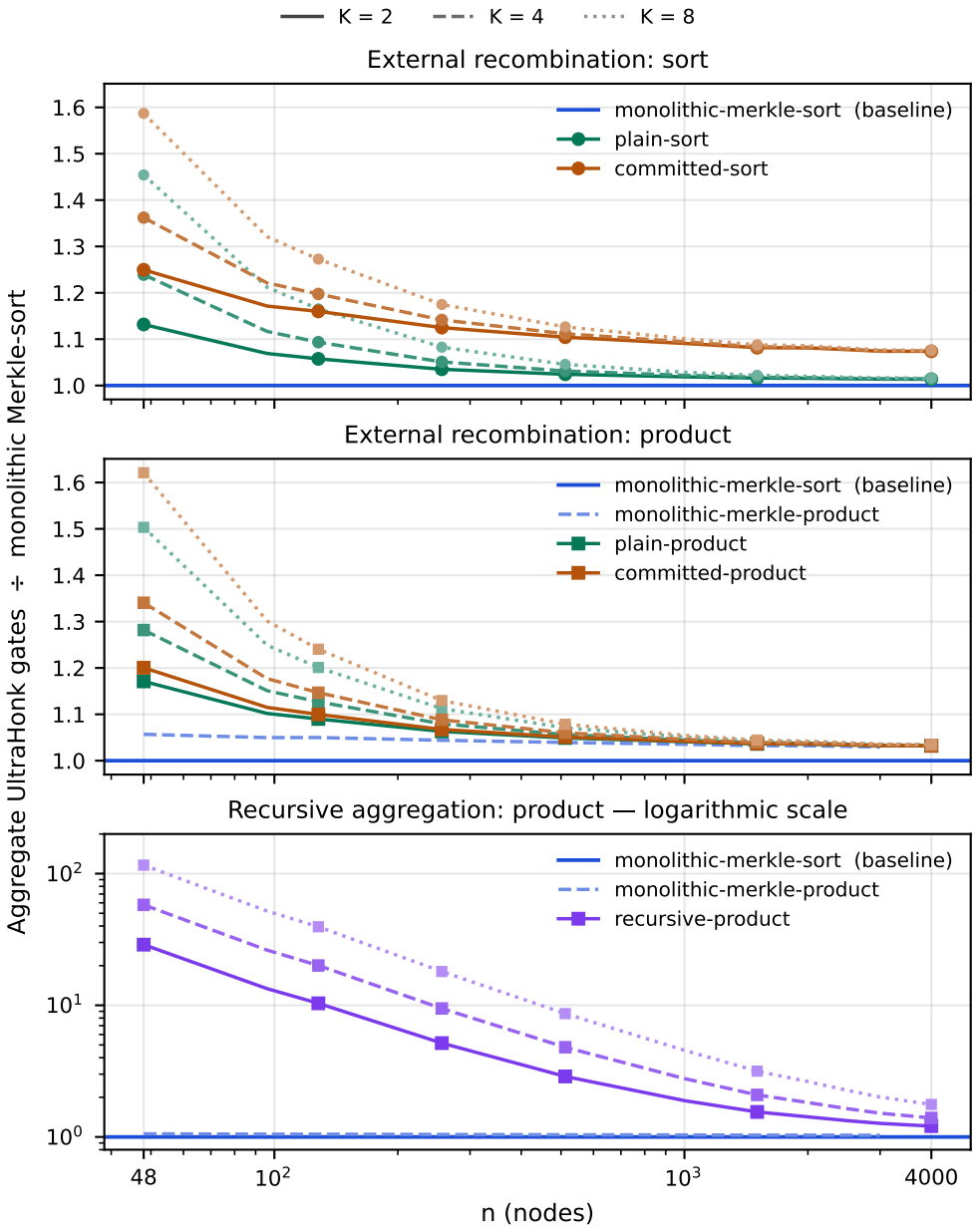


Figure 6.3: Aggregate UltraHonk gates for the five composite constructions relative to monolithic Merkle-sort at the same n. Each external total is given by $K \cdot G_{\text{segment}} + G_{\text{glue}}$, and each recursive total is $K \cdot G_{\text{inner}} + G_{\text{outer}}$. The external panels use the same linear vertical scale, while the recursive panel is logarithmic. The faint dashed curve in both product panels is the mechanism-matched monolithic Merkle-product control; it ends at $n = 3000$, its largest successful measurement. Lines join measured checkpoints. Sizes without a corresponding monolithic-sort denominator are omitted.

Table 6.3 records the exact representative values from the sweep under the common sort denominator.

Table 6.3: Selected aggregate gate ratios relative to monolithic Merkle-sort at the same n. The monolithic-product row supplies the mechanism-matched control under this normalization. Its $n = 4000$ entry is unavailable.

Construction	K	$n = 48$	$n = 512$	$n = 1504$	$n = 3000$	$n = 4000$
monolithic product	–	1.057	1.039	1.032	1.030	–
plain sort	2	1.132	1.024	1.016	1.014	1.014
plain sort	4	1.239	1.031	1.018	1.015	1.014
plain sort	8	1.454	1.046	1.022	1.017	1.016
committed sort	2	1.250	1.104	1.082	1.074	1.074
committed sort	4	1.362	1.112	1.084	1.075	1.074
committed sort	8	1.587	1.127	1.088	1.077	1.076
plain product	2	1.171	1.049	1.036	1.032	1.032
plain product	4	1.282	1.056	1.038	1.033	1.033
plain product	8	1.503	1.071	1.042	1.035	1.034
committed product	2	1.200	1.051	1.037	1.033	1.032
committed product	4	1.341	1.060	1.039	1.034	1.033
committed product	8	1.621	1.079	1.045	1.036	1.035
recursive product	2	28.88	2.881	1.549	1.268	1.209
recursive product	4	57.93	4.803	2.086	1.515	1.394
recursive product	8	116.03	8.645	3.161	2.010	1.765

6.4.2 Distribution Across Proving Tasks

We use $n = 3000$, the largest checkpoint containing all five composite constructions, every value of K, and both monolithic controls. Monolithic product was attempted at $n = 4000$, but the circuit crossed the next padded proving-domain boundary and exceeded the host's available memory.

Figure 6.4 splits each aggregate total into K segment invocations and one recombination stage. Horizontal rules separate the segment block into individual proving units, while the hatched cap and its annotation report the recombiner and its share

$$\frac{G_{\text{recombiner}}}{K \cdot G_{\text{segment}} + G_{\text{recombiner}}}. \tag{6.5}$$

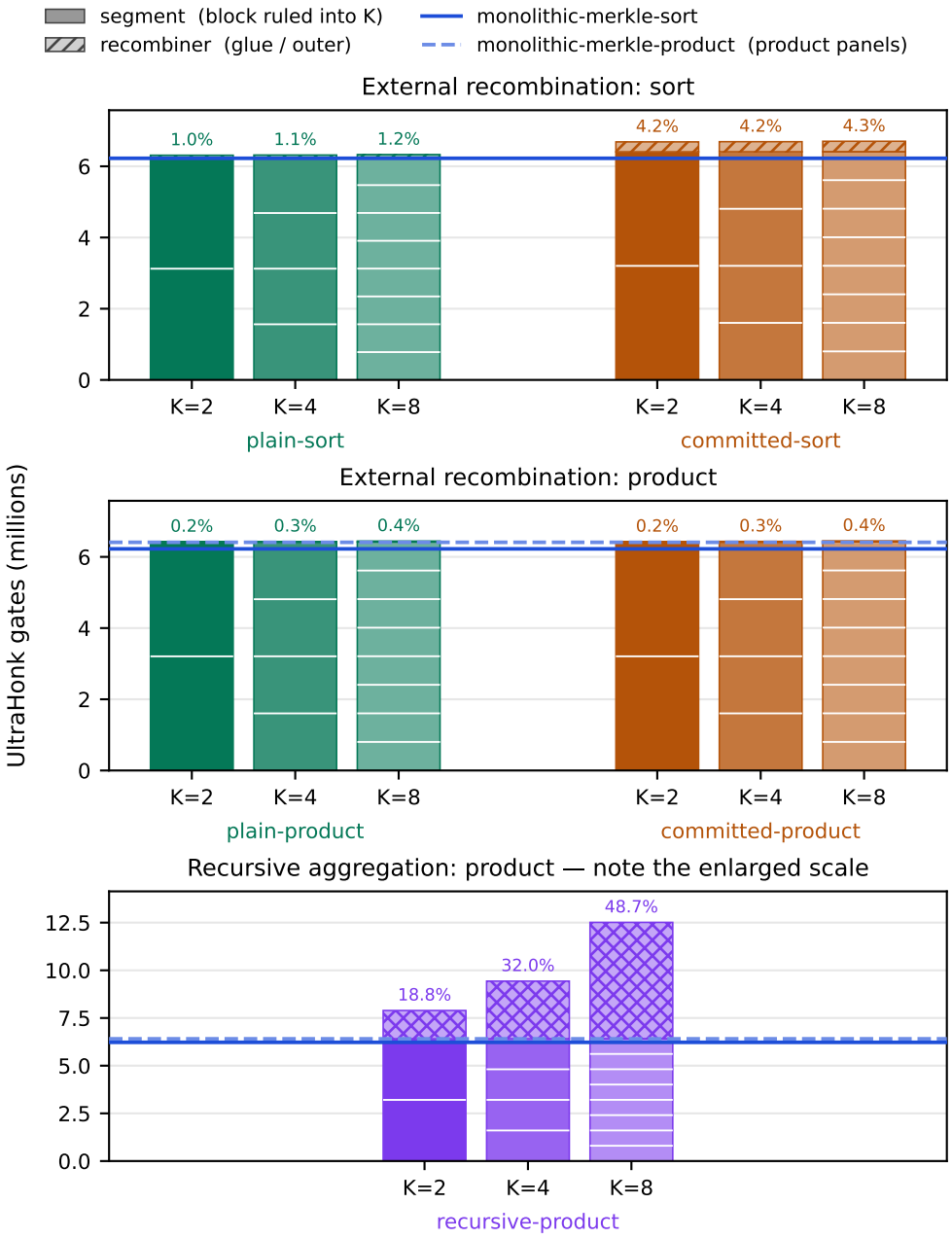


Figure 6.4: Distribution of aggregate UltraHonk gates at $n = 3000$. Each bar contains K segment-circuit invocations and one recombination circuit. The unhatched block is the aggregate segment work and is divided into K equal slabs, each corresponding to one segment circuit; the hatched cap is the glue or recursive outer circuit. Percentages report the recombiner's share of the complete bar. Solid horizontal lines show monolithic Merkle-sort, while dashed lines in the product panels show monolithic Merkle-product. The external panels share a vertical scale; the recursive panel uses an enlarged scale.

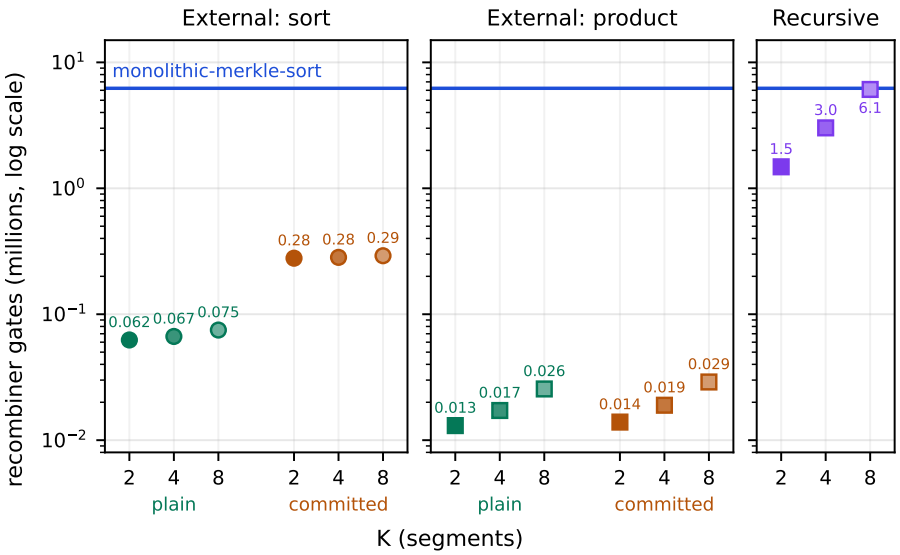


Figure 6.5: Glue and outer UltraHonk gates at $n = 3000$ on a shared logarithmic scale; annotations are in millions. The horizontal line is monolithic Merkle-sort. External proof checks and public-input reconciliation are excluded.

Across all five constructions, the combined segment block changes little with K while one segment becomes smaller. The constructions follow the same pattern: segmentation divides the dominant route work into smaller proving units, and the aggregate total retains that work.

The conservation is approximate and not exact because increasing K transfers boundary arcs to recombination, repeats fixed segment constraints, and changes the local coverage checks. The stacked scale compresses the external recombination caps, so Figure 6.5 shows the same glue and outer components on a shared logarithmic axis.

Within the plain group, sort glue processes the same n node values for every K, while product glue combines K compact fingerprints with the n-factor target product. Both process more boundaries and summaries as K grows, but the product recombiner remains much smaller because it avoids the global sort.

The committed group isolates the cost of hiding those interfaces. Committed-sort glue rises from the 0.062–0.075 million plain-sort range to 0.279–0.291 million gates because it re-folds node-dependent openings

Table 6.4: Component and aggregate UltraHonk gates at $n = 3000$, in millions. Monolithic controls are 6.225 million gates for sort and 6.409 million for product; shares use unrounded counts.

Construction	K	Segment	K Segments	Recombination	Aggregate	Recombiner share
plain sort	2	3.124	6.248	0.062	6.311	1.0%
plain sort	4	1.562	6.250	0.067	6.316	1.1%
plain sort	8	0.782	6.253	0.075	6.328	1.2%
committed sort	2	3.203	6.407	0.279	6.685	4.2%
committed sort	4	1.602	6.408	0.283	6.691	4.2%
committed sort	8	0.801	6.412	0.291	6.703	4.3%
plain product	2	3.206	6.413	0.013	6.426	0.2%
plain product	4	1.604	6.414	0.017	6.432	0.3%
plain product	8	0.802	6.418	0.026	6.443	0.4%
committed product	2	3.207	6.413	0.014	6.427	0.2%
committed product	4	1.604	6.416	0.019	6.435	0.3%
committed product	8	0.803	6.421	0.029	6.450	0.4%
recursive product	2	3.206	6.413	1.481	7.894	18.8%
recursive product	4	1.604	6.414	3.019	9.433	32.0%
recursive product	8	0.802	6.418	6.094	12.512	48.7%

before global coverage. Committed-product stays at 0.014–0.029 million, close to the 0.013–0.026 million plain-product range, because each opening has fixed size. Hiding cost therefore follows summary width.

Each additional segment adds a constrained proof verification to the recursive outer circuit. It grows from 1.481 million gates at $K = 2$ to 3.019 million at $K = 4$ and 6.094 million at $K = 8$, raising its aggregate share from 18.8% to 48.7%. At $K = 8$, the outer circuit alone approaches either monolithic control, before accounting for the eight inner proofs. These gates move verification and recombination behind one final proof.

At fixed n, increasing K shrinks one segment while the combined segment gate count remains almost unchanged. Whether this lowers the capacity requirement depends on the glue or outer task and on backend padding; the next section measures the complete peak directly.

6.5 Peak Prover Memory

The preceding section counted every proof invocation. Peak memory instead asks whether decomposition reduces the capacity required by any one

proving task, one possible advantage under RQ2. A composite architecture improves on the monolithic baseline only if its recombination task also remains below it.

For an externally recombined construction, the complete peak is

$$P_{\text{external}}(n, K) = \max\left[\left(\max_{0 \le i < K} P_i^{\text{seg}}\right), P_{\text{glue}}\right]. \tag{6.6}$$

For recursion, it is

$$P_{\text{recursive}}(n, K) = \max\left[\left(\max_{0 \le i < K} P_i^{\text{inner}}\right), P_{\text{outer}}\right]. \tag{6.7}$$

Within each matched run, the largest segment or inner invocation is compared with the glue or outer invocation, and the median is taken only after this architectural peak is derived. Under the measured sequential schedule, only one benchmark task is active at a time, so their allocations are not added. Under distribution, the same equations describe the largest capacity required from one worker, not aggregate memory across concurrent workers.

6.5.1 End-to-End Memory Scaling

Figure 6.6 reports the complete peak across the evaluation grid. Each composite point already contains the maximum over its segment and recombination tasks.

The sort constructions converge as n grows. At $n = 4000$, plain-sort and committed-sort both require approximately 8.8 GiB for every measured K: shorter segments no longer reduce the complete peak. Their overlap does not make the commitment layer free; Section 6.4.2 records its additional constrained work. The similar peaks show instead that both constructions encounter the same memory floor.

The product curves remain separated by K. At $n = 4000$, their peaks fall from approximately 6.0 GiB at $K = 2$ to 3.2 GiB at $K = 4$ and 1.6 GiB when $K = 8$, so the limiting task continues to shrink with the segment. The reduction is stepped rather than proportional because backend allocations do not vary continuously with circuit size. At $n = 3000$, the largest checkpoint

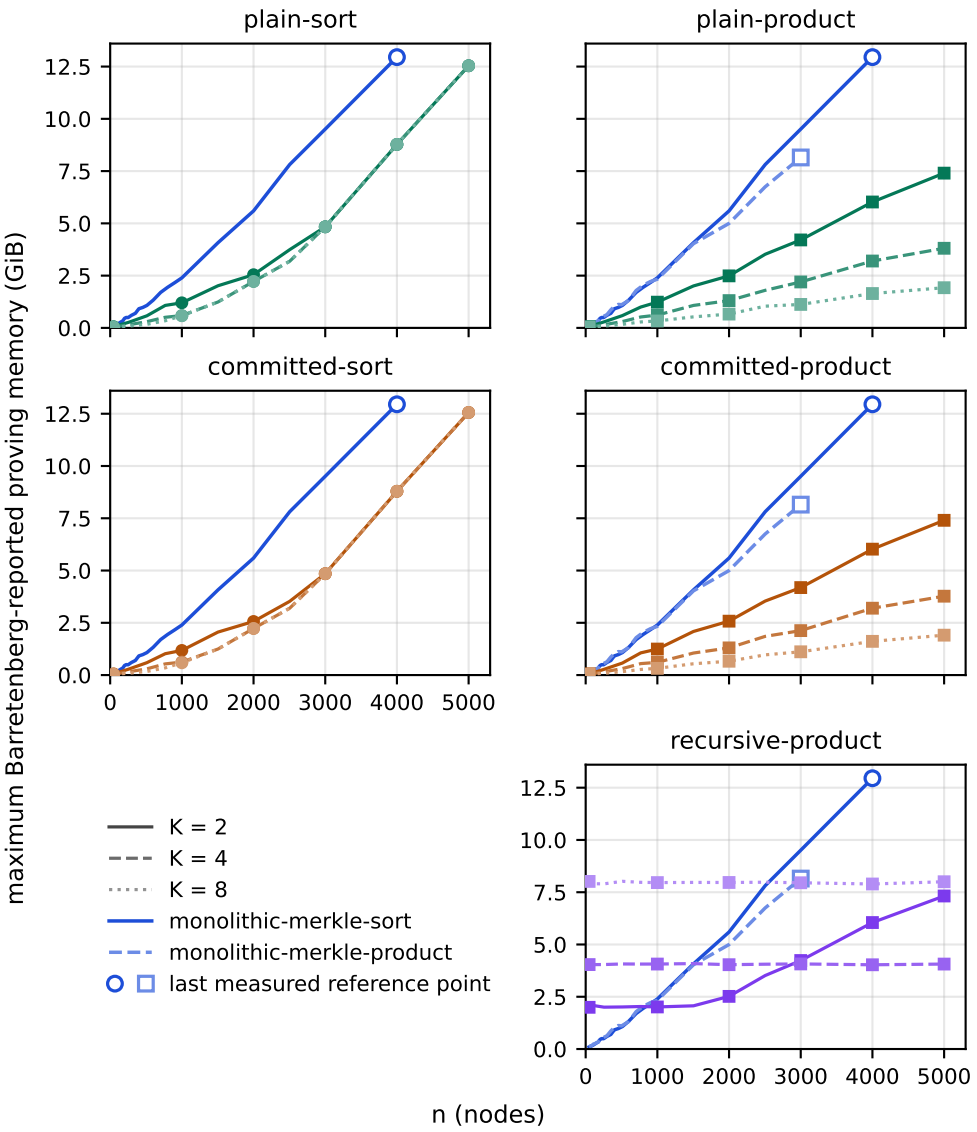


Figure 6.6: Median end-to-end maximum Barretenberg-reported proving memory for the five composite constructions. Within each matched run, the complete peak is the maximum over the K segment invocations and the glue or outer invocation. All panels use the same linear axes. The solid reference is monolithic Merkle-sort; product-based panels also show monolithic Merkle-product as a faint dashed control. Open markers identify the last measured reference points. Lines join measured checkpoints and are not extrapolated.

shared with the monolithic-product control, monolithic product itself uses less memory than monolithic sort; the composite curves nevertheless remain below this control, isolating an additional reduction from decomposition.

Recursion exposes the opposing effect. Increasing K shrinks each inner task but gives the outer circuit more proofs to verify. At $n = 4000$, $K = 4$ has the lowest measured recursive peak, below both $K = 2$ and $K = 8$.

6.5.2 Component Bottlenecks

We fix $n = 4000$, the largest checkpoint shared by the canonical monolithic baseline and every composite configuration, and separate the largest segment or inner invocation from recombination. The monolithic-product control remains visible in Figure 6.6 through its last successful measurement and is not extrapolated to this checkpoint.

Sort is glue-bound. From $K = 2$ to $K = 8$, plain-sort segment memory falls from 5.96 to 1.38 GiB and committed-sort from 5.98 to 1.71 GiB, while their glue circuits remain near 8.8 GiB. Both recombiners retain the global n-node coverage operation, with committed-sort also opening the hidden summaries. That work does not shrink with M, so the glue fixes the complete peak; the monolithic baseline requires roughly 1.47 × as much memory.

Product is segment-bound. Its glue remains between approximately 0.04 and 0.06 GiB, while segment memory falls from approximately 6.0 to 1.6 GiB. Compact fingerprints and chain summaries keep recombination below the segment task throughout the measured range. Plain-product and committed-product are nearly identical on this metric despite their different constrained work and public surfaces (Section 6.4.2, Section 5.3.4).

Recursion crosses between these regimes. At $K = 2$, the 6.05-GiB inner task exceeds the 2.08-GiB outer stage. At $K = 4$, the inner task falls to 3.20 GiB while the outer rises to 4.03 GiB; at $K = 8$, the corresponding values are 1.66 and 7.89 GiB. The measured peak is therefore inner-bound at $K = 2$ and outer-bound at $K = 4$ and $K = 8$.

These memory regimes are compatible with the gate distribution in Section 6.4. Recombiner share uses an aggregate denominator containing all K segments, whereas peak memory compares recombination with the largest single segment. A small share of total gates can therefore determine the capacity requirement. Memory also changes at padded proving-domain

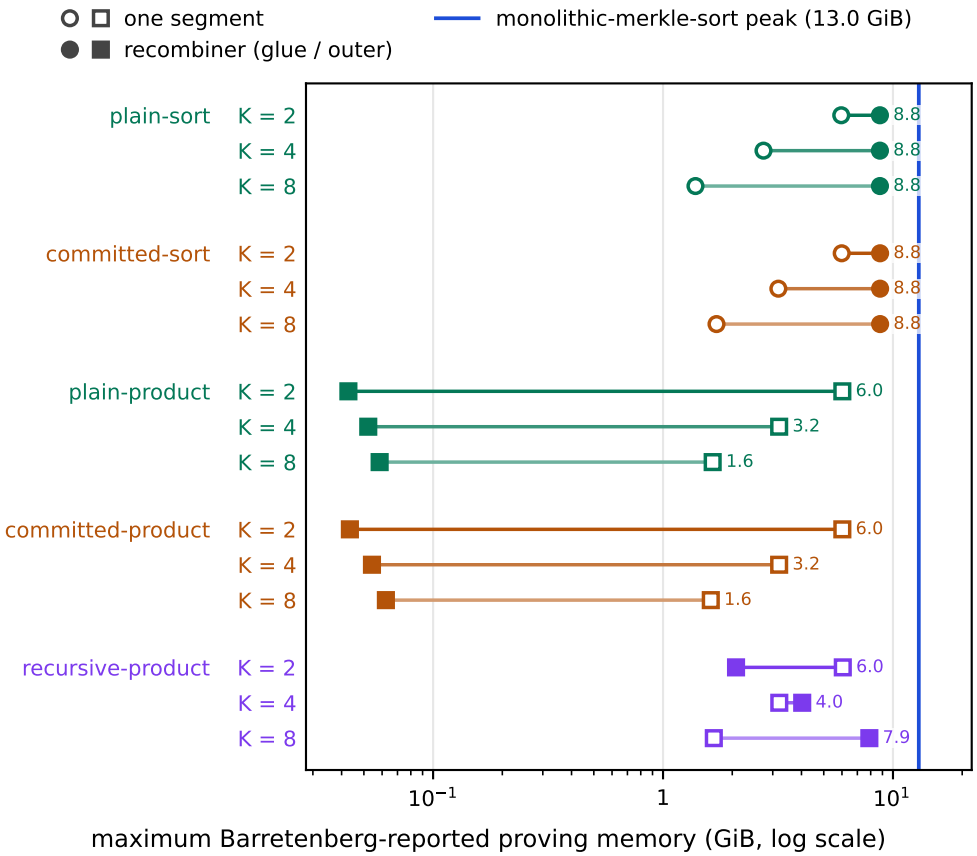


Figure 6.7: Component proving-memory profiles at $n = 4000$, the largest checkpoint shared by the canonical monolithic baseline and every composite configuration. Open markers report the median run-level largest-segment memory, while filled markers report median glue or outer memory. Complete peaks are medians of the run-level maxima; at this checkpoint, each coincides with the rightmost component median. Connecting lines associate the two stages and do not represent additive memory or uncertainty. The vertical line marks the median 12.95-GiB monolithic Merkle-sort peak, displayed as 13.0 GiB. The logarithmic axis keeps the small product recombination stages visible.

boundaries rather than continuously with gate count, explaining the similar plain and committed peaks despite their different constraint counts.

Table 6.5 preserves the exact component and architectural values under the common monolithic-sort baseline.

Table 6.5: Component and complete maximum Barretenberg-reported proving memory at $n = 4000$, in GiB. Segment and recombination values are medians across matched runs. Peak is the median of the run-level maxima; for every row, it coincides with the larger displayed component median. Baseline / peak is 12.95 GiB divided by that peak.

Construction	K	Largest segment	Glue/outer	Peak	Baseline / peak
plain sort	2	5.957	8.775	8.775	1.48×
plain sort	4	2.735	8.775	8.775	1.48×
plain sort	8	1.384	8.777	8.777	1.48×
committed sort	2	5.984	8.789	8.789	1.47×
committed sort	4	3.174	8.790	8.790	1.47×
committed sort	8	1.708	8.791	8.791	1.47×
plain product	2	6.022	0.043	6.022	2.15×
plain product	4	3.199	0.052	3.199	4.05×
plain product	8	1.644	0.059	1.644	7.88×
committed product	2	6.023	0.043	6.023	2.15×
committed product	4	3.199	0.054	3.199	4.05×
committed product	8	1.613	0.062	1.613	8.03×
recursive product	2	6.050	2.077	6.050	2.14×
recursive product	4	3.203	4.029	4.029	3.21×
recursive product	8	1.663	7.893	7.893	1.64×

Every composite configuration also succeeds at $n = 5000$, beyond the successfully measured monolithic range. The sort constructions have converged near 12.5–12.6 GiB, product with $K = 8$ remains near 1.9 GiB, and recursive product with $K = 4$ remains near 4.1 GiB. This extends the successfully evaluated range on the fixed host without establishing $n = 5000$ as a new ceiling; the sort constructions are already close to the host budget, while product and selected recursive configurations retain more headroom.

6.6 End-to-End Prover Time

Prover time accounts for every task required to complete the proof. For an invocation j, its online prover time is

$$t_j = t_j^{\text{witness}} + t_j^{\text{proof}}, \tag{6.8}$$

where the two terms are the measured witness-generation and proof-generation intervals. Compilation, verification-key derivation, instance preparation, and verification remain outside this online prover metric under Section 6.1.2. We compare the same component measurements under sequential execution and full distribution.

6.6.1 Sequential and Distributed Scaling

On one host, proving tasks execute one at a time, although each invocation retains Barretenberg's internal parallelism. The complete schedule is

$$T_{\text{seq}} = \sum_{j} t_j, \tag{6.9}$$

where j ranges over every segment or inner invocation and the required glue or outer invocation.

Figure 6.8 shows that every construction becomes slower as n grows, and increasing K does not systematically reduce the sequential total. Shorter segments may reduce segment proving time, but there are more segments and recombination adds work; local crossings among the K curves reflect backend padding and timing variation rather than distinct regimes.

At $n = 4000$, monolithic sort takes 147.8 seconds and the composite schedules span approximately 149–246 seconds. At $n = 5000$, every composite completes sequentially in approximately 174–261 seconds, while monolithic Merkle-sort exceeds the host memory budget during proof generation. Decomposition therefore extends the successfully evaluated range at the cost of serializing multiple proving tasks.

Sequential execution leaves the independence of the proof tasks unused. External segment and glue proofs depend only on instance data and can run concurrently, so completion waits for the slower task:

$$T_{\text{dist}}^{\text{external}} = \max\left[\left(\max_{0 \le i < K} t_i^{\text{segment}}\right), t^{\text{glue}}\right]. \tag{6.10}$$

Recursion parallelizes the inner proofs but retains a dependent outer task:

$$T_{\text{dist}}^{\text{recursive}} = \max_{0 \le i < K} t_i^{\text{inner}} + t^{\text{outer}}. \tag{6.11}$$

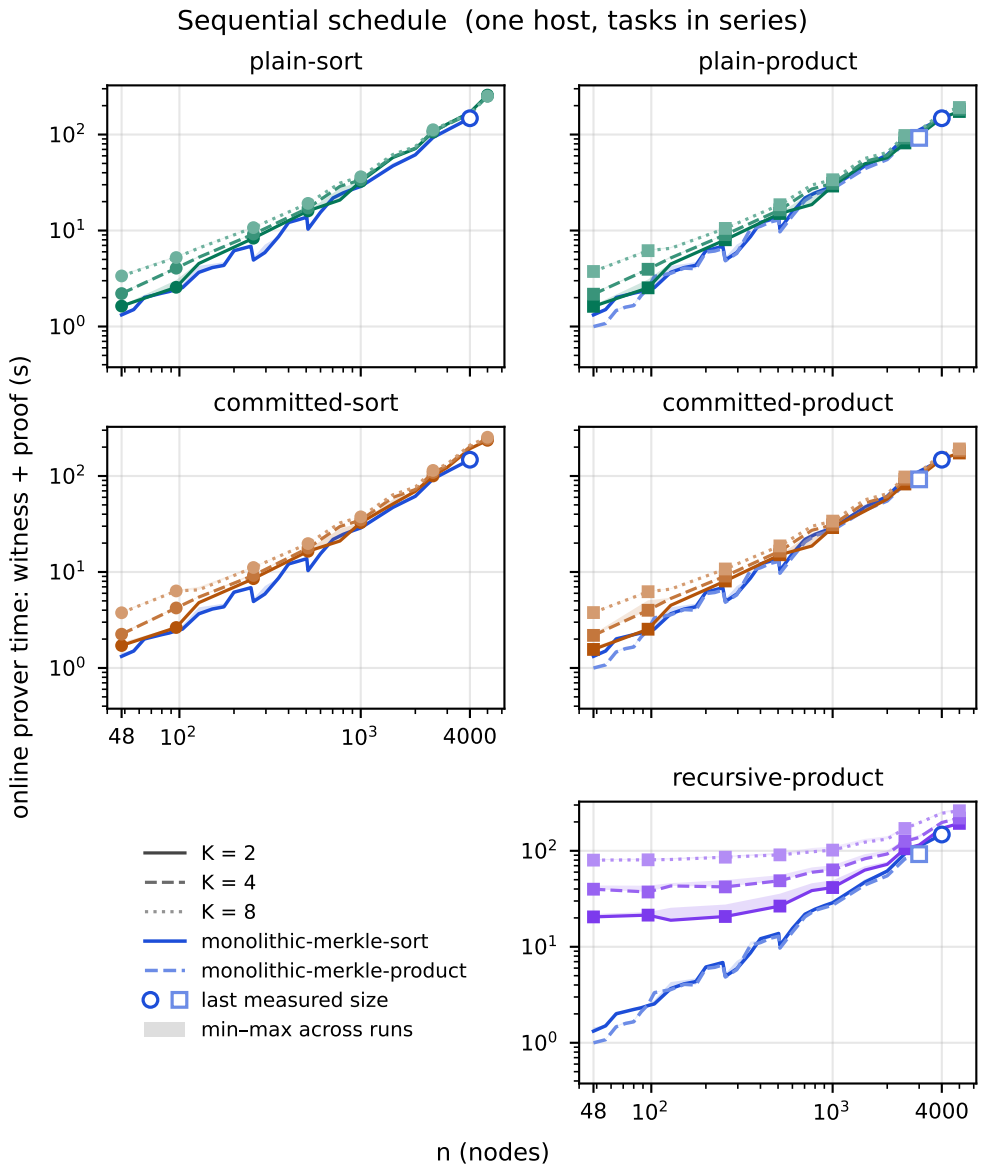


Figure 6.8: Sequential online prover time for the five composite constructions. Every point sums witness and proof generation over all K segment or inner invocations and the glue or outer invocation within a matched run. Curves show the lower-envelope complete time and bands span the observed run-level minimum–maximum range. Both axes are logarithmic and are shared with Figure 6.9. The solid reference is monolithic Merkle-sort; product-based panels also show monolithic Merkle-product as a faint dashed control. Open markers identify the last measured reference points. Lines join measured checkpoints and are not extrapolated.

These estimates compose isolated measurements rather than observing a deployed cluster. Full external distribution assumes up to $K + 1$ workers comparable to the benchmark host; recursion assumes K inner workers followed by one outer worker.

Figure 6.9 shows that distribution changes waiting time, not total computation. External sort initially benefits from shorter segments, then converges as glue becomes critical. At $n = 4000$, plain-sort takes 38.3 seconds at $K = 4$ and 37.5 seconds at $K = 8$, while committed-sort remains near 41.5 seconds for both.

Product retains the separation by K: plain-product falls from 75.3 to 40.0 and 21.0 seconds, with committed-product nearly identical. Its critical task continues to shrink with the segment instead of reaching a glue floor.

Recursion retains a dependent tail. Its estimates are 96.5, 81.0, and 101.4 seconds for $K = 2$, 4, and 8: four segments give the lowest value at this checkpoint, while eight transfer enough work to the outer stage to reverse the benefit of shorter inner tasks.

Every distributed estimate at $n = 4000$ lies below the 147.8-second monolithic time only after additional machines are introduced. The same architectural regimes persist at $n = 5000$, where no monolithic comparison is available.

6.6.2 Schedule Bottlenecks

We use $n = 4000$, the largest checkpoint shared by monolithic sort and every composite configuration. Figure 6.10 separates aggregate sequential work, the distributed critical path, and the witness and proof phases that produce each schedule. Monolithic product is absent rather than extrapolated.

Sort crosses from segment-bound to glue-bound. In plain-sort, the longest segment exceeds glue by 64.6 to 40.6 seconds at $K = 2$; at $K = 8$, glue remains at 37.5 seconds while the segment falls to 16.1 seconds. Committed-sort follows the same pattern. Its global coverage glue also spends substantial time generating the witness, so gate count alone does not predict the timing floor.

Product remains segment-bound. Its glue finishes in less than one second, while the longest segment falls from approximately 75 to 40 and 21 seconds.

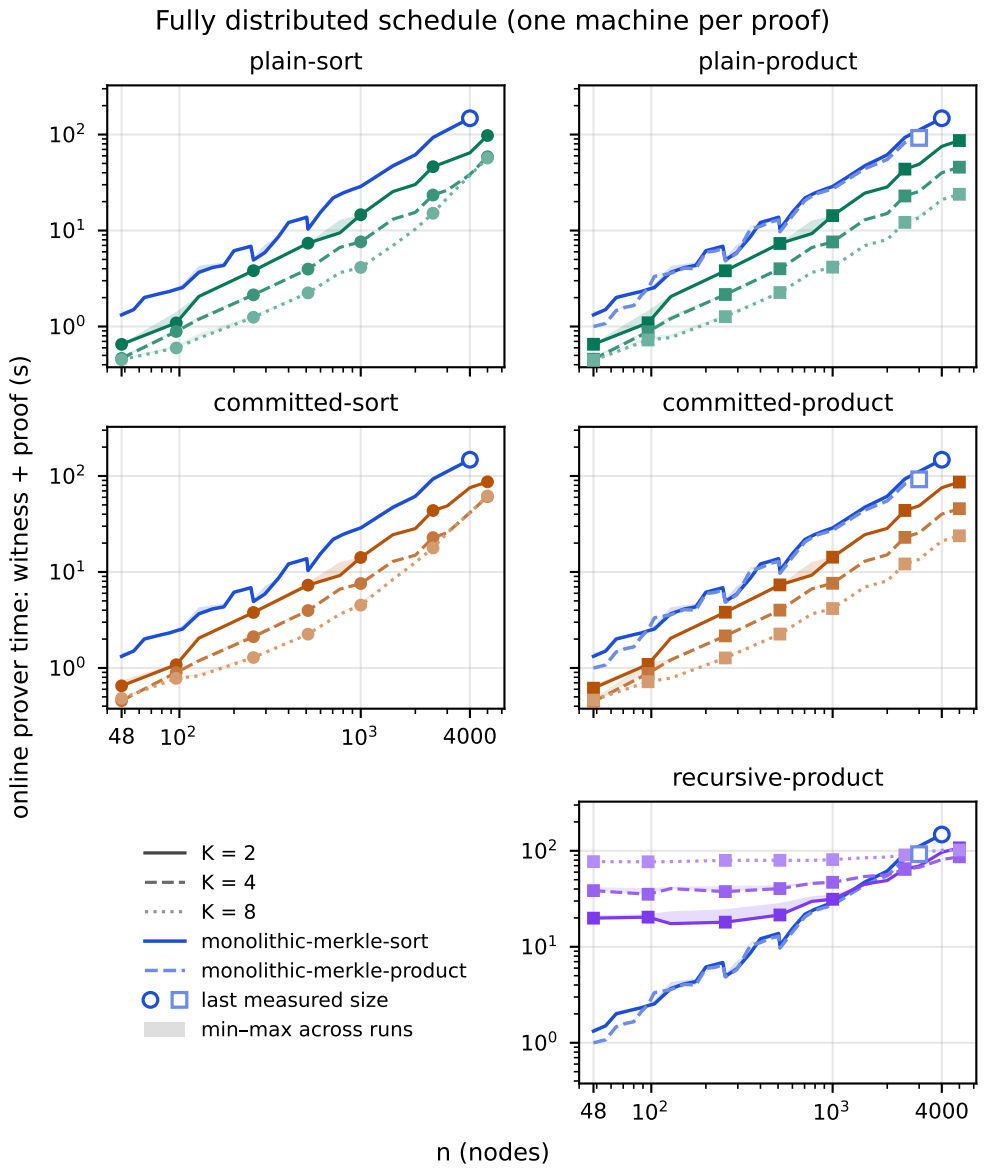


Figure 6.9: Fully distributed online prover-time estimates composed from the uncontended component runs. External completion is the maximum of the longest segment task and the independent glue task; recursive completion is the longest inner task followed by the dependent outer task. Curves show the lower-envelope run-level completion time and bands span the observed minimum–maximum range. The logarithmic axes are identical to Figure 6.8. The monolithic curves are single-host references: comparison with them is not hardware-normalized. The model assumes one comparable machine for each proof task that runs in parallel and excludes communication, coordination, and competition for shared host resources.

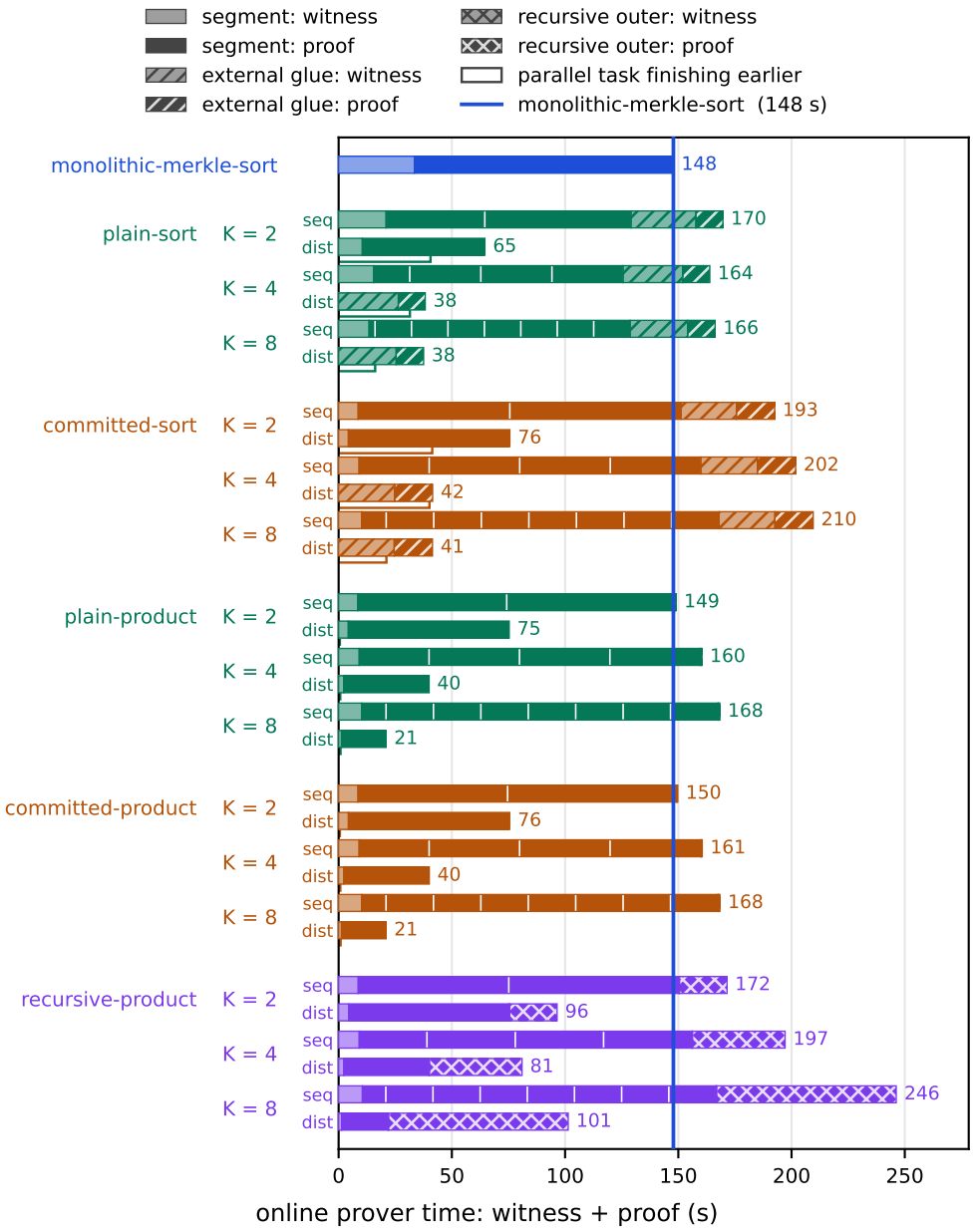


Figure 6.10: Sequential and fully distributed online prover-time composition for all constructions at $n = 4000$. Filled distributed bars show the critical task or dependent task sequence; for external recombination, the unfilled outline is the parallel task that finishes earlier and is not added. Hatching distinguishes glue and the recursive outer stage. White rules divide aggregate segment witness-plus-proof time into K equal normalization shares rather than measured invocation boundaries. The blue bar and vertical rule mark the 147.8-second monolithic Merkle-sort reference. Monolithic product is unavailable at this checkpoint.

Distribution can therefore approach the available K-way reduction even though the sequential total still contains every invocation.

In recursion, the longest inner task shrinks from approximately 75 to 40 and 22 seconds, while outer proof generation grows from 21 to 41 and 79 seconds. The $K = 4$ row is the shortest dependent sequence; at $K = 8$, constrained verification makes the outer stage dominant.

Table 6.6 records the exact complete times and the schedule compression $T_{\text{seq}}/T_{\text{dist}}$. The product mechanisms and structural-privacy levels differ, so the common table maps the timing frontier rather than ranking every row as like-for-like (Section 6.4). For RQ2, the timing result is conditional: decomposition can extend the successfully evaluated range on one constrained host at the cost of serialized time, or expose independent work that additional workers can overlap.

Table 6.6: Sequential and fully distributed online prover time at $n = 4000$. Times are lower-envelope composed schedules in seconds. $T_{\text{seq}}/T_{\text{dist}}$ uses unrounded values. Distributed times assume additional comparable workers and are not measured cluster speedups. Product rows are not mechanism-matched comparisons with monolithic product.

Construction	K	Sequential time (s)	Fully distributed time (s)	$T_{\text{seq}}/T_{\text{dist}}$
monolithic sort	–	147.8	147.8	1.00×
plain sort	2	169.7	64.6	2.63×
plain sort	4	163.9	38.3	4.28×
plain sort	8	166.3	37.5	4.43×
committed sort	2	192.5	75.6	2.55×
committed sort	4	201.8	41.5	4.86×
committed sort	8	209.6	41.5	5.06×
plain product	2	149.0	75.3	1.98×
plain product	4	160.4	40.0	4.01×
plain product	8	168.3	21.0	8.03×
committed product	2	149.8	75.5	1.98×
committed product	4	160.6	40.0	4.01×
committed product	8	168.4	21.0	8.02×
recursive product	2	171.5	96.5	1.78×
recursive product	4	197.2	81.0	2.43×
recursive product	8	246.2	101.4	2.43×

6.7 Verifier Load

External recombination presents the final verifier with K segment proofs, one glue proof, and public values that must be reconciled across them. Recursion consumes the inner proofs as private witness data and exposes only the outer proof. We account for the encoded proof and public-input artifacts and the wall-clock time of the required `bb verify` commands. Reconciliation remains necessary for soundness, but we report it as a number of equality checks rather than timing an application-specific coordinator.

6.7.1 Proof Package and Public Surface

Every recorded UltraHonk proof is 14,656 bytes, matching the monolithic result in Section 6.3.2. An external proof package contains $K + 1$ such proofs; monolithic and recursive constructions expose one.

Each proof also carries its own public-input vector. For an externally recombined construction, the aggregate number of public-input slots processed by the final verifier is

$$S_{\text{pub}}^{\text{external}} = K \cdot S_{\text{pub}}^{\text{segment}} + S_{\text{pub}}^{\text{glue}}. \tag{6.12}$$

Table 6.7 applies this accounting to every construction and also records the equalities required after the individual proofs have verified.

Table 6.7: Final-verifier interface accounting. External aggregate slots equal K segment interfaces plus the glue interface, using $K \cdot M = n$ for plain-sort. Each slot occupies 32 bytes in the current artifact format. External equality checks bind the proof set after backend acceptance. Recursion performs the corresponding binding inside the outer circuit.

Construction	Proof checks	Aggregate public-input slots	External equality checks
monolithic sort	1	2	none
plain sort	$K + 1$	$2n + 7K + 2$	$n + 4K$
committed sort	$K + 1$	$3K + 2$	$2K$
plain product	$K + 1$	$15K + 4$	$9K$
committed product	$K + 1$	$4K + 3$	$3K$
recursive product	1	2	none

Plain-sort duplicates the node list across the segment and glue interfaces, so its aggregate public surface grows with n as well as K. The compact external constructions depend only on K, while recursion retains the monolithic two-field interface. The equality counts describe a separate layer: they are application-level bindings performed after backend acceptance and are not additive with the public-input slots.

Excluding setup verification keys, the encoded payload consists of the proof files plus 32 bytes for each public-input slot. Every evaluated package remains below 0.4 MiB, while recursive-product and monolithic sort remain at approximately 14.4 KiB.

6.7.2 Backend Verification Time

For an externally recombined construction, the backend time is the sequential sum

$$V_{\mathrm{BB}}^{\mathrm{external}} = \sum_{i=0}^{K-1} v_i^{\mathrm{segment}} + v^{\mathrm{glue}}. \tag{6.13}$$

Recursive and monolithic verification each require one command. Complete values are composed within matched runs before lower-envelope selection under Section 6.1.2. The metric includes process startup and artifact loading as well as proof verification. It excludes the application-level reconciliation counted in Table 6.7, so it measures the backend portion of verifier load rather than complete application latency.

Figure 6.11 shows this backend time across the complete evaluation grid. Verification time separates primarily by proof count. External constructions require one backend invocation for every segment proof and one for glue, so their curves rise with K. Differences among constructions at the same K are small and variable, and no construction exhibits systematic growth with n over the measured grid. Even plain-sort's growing public-input vector does not produce a resolved wall-clock trend above command-level variation, so the grid does not isolate the marginal cost of additional public inputs.

At $n = 4000$, external backend verification spans 27.6–126.3 milliseconds, approximately 1.5×–6.7× the monolithic-sort reference of 18.8 milliseconds,

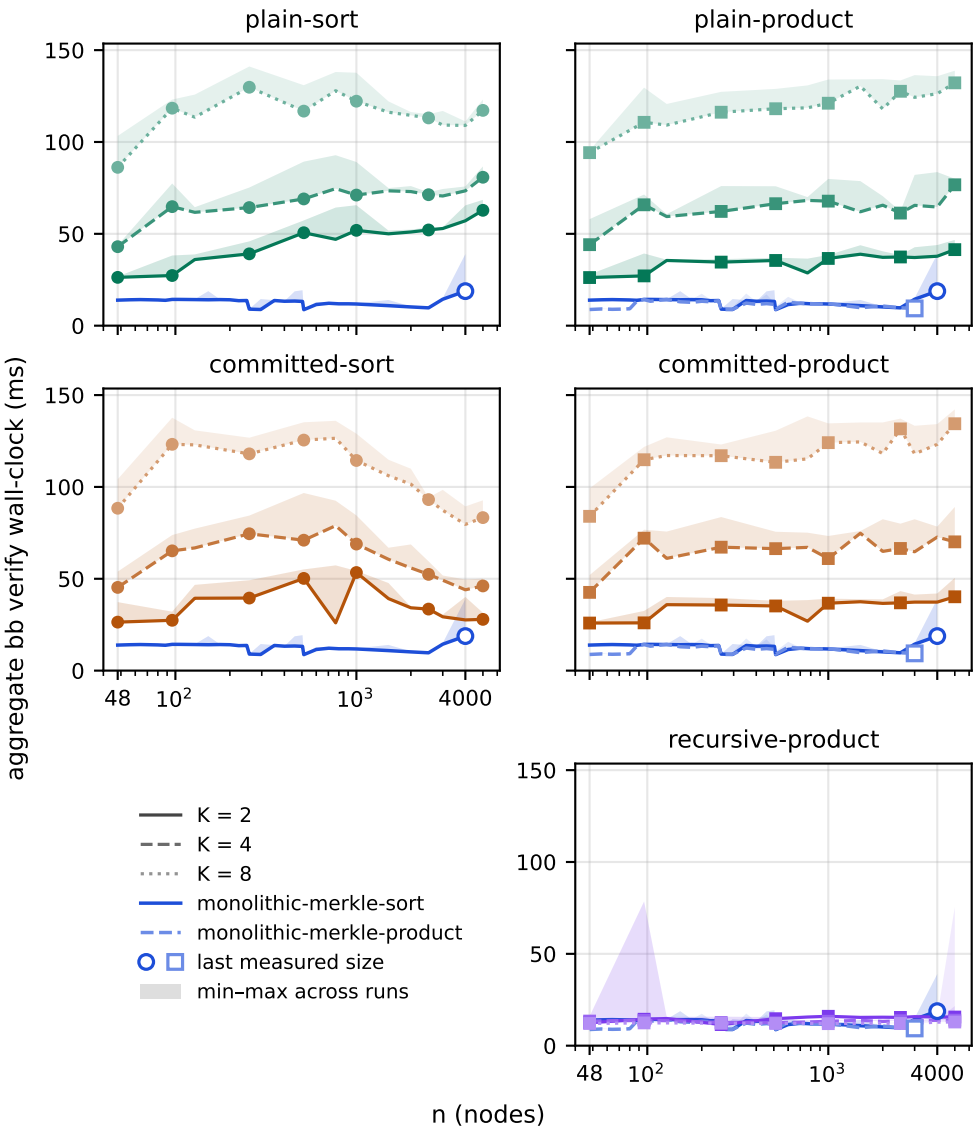


Figure 6.11: Aggregate `bb verify` wall-clock across the evaluation grid. External points sum the $K + 1$ required backend commands within each matched run; recursive points contain only the final outer verification. Curves show lower-envelope complete times and bands span the observed run-level minimum–maximum intervals. All panels use the same linear time scale and a logarithmic n-axis. Open markers identify the last measured reference points.

while recursive verification spans 12.8–15.6 milliseconds. These absolute times remain small beside prover schedules of tens or hundreds of seconds. Backend verification therefore does not determine architectural feasibility on the evaluated host and stack.

7 Discussion

This chapter combines the progression of Chapters 4 and 5 with the results of the evaluation in Chapter 6 to answer the research questions, delimit the conclusions, and identify the resulting architectural choices.

7.1 Research Findings

The answers to the research questions require direct cost rankings within matched coverage and structural privacy. Comparisons across those boundaries instead measure what is saved or paid by changing a guarantee or disclosure surface.

7.1.1 Monolithic Design and Behavior

A credible baseline must remove avoidable circuit costs before any difference is attributed to decomposition. Chapter 4 compares four permutation mechanisms, rejects the quadratic pairwise check, and selects sorting as the canonical exact mechanism (Section 4.2.5). The refined construction then replaces the cost matrix's quadratic proof-system public surface with one Merkle root (Section 4.4).

After refinement, the complete relation remains one proving task. Its circuit and prover requirements grow with n, with backend padding making the progression stepped (Section 6.3.1, Section 6.3.2). Maximum Barretenberg-reported proving memory rises from 2.39 GiB at $n = 1000$ to 12.95 GiB when $n = 4000$, the largest successful monolithic checkpoint. The $n = 5000$ attempt exceeds the host's memory budget. Proof size remains 14,656 bytes and

the verifier still receives the two-value interface $(\textsc{root}, T)$. The result is a comparison-ready baseline within the fixed stack: its verifier interface remains succinct, but one prover still absorbs growing whole-route work and memory.

7.1.2 Advantages of Decomposition

Against this baseline, decomposition divides the route-level work among smaller tasks and adds recombination. The combined segment work remains close to the whole-route computation, so its possible advantages lie in reducing the largest task, extending the range feasible on fixed hardware, and exposing work that can be scheduled independently.

The external constructions exhibit two recombination regimes. Sort glue retains the global sort over all n nodes and becomes the limiting task: when $n = 4000$, both sort variants remain near 8.8 GiB for every measured K, below the 12.95-GiB monolithic baseline but fixed by the same glue floor. Product glue combines compact fingerprints and chain summaries and remains smaller than a segment, allowing the complete peak to fall to approximately 1.6 GiB at $K = 8$ (Figure 6.7). This lower profile depends on the computational product-coverage mechanism and is not an unconditional improvement over exact sorting (Section 5.2.3).

Recursion creates a third regime: increasing K shrinks the inner tasks but enlarges the outer circuit through additional constrained proof verifications. The lowest measured recursive peak at $n = 4000$ therefore occurs at $K = 4$, before the outer stage becomes dominant. Every composite construction also completes at $n = 5000$, beyond the successful monolithic range, although the sort variants already approach the host budget. Segmentation is consequently a resource-allocation parameter rather than a quantity to maximize.

Scheduling introduces a separate trade-off. On one host, every composite schedule at the shared $n = 4000$ checkpoint is at least as long as the monolithic one. External segment and glue tasks can run independently, whereas recursion retains a dependent outer stage. The distributed estimates exploit this distinction by adding comparable workers and are not hardware-

normalized speedups. The capacity requirement falls when the largest composite task stays below the monolithic proving task.

7.1.3 Recombination and Structural Privacy

Segmentation removes the global view that lets one circuit enforce the complete claim. Recombination must restore agreement on the matrix root and threshold, continuity across segment boundaries and cycle closure, aggregation of the partial costs and the threshold check, and coverage of every node (Section 5.1.3). The constructions share this contract but differ in how they represent its evidence, what crosses the segment boundary, and who checks it.

The plain constructions publish the summaries required for external recombination: node sets or testable fingerprints, together with endpoints and partial costs. Commitments hide the summary contents behind blinded digests while retaining $K + 1$ external proofs and reconciliation; committed product also retains the public whole-route challenge X. Recursive product instead supplies the inner proofs and their interfaces as private outer-circuit witness data, leaving the final verifier with one proof over $(\textsc{root}, T)$ and no public per-segment values.

These interfaces place the corresponding costs differently. Commitments add hiding and opening work while leaving proof checks and public-input reconciliation to the final verifier. Recursion absorbs those checks into a dependent outer proving stage whose constrained verification cost grows with K. Plain-to-committed comparisons measure the price of hiding within one coverage family, while external-to-recursive product measures the cost of removing the public segment interface. Sort-to-product comparisons also change the coverage representation and soundness properties. The recombination interface determines the attainable structural privacy and whether the final verifier, glue prover, or recursive outer prover enforces the contract.

Together, the profiles in Table 7.1 answer the leading question from Section 1.3. The monolithic architecture keeps the global relation in one growing proving task and gives the verifier one proof over a compact

Table 7.1: Architectural synthesis of the structural properties in Chapter 5 and the measurements in Chapter 6. Memory limiters describe the observed large-instance regime rather than universal asymptotes. Costs are directly comparable only under matched coverage and structural privacy.

Construction	Coverage and disclosure	Memory limiter	Prover schedule	Final verifier
monolithic sort	exact; no partition surface	whole-route circuit	one task	one proof; two inputs
plain sort	exact; public node sets, endpoints, and costs	glue	independent segments and glue	$K + 1$ proofs; reconciliation; payload grows with n
committed sort	exact; blinded segment digests	glue	independent tasks with hiding overhead	$K + 1$ proofs; digest reconciliation
plain product	computational; testable summaries	segment	independent segments and compact glue	$K + 1$ proofs; payload grows with K
committed product	computational; blinded digests and public X	segment	independent tasks and compact committed glue	$K + 1$ proofs; digest reconciliation
recursive product	computational; no final per-segment values	segment/outer crossover	parallel inner stage; dependent outer stage	one proof; two inputs

interface. Composite architectures create smaller, independently schedulable tasks, while recombination restores the conditions that no segment can prove alone. Whether this change is useful depends on the recombination task, coverage guarantee, disclosure boundary, available workers, and final-verifier capacity.

7.2 Beyond TSP Proofs

TSP exposes the semantic cut behind this study. Arc authentication and internal cost accumulation can be checked within a segment, while global coverage, continuity between segments, cycle closure, and the threshold depend on the complete route.

The same approach may apply when a relation admits a natural witness partition and each local proof can establish a meaningful claim from its part. Recombination must then enforce the conditions that cross the partition through explicit boundary evidence. Depending on the application, those

conditions may be an aggregate bound, uniqueness across a collection, continuity between steps, or agreement on shared state. A relation-aware design therefore asks where to cut the relation, what each piece can prove, which global properties the cut removes, what evidence must cross the boundary, and where the complete claim should be restored.

7.3 Limitations

The measurements characterize one TSP relation, one canonical satisfying instance for each n, and one consumer host and proving stack (Section 6.1.1). Circuit shape, gate count, proof topology, and dominant backend allocations depend principally on the structural parameters, but unconstrained witness generation can remain input-sensitive. Absolute times, memory requirements, padded-domain transitions, and capacity boundaries may change with the instance family, hardware, compiler, backend, or software release. The metric definitions and preprocessing and setup exclusions follow Section 6.1.2; verifier timing also excludes application-specific reconciliation (Section 6.7).

The evaluation covers selected values of n, $K \in \{2, 4, 8\}$, and equal-length segments. The refined monolithic baseline is comparison-ready, but it could include further improvements, and the five composite constructions are selected points rather than an exhaustive frontier. The study therefore identifies neither a universally optimal K nor a universally best proof architecture; unequal partitions, other coverage mechanisms, or other recombination strategies may shift the balance.

No networked cluster was deployed. Distributed completion times compose uncontended component runs and exclude communication, coordination, contention, failures, retries, and aggregate resource use. The model assumes workers controlled by one logical prover and does not address malicious workers or confidentiality of the witness material assigned to them.

The targeted cheating tests in Section 6.2 are implementation sanity checks, not a formal security proof, external audit, or verification of the complete compiler and backend toolchain. The security claims remain conditional on UltraHonk, Fiat–Shamir, Poseidon2, product-based coverage,

commitment binding and hiding, and authenticated verification keys. The implementation also lacks explicit domain separation among its Poseidon2 uses (Section 5.3.1).

7.4 Future Work

The central technical target is recombination. An exact-coverage representation that avoids the global sort could reduce the glue floor without adding the probabilistic error of product coverage. Folding, aggregation, or specialized recursive verification may also reduce the cost of absorbing local proofs or the number of checks exposed to the final verifier (Section 2.4). A related design could occupy the gap between external commitments and full recursion, hiding segment structure and reducing the final proof set without reproducing the complete outer-circuit cost.

The relation-aware method of Section 7.2 should be tested on another globally constrained relation, starting from a credible monolithic baseline and an explicit recombination contract. Repeating the comparison under another compiler, proof system, hardware configuration, or instance distribution would help distinguish effects of semantic decomposition from behavior specific to Noir, UltraHonk, and the benchmark host.

Deployment-aware decomposition should choose the segment count and sizes from worker memory, worker count, acceptable completion time, verifier load, and structural-privacy requirements, rather than fixing equal-length segments and $K \in \{2, 4, 8\}$. Unequal or adaptive partitions should then be evaluated on a networked deployment that measures communication, coordination, contention, failures, and aggregate resource use omitted from the schedules in Section 6.6.

Before deployment, the construction requires stronger assurance. The implementation should separate its Poseidon2 domains explicitly, formalize the security of the complete composition, extend adversarial testing, and undergo external review. It would also need authenticated verification keys and build artifacts, together with an explicit worker-trust and witness-confidentiality model.

7.5 Conclusions

For the TSP relation studied here, decomposition moves global coverage, continuity, cycle closure, and total-cost enforcement into recombination. Segment proofs establish authenticated local arcs and partial costs, but not the complete claim. Glue or recursion must restore that claim without recreating the whole-route bottleneck.

Recombination also sets the guarantee and disclosure boundary. Sort glue preserves exact coverage but retains a global sort. Product fingerprints yield the lower measured memory profile at the price of a negligible probabilistic soundness error. Plain external constructions expose segment summaries; commitments hide their contents but retain $K + 1$ proofs, a visible partition topology, and final-verifier reconciliation. Recursion keeps the inner proofs and interfaces private from the final verifier, which receives one proof over $(\textsc{root}, T)$, but adds a dependent outer proving stage.

With one sufficiently capable prover, the refined monolithic construction remains the simplest choice: it preserves exact coverage, hides the route and cost, and produces one proof. A memory-constrained prover can run a composite construction sequentially at a lower peak and longer completion time; multiple workers can schedule external tasks concurrently. Among the evaluated composites, exact coverage selects sort, whereas product offers the lower measured memory profile when its probabilistic guarantee is acceptable. For external recombination, commitments hide segment summaries but retain the proof collection. Recursion fits deployments that require one final proof and can support the dependent outer stage, especially when verification is constrained or widely replicated.

The value of decomposition depends on whether recombination restores the global claim within the deployment's memory, scheduling, verification, and disclosure constraints.

References

[1] L. Callegari, *ZK-TSP architectures*. [Online]. Available: https://github.com/lcalllegari1/zk-tsp

[2] J.-J. Quisquater *et al.*, "How to Explain Zero-Knowledge Protocols to Your Children," in *Advances in Cryptology — CRYPTO '89*, Springer New York, 1990, pp. 628–631. [Online]. Available: https://doi.org/10.1007/0-387-34805-0_60

[3] S. Goldwasser, S. Micali, and C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989, [Online]. Available: https://doi.org/10.1137/0218012

[4] E. Ben-Sasson *et al.*, "Zerocash: Decentralized Anonymous Payments from Bitcoin," in *Proceedings of the IEEE Symposium on Security & Privacy (Oakland)*, IEEE, 2014, pp. 459–474. [Online]. Available: https://doi.org/10.1109/SP.2014.36

[5] Self, *Self: Privacy-Preserving Identity Protocol*. [Online]. Available: https://github.com/selfxyz/self

[6] A. Stapelberg, "Opening up Zero-Knowledge Proof technology to promote privacy in age assurance," Jul. 03, 2025, *Google*. [Online]. Available: https://blog.google/innovation-and-ai/technology/safety-security/opening-up-zero-knowledge-proof-technology-to-promote-privacy-in-age-assurance/

[7] M. Babel and J. Sedlmeir, "Bringing data minimization to digital wallets at scale with general-purpose zero-knowledge proofs," 2023. [Online]. Available: https://doi.org/10.48550/arxiv.2301.00823

[8] S. Brands, *Rethinking Public Key Infrastructures and Digital Certificates*. The MIT Press, 2000. [Online]. Available: https://doi.org/10.7551/mitpress/5931.001.0001

[9] J. Camenisch and A. Lysyanskaya, "An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation," in *Lecture Notes in Computer Science*, Springer Berlin Heidelberg, 2001, pp. 93–118. [Online]. Available: https://doi.org/10.1007/3-540-44987-6_7

[10] J. Bruestle, P. Gafni, and RISC Zero Team, "RISC Zero zkVM: Scalable, Transparent Arguments of RISC-V Integrity," RISC Zero, 2023. [Online]. Available: https://dev.risczero.com/proof-system-in-detail.pdf

[11] U. Roy, "Introducing SP1: A Performant, 100% Open-Source, Contributor-Friendly zkVM," Feb. 14, 2024, *Succinct*. [Online]. Available: https://blog.succinct.xyz/introducing-sp1/

[12] A. Arun, S. Setty, and J. Thaler, "Jolt: SNARKs for Virtual Machines via Lookups," 2023, 2023/1217. [Online]. Available: https://eprint.iacr.org/2023/1217

[13] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again," in *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS)*, ACM, 2012, pp. 326–349. [Online]. Available: https://doi.org/10.1145/2090236.2090263

[14] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly Practical Verifiable Computation," in *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*, IEEE, 2013, pp. 238–252. [Online]. Available: https://doi.org/10.1109/SP.2013.47

[15] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, "SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge," in *Advances in Cryptology — CRYPTO 2013, Part II*, Springer, 2013, pp. 90–108. [Online]. Available: https://eprint.iacr.org/2013/507

[16] G. Brassard, D. Chaum, and C. Crépeau, "Minimum Disclosure Proofs of Knowledge," *Journal of Computer and System Sciences*, vol. 37, no. 2, pp. 156–189, 1988, [Online]. Available: https://doi.org/10.1016/0022-0000(88)90005-0

[17] W. Diffie and M. E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976, [Online]. Available: https://doi.org/10.1109/TIT.1976.1055638

[18] R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978, [Online]. Available: https://doi.org/10.1145/359340.359342

[19] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[20] G. Dantzig, R. Fulkerson, and S. Johnson, "Solution of a Large-Scale Traveling-Salesman Problem," *Journal of the Operations Research Society of America*, vol. 2, no. 4, pp. 393–410, 1954, [Online]. Available: https://doi.org/10.1287/opre.2.4.393

[21] O. Goldreich, S. Micali, and A. Wigderson, "Proofs that Yield Nothing But Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems," *Journal of the ACM*, vol. 38, no. 3, pp. 691–729, 1991, [Online]. Available: https://doi.org/10.1145/116825.116852

[22] M. Blum, "How to Prove a Theorem So No One Else Can Claim It," in *Proceedings of the International Congress of Mathematicians*, 1986, pp. 1444–1451. [Online]. Available: https://www.mathunion.org/fileadmin/ICM/Proceedings/ICM1986.2/ICM1986.2.ocr.pdf

[23] R. Igari, S. Odaka, Y. Komano, and T. Mizuki, "Efficient Physical ZKP Protocols for Hamiltonian Cycle Problem and Traveling Salesman Problem," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences (advance online publication)*, Mar. 2026, [Online]. Available: https://doi.org/10.1587/transfun.2025DMP0010

[24] H. Wu, W. Zheng, A. Chiesa, R. A. Popa, and I. Stoica, "DIZK: A Distributed Zero Knowledge Proof System," in *27th USENIX Security Symposium (USENIX Security 18)*, USENIX Association, 2018, pp. 675–692. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/wu

[25] T. Liu, T. Xie, J. Zhang, D. Song, and Y. Zhang, "Pianist: Scalable zkRollups via Fully Distributed Zero-Knowledge Proofs," in *Proceedings of the 2024 IEEE Symposium on Security and Privacy*, IEEE, 2024, pp. 1777–1793. [Online]. Available: https://doi.org/10.1109/SP54263.2024.00035

[26] T. Xie *et al.*, "zkBridge: Trustless Cross-chain Bridges Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2022, pp. 3003–3017. [Online]. Available: https://doi.org/10.1145/3548606.3560652

[27] W. Li, Z. Zhang, Y. Li, P. Zhu, C. Hong, and J. Liu, "Soloist: Distributed SNARK for R1CS with Constant Proof Size," in *Advances in Cryptology — EUROCRYPT 2026, Part VII*, Springer, 2026, pp. 63–94. [Online]. Available: https://doi.org/10.1007/978-3-032-25336-1_3

[28] W. Wang, F. Shi, D. Vilardell, and F. Zhang, "Cirrus: Performant and Accountable Distributed SNARK," in *Proceedings of the 2026 Network and Distributed System Security Symposium*, The Internet Society, 2026. [Online]. Available: https://doi.org/10.14722/ndss.2026.240668

[29] Y. Yu *et al.*, "HyperFond: A Transparent and Post-Quantum Distributed SNARK with Polylogarithmic Communication," in *Proceedings of the 21st ACM ASIA Conference on Computer and Communications Security (ASIA CCS 2026)*, ACM, 2026, pp. 869–885. [Online]. Available: https://doi.org/10.1145/3779208.3805973

[30] M. Rosenberg, T. Mopuri, H. Hafezi, I. Miers, and P. Mishra, "Hekaton: Horizontally-Scalable zkSNARKs via Proof Aggregation," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2024, pp. 929–940. [Online]. Available: https://eprint.iacr.org/2024/1208

[31] A. Baweja, P. Mishra, T. Mopuri, K. Newatia, and S. Wang, "Scribe: Low-memory SNARKs via Read-Write Streaming," in *35th USENIX Security Symposium (USENIX Security 26)*, USENIX Association, 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/baweja

[32] C. Pappas and D. Papadopoulos, "Sparrow: Space-Efficient zkSNARK for Data-Parallel Circuits and Applications to Zero-Knowledge Decision Trees," 2024, 2024/1631. [Online]. Available: https://eprint.iacr.org/2024/1631

[33] A. Ozdemir and D. Boneh, "Experimenting with Collaborative zk-SNARKs: Zero-Knowledge Proofs for Distributed Secrets," in *31st USENIX Security Symposium (USENIX Security 22)*, USENIX Association, 2022, pp. 4291–4308. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/ozdemir

[34] A. Chiesa and E. Tromer, "Proof-Carrying Data and Hearsay Arguments from Signature Cards," in *Proceedings of the 1st Symposium on Innovations in Computer Science (ICS 2010)*, Tsinghua University Press, 2010, pp. 310–331. [Online]. Available: https://people.eecs.berkeley.edu/~alexch/docs/CT10.pdf

[35] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC 2013)*, ACM, 2013, pp. 111–120. [Online]. Available: https://doi.org/10.1145/2488608.2488623

[36] A. Chiesa, E. Tromer, and M. Virza, "Cluster Computing in Zero Knowledge," in *Advances in Cryptology — EUROCRYPT 2015, Part II*, Springer, 2015, pp. 371–403. [Online]. Available: https://doi.org/10.1007/978-3-662-46803-6_13

[37] S. Bowe, J. Grigg, and D. Hopwood, "Halo: Recursive Proof Composition without a Trusted Setup," 2019, 2019/1021. [Online]. Available: https://eprint.iacr.org/2019/1021

[38] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes," in *Advances in Cryptology — CRYPTO 2022*, Springer, 2022, pp. 359–388. [Online]. Available: https://eprint.iacr.org/2021/370

[39] W. Nguyen, T. Datta, B. Chen, N. Tyagi, and D. Boneh, "Mangrove: A Scalable Framework for Folding-based SNARKs," in *Advances in Cryptology — CRYPTO 2024, Part X*, Springer, 2024, pp. 308–344. [Online]. Available: https://eprint.iacr.org/2024/416

[40] J. Groth, "On the Size of Pairing-Based Non-interactive Arguments," in *Advances in Cryptology — EUROCRYPT 2016, Part II*, Springer, 2016, pp. 305–326. [Online]. Available: https://doi.org/10.1007/978-3-662-49896-5_11

[41] N. Gailly, M. Maller, and A. Nitulescu, "SnarkPack: Practical SNARK Aggregation," in *Financial Cryptography and Data Security 2022*, Springer, 2022, pp. 203–229. [Online]. Available: https://eprint.iacr.org/2021/529

[42] M. Campanelli, D. Fiore, and A. Querol, "LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2019, pp. 2075–2092. [Online]. Available: https://eprint.iacr.org/2019/142

[43] J. Zhu, S. Bai, X. Hu, H. Zhou, and G. Yang, "LaPathZK: Parallel Threshold Path zk-SNARK Leveraging High-Dimensional Lattices," *Information Sciences*, vol. 755, Art. no. 123761, 2026, [Online]. Available: https://doi.org/10.1016/j.ins.2026.123761

[44] M. Bellare and O. Goldreich, "On Defining Proofs of Knowledge," in *Advances in Cryptology — CRYPTO '92, Lecture Notes in Computer Science, vol. 740*, Springer, 1992, pp. 390–420. [Online]. Available: https://doi.org/10.1007/3-540-48071-4_28

[45] Noir Contributors, *Noir, version 1.0.0-beta.20*. (2026). [Online]. Available: https://github.com/noir-lang/noir/releases/tag/v1.0.0-beta.20

[46] Aztec Labs, *Barretenberg, version 5.0.0-nightly.20260324*. (2026). [Online]. Available: https://github.com/AztecProtocol/aztec-packages/tree/v5.0.0-nightly.20260324/barretenberg

[47] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," 2019, 2019/953. [Online]. Available: https://eprint.iacr.org/2019/953

[48] M. Blum, W. Evans, P. Gemmell, S. Kannan, and M. Naor, "Checking the Correctness of Memories," in *Proceedings of the 32nd Annual Symposium on Foundations of Computer Science*, IEEE Computer Society, 1991, pp. 90–99. [Online]. Available: https://doi.org/10.1109/SFCS.1991.185352

[49] A. Gabizon and Z. J. Williamson, "plookup: A Simplified Polynomial Protocol for Lookup Tables," 2020, 2020/315. [Online]. Available: https://eprint.iacr.org/2020/315

[50] P. Rogaway and T. Shrimpton, "Cryptographic Hash-Function Basics: Definitions, Implications, and Separations for Preimage Resistance, Second-Preimage Resistance, and Collision Resistance," in *Fast Software Encryption — FSE 2004, Lecture Notes in Computer Science, vol. 3017*, Springer, 2004, pp. 371–388. [Online]. Available: https://iacr.org/archive/fse2004/30170373/30170373.pdf

[51] M. Bellare and P. Rogaway, "Random Oracles Are Practical: A Paradigm for Designing Efficient Protocols," in *Proceedings of the 1st ACM Conference on Computer and Communications Security*, Association for Computing Machinery, 1993, pp. 62–73. [Online]. Available: https://doi.org/10.1145/168588.168596

[52] L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: A Faster Version of the Poseidon Hash Function," in *Progress in Cryptology — AFRICACRYPT 2023*, Springer, 2023. [Online]. Available: https://eprint.iacr.org/2023/323

[53] M. Naor, "Bit Commitment Using Pseudorandomness," *Journal of Cryptology*, vol. 4, no. 2, pp. 151–158, 1991, [Online]. Available: https://doi.org/10.1007/BF00196774

[54] S. Halevi and S. Micali, "Practical and Provably-Secure Commitment Schemes from Collision-Free Hashing," in *Advances in Cryptology — CRYPTO '96, Lecture Notes in Computer Science, vol. 1109*, Springer, 1996, pp. 201–215. [Online]. Available: https://people.csail.mit.edu/shaih/pubs/hm96.pdf

[55] T. P. Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing," in *Advances in Cryptology — CRYPTO '91, Lecture Notes in Computer Science, vol. 576*, Springer, 1991, pp. 129–140. [Online]. Available: https://doi.org/10.1007/3-540-46766-1_9

[56] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-Size Commitments to Polynomials and Their Applications," in *Advances in Cryptology — ASIACRYPT 2010, Lecture Notes in Computer Science, vol. 6477*, Springer, 2010, pp. 177–194. [Online]. Available: https://doi.org/10.1007/978-3-642-17373-8_11

[57] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87, Lecture Notes in Computer Science, vol. 293*, Springer, 1987, pp. 369–378. [Online]. Available: https://doi.org/10.1007/3-540-48184-2_32

[58] J. T. Schwartz, "Fast Probabilistic Algorithms for Verification of Polynomial Identities," *Journal of the ACM*, vol. 27, no. 4, pp. 701–717, 1980, [Online]. Available: https://doi.org/10.1145/322217.322225

[59] R. E. Zippel, "Probabilistic Algorithms for Sparse Polynomials," in *Symbolic and Algebraic Computation — EUROSAM '79, Lecture Notes in Computer Science, vol. 72*, Springer, 1979, pp. 216–226. [Online]. Available: https://doi.org/10.1007/3-540-09519-5_73

[60] A. Fiat and A. Shamir, "How to Prove Yourself: Practical Solutions to Identification and Signature Problems," in *Advances in Cryptology — CRYPTO '86*, Springer Berlin Heidelberg, 1986, pp. 186–194. [Online]. Available: https://doi.org/10.1007/3-540-47721-7_12

[61] J. Groth, M. Kohlweiss, M. Maller, S. Meiklejohn, and I. Miers, "Updatable and Universal Common Reference Strings with Applications to zk-SNARKs," in *Advances in Cryptology -- CRYPTO 2018*, Springer, 2018, pp. 698–728. [Online]. Available: https://doi.org/10.1007/978-3-319-96878-0_24

[62] J. Kuszmaul, "Verkle Trees," in *MIT PRIMES Conference*, MIT PRIMES, 2018. [Online]. Available: https://math.mit.edu/research/highschool/primes/materials/2018/Kuszmaul.pdf

[63] A. Zapico, V. Buterin, D. Khovratovich, M. Maller, A. Nitulescu, and M. Simkin, "Caulk: Lookup Arguments in Sublinear Time," 2022, 2022/621. [Online]. Available: https://eprint.iacr.org/2022/621

[64] L. Eagen, D. Fiore, and A. Gabizon, "cq: Cached Quotients for Fast Lookups," 2022, 2022/1763. [Online]. Available: https://eprint.iacr.org/2022/1763

[65] S. Setty, J. Thaler, and R. S. Wahby, "Unlocking the Lookup Singularity with Lasso," 2023, 2023/1216. [Online]. Available: https://eprint.iacr.org/2023/1216